Iterated Dynamics

Table of Contents

New Features in 1.5 . . . 5
Introduction . . . 5
  Conditions on Use . . . 6
Iterated Dynamics Commands . . . 6
  Getting Started . . . 7
  Plotting Commands . . . 7
  Zoom Box Commands . . . 9
  Color Cycling Commands . . . 11
  Palette Editing Commands . . . 12
  Image Save/Restore Commands . . . 15
  Parameter Save/Restore Commands . . . 16
  "3D" Commands . . . 18
  Interrupting and Resuming . . . 18
  Orbits Window . . . 19
  View Window . . . 19
  Video Mode Function Keys . . . 20
  Browse Commands . . . 21
  Evolver Commands . . . 22
  RDS Commands . . . 23
  Hints . . . 23
  Id on Linux and macOS . . . 23
Fractal Types . . . 24
  The Mandelbrot Set . . . 24
  Julia Sets . . . 26
  Julia Toggle Spacebar Commands . . . 27
  Inverse Julias . . . 27
  Ant Automaton . . . 29
  Barnsley IFS Fractals . . . 30
  Barnsley Mandelbrot/Julia Sets . . . 31
  Bifurcation . . . 32
  Burning Ship . . . 34
  Cellular Automata . . . 35
  Circle . . . 36
  Diffusion Limited Aggregation . . . 37
  DivideBrot5 . . . 37
  Dynamic System . . . 38

Escher-Like Julia Sets . . . 39
fn||fn Fractals . . . 40
Formula . . . 41
Frothy Basins . . . 45
Gingerbreadman . . . 46
Halley . . . 46
Henon Attractors . . . 47
HyperComplex . . . 47
Icon . . . 48
Julibrots . . . 49
Kam Torus . . . 50
L-Systems . . . 51
Lambda Sets . . . 53
Lambdafn . . . 53
Latoocarfian . . . 54
Lorenz Attractors . . . 55
Lyapunov Fractals . . . 56
Magnetic Fractals . . . 57
Mandelbrot Mix 4 . . . 59
Mandelcloud . . . 59
Mandelfn . . . 60
Mandellambda Sets . . . 60
Martin Attractors . . . 61
Newton Domains of Attraction . . . 62
Newton . . . 63
Complex Newton . . . 63
Orbit Fractals . . . 63
Peterson Variations . . . 64
Phoenix . . . 65
Pickover Attractors . . . 65
Pickover Mandelbrot/Julia Types . . . 66
Pickover Popcorn . . . 67
Plasma Clouds . . . 68
Quartic Mandelbrot/Julia . . . 69
Quaternion . . . 69
Rossler Attractors . . . 70
Scott Taylor / Lee Skinner Variations . . . 71
Sierpinski Gasket . . . 72
Test Stub . . . 72
Unity . . . 73
Volterra-Lotka Fractals . . . 73

Doodads, Bells, and Whistles . . . 75
Drawing Method . . . 75
Color Maps . . . 77
Autokey Mode . . . 77
Distance Estimator Method . . . 80
Inversion . . . 81
Decomposition . . . 82
Logarithmic Palettes and Color Ranges . . . 82
Biomorphs . . . 83
Continuous Potential . . . 84
Starfields . . . 85
Bailout Test . . . 86
Parameter Explorer/Evolver . . . 87
Random Dot Stereograms (RDS) . . . 90
Freestyle mode tutorial . . . 92
"3D" Images . . . 92
3D Overview . . . 92
3D Mode Selection . . . 93
Select Fill Type Screen . . . 96
Stereo 3D Viewing . . . 97
Rectangular Coordinate Transformation . . . 98
3D Color Parameters . . . 98
Light Source Parameters . . . 99
Spherical Projection . . . 100
3D Overlay Mode . . . 100
Making Terrains . . . 101
Making 3D Slides . . . 102
Interfacing with Ray Tracing Programs . . . 102
Command Line Parameters, Parameter Files, Batch Mode . . . 103
Introduction to Parameters . . . 103
Using the Command Line . . . 104
Libraries . . . 104
Setting Defaults (sstools.ini File) . . . 105
Parameter Files and the `@` Command . . . 106
General Parameter Syntax . . . 107
Startup Parameters . . . 107
Calculation Mode Parameters . . . 109
Fractal Type Parameters . . . 110
Image Calculation Parameters . . . 110
Color Parameters . . . 112
Doodad Parameters . . . 115

File Parameters . . . 115
Video Parameters . . . 117
Sound Parameters . . . 117
Sound Controls . . . 118
Advanced Sound Controls . . . 119
Envelopes . . . 121
3D Parameters . . . 122
Batch Mode . . . 123
Browser Parameters . . . 124
Passes Parameters . . . 125
Screen Coordinates Screen . . . 126
Image Coordinates Screen . . . 126
Hardware Support . . . 126
Notes on Video Modes, "Standard" and Otherwise . . . 126
"Disk-Video" Modes . . . 127
Customized Video Modes, id.cfg . . . 127
Common Problems . . . 128
Fractals and the PC . . . 128
A Little History . . . 128
Before Mandelbrot . . . 128
Who Is This Guy, Anyway? . . . 129
A Little Code . . . 130
Periodicity Logic . . . 130
Arbitrary Precision and Deep Zooming . . . 131
Perturbation Algorithm . . . 132
The Id "Fractal Engine" Architecture . . . 132
Mathematics of the Fractal Types . . . 134
Summary of Fractal Types . . . 134
Inside=bof60|bof61|zmag|fmod|period|atan . . . 142
Inside=epscross|startrail . . . 143
Finite Attractors . . . 143
Trig Identities . . . 145
Quaternion and Hypercomplex Algebra . . . 146
Stone Soup With Pixels: The Authors . . . 148
The Stone Soup Story . . . 148
A Word About the Authors . . . 149
Main Author of Iterated Dynamics . . . 149
Main Authors of Fractint . . . 151
Distribution of Iterated Dynamics . . . 152
Contacting the Author . . . 153
GIF Save File Format . . . 153

Other Fractal Programs 153
Bibliography 153
Revision History 155

New Features in 1.5

The source code for Iterated Dynamics lives on github: https://github.com/LegalizeAdulthood/iterated-dynamics

Found a bug? File an issue on our github project! https://github.com/LegalizeAdulthood/iterated-dynamics/issues

Version 1.5 is an interim release on the way to version 2.0.

New Features in Version 1.5

- The Tab screen now reports the number of reference orbits used for perturbation rendering.

Bugs Fixed in Version 1.5

- Cross-hair cursors now repaint while tracking mouse movement in the palette editor and Julia point picker.
- The Orbits Window now toggles on completed images and remains shown after backing out of menus unless it is toggled off.
- Iterated Dynamics is now built for FreeBSD, OpenBSD and NetBSD (Fixes #439, #444, #445)

Code Cleanup in Version 1.5

- Runtime fractal dispatch now uses explicit dispatch state for selected function pointers, leaving the FractalSpecific table as constant metadata.
- Replaced platform-sensitive bifurcation image gold tests with direct sequence and column behavior tests.
- Unused debug flags were removed and documentation updated.
- Image regression tests added for popcorn types.

Introduction

Iterated Dynamics, or Id for short, plots and manipulates images of "objects" — actually, sets of mathematical points — that have fractal dimension. See Fractals and the PC for some historical and mathematical background on fractal geometry, a discipline named and popularized by mathematician Benoit Mandelbrot. For now, these sets of points have three important properties:

1) They are generated by relatively simple calculations repeated over and over, feeding the results of each step back into the next — something computers can do very rapidly.

2) They are, quite literally, infinitely complex: they reveal more and more detail without limit as

you plot smaller and smaller areas. Id lets you "zoom in" by positioning a small box and hitting `Enter` to redraw the boxed area at full-screen size; its maximum linear "magnification" is over a trillionfold.

3) They can be astonishingly beautiful, especially when assigning colors to selected points, and to "animate" the images by quickly shifting those color assignments.

For a demonstration of some of Id's features, run the demonstration file included with this release (demo.bat) by typing "demo" at the command prompt. You can stop the demonstration at any time by pressing `Esc`.

The name Iterated Dynamics was chosen because the program generates its images by iterating some kind of dynamical system exhibiting non-linear or chaotic behavior.

Like many open source projects, Iterated Dynamics is an experiment in collaboration. While being an offshoot of its ancestor, Fractint, Iterated Dynamics in no way changes the spirit of collaboration started with that program by its original author Bert Tyler.

See The Stone Soup Story and A Word About the Authors for information about the authors, and see Contacting the Author for how to contribute your own ideas and code.

Conditions on Use

Iterated Dynamics is open source software. The copyright is retained by the primary author, Richard Thomson. Legacy code is copyright the Stone Soup Group.

Iterated Dynamics may be freely copied and distributed in unmodified form but may not be sold. Iterated Dynamics may be used personally or in a business - if you can do your job better by using Iterated Dynamics, or using images from it, that's great! It may not be given away with commercial products without explicit permission from Richard Thomson.

There is no warranty of Iterated Dynamics's suitability for any purpose, nor any acceptance of liability, express or implied.

Contribution policy

Don't want money. Got money. Want admiration.

Source code for Iterated Dynamics is freely available on github at https://github.com/LegalizeAdulthood/iterated-dynamics

See the file LICENSE.txt for details on distributing the program.

Iterated Dynamics Commands

Getting Started

To start the program, locate the Id executable in the installation directory. The program displays an initial "credits" screen. If Id doesn't start properly, please see Common Problems.

Hitting `Enter` gets you from the initial screen to the main menu. You can select options from the menu by moving the highlight with the arrow keys (`Left`, `Right`, `Up`, `Down`) and pressing `Enter`, or you can enter commands directly.

As soon as you select a video mode, Id begins drawing an image - the "full" Mandelbrot set if you haven't selected another fractal type.

For a quick start, after starting Id try one of the following:

- 800x600 small size image: `F6`
- 1024x768 medium size image: `F7`
- 1200x900 large size image: `F8`

After the initial Mandelbrot image has been displayed, try zooming into it (see Zoom Box Commands) and color cycling (see Color Cycling Commands). Once you're comfortable with these basics, start exploring other functions from the main menu.

Help is available from the menu and at most other points in Id by pressing the `F1` key.

At any time, you can hit a command key to select a function. You do not need to wait for a calculation to finish, nor do you have to return to the main menu.

When entering commands, note that for the "typewriter" keys, upper and lower case are equivalent in most situations, e.g. `B` and `Shift` + `B` have the same result. When an upper case letter is required, such as telling Id to delete the current file in browse mode, it will be described as `Shift` + `D`.

Many commands and parameters can be passed to Id as command-line arguments or read from a configuration file; see Introduction to Parameters for details.

Plotting Commands

Function keys & various combinations are used to select a video mode and redraw the screen. For a quick start try one of the following:

- 800x600 small size image: `F6`
- 1024x768 medium size image: `F7`
- 1200x900 large size image: `F8`

`F1` Display a help screen. The function keys available in help mode are displayed at the bottom of the help screen.

`M` or `Esc` Return from a displayed image to the main menu.

`Esc` From the main menu, `Esc` is used to exit from Id.

`Delete` Same as choosing "select video mode" from the main menu. Goes to the "select video mode" screen. See Video Mode Function Keys.

`H` Redraw the previous image in the circular history buffer, revisiting fractals you previously generated this session in reverse order. Id saves the last ten images worth of information including fractal type, coordinates, colors, and all options. Image information is saved only when some item changes. After ten images the circular buffer wraps around and earlier information is overwritten. You can set image capacity of the history feature using the maxhistory=<nnn> command.

`Ctrl` + `H` Redraw the next image in the circular history buffer. Use this to return to images you passed by when using `H`.

`Tab` Display the current fractal type, parameters, video mode, screen or (if displayed) zoom-box coordinates, maximum iteration count, and other information useful in keeping track of where you are. The Tab function is non-destructive - if you press it while in the midst of generating an image, you will continue generating it when you return. The Tab function tells you if your image is still being generated or has finished - a handy feature for those overnight, 1024x768 resolution fractal images. If the image is incomplete, it also tells you whether it can be interrupted and resumed. (Any function other than `Tab` and `F1` counts as an "interrupt".)

For perturbation images, the Tab screen also shows how many reference orbits have been selected.

The Tab screen also includes a pixel-counting function, which will count the number of pixels colored in the inside color. This gives an estimate of the area of the fractal. Note that the inside color must be different from the outside color(s) for this to work; inside=0 is a good choice.

`T` Select a fractal type. Move the cursor to your choice (or type the first few letters of its name) and hit `Enter`. Next you will be prompted for any parameters used by the selected type - hit `Enter` for the defaults. See Fractal Types for a list of supported types. `X` Select a number of eXtended options. Brings up a full-screen menu of options, any of which you can change at will. These options are: "passes=" - see Drawing Method Floating point toggle - see <F> key description below "maxiter=" - see Image Calculation Parameters "inside=" and "outside=" - see Color Parameters "savename=" filename - see File Parameters "overwrite=" option - see File Parameters "sound=" option - see Sound Parameters "logmap=" - see Logarithmic Palettes and Color Ranges "biomorph=" - see Biomorphs "decomp=" - see Decomposition "fillcolor=" - see Drawing Method

`Y` More options which we couldn't fit under the `X` command: "finattract=" - see Finite Attractors "potential=" parameters - see Continuous Potential "invert=" parameters - see Inversion "distest=" parameters - see Distance Estimator Method "cyclerange=" - see Color Cycling Commands

`P` Options that apply to the Passes feature: "periodicity=" - see Periodicity Logic "orbitdelay=" - see Passes Parameters "orbitinterval=" - see Passes Parameters "screencoords=" - see Passes Parameters "orbitdrawmode=" - see Passes Parameters

`Z` Modify the parameters specific to the currently selected fractal type. This command lets you modify the parameters which are requested when you select a new fractal type with the `T` command, without having to repeat that selection. You can enter "e" or "p" in column one of the input fields to get the numbers e and pi (2.71828... and 3.14159...). From the fractal parameters screen, you can press `F6` to bring up a sub parameter screen for the coordinates of the image's corners. With selected fractal types, `Z` allows you to change the Bailout Test.

`+` or `-` Switch to color-cycling mode and begin cycling the palette by shifting each color to the next "contour." See Color Cycling Commands.

`C` Switch to color-cycling mode but do not start cycling. The normally black "overscan" border of the screen changes to white. See Color Cycling Commands.

`E` Enter Palette-Editing Mode. See Palette Editing Commands.

`Space` Toggle between Mandelbrot set images and their corresponding Julia-set images. Read the notes in Fractal Types, Julia Sets before trying this option if you want to see anything interesting.

`J` Toggle between Julia escape time fractal and the Inverse Julia orbit fractal. See Inverse Julias

`Enter` Enter is used to resume calculation after a pause. It is only necessary to do this when there is a message on the screen waiting to be acknowledged, such as the message shown after you save an image to disk.

`I` Modify 3D transformation parameters used with 3D fractal types such as "Lorenz3D" and 3D "IFS" definitions, including the selection of "funny glasses" red/blue 3D.

`A` Convert the current image into a fractal 'starfield'. See Starfields.

`Ctrl` + `A` Unleash an image-eating ant automaton on current image. See Ant Automaton.

`Ctrl` + `S` (or `K`) Convert the current image into a Random Dot Stereogram (RDS). See Random Dot Stereograms (RDS).

`O` (the letter, not the number) If pressed while an image is being generated, toggles the display of intermediate results — the "orbits" Id uses as it calculates values for each point. Slows the display a bit, but shows you how clever the program is behind the scenes. (See "A Little Code" in Fractals and the PC.)

`D` Shell to a command prompt. Return to Id by exiting your command shell, usually by entering the command "exit".

`Insert` Restart at the "credits" screen and reset most parameters to their initial state. Parameters which are not reset are: savename, lightname, video, startup filename. This also resets compatibility checks to the current Id version. In parameter files, a bare reset selects Fractint 17.30 compatibility; use an explicit reset= value to preserve behavior for old parameter sets.

`L` Enter Browsing Mode. See Browse Commands.

`Ctrl` + `E` Enter Explorer/Evolver Mode. See Evolver Commands.

Zoom Box Commands

Zoom Box functions can be invoked while an image is being generated or when it has been completely drawn. Zooming is supported for most fractal types, but not all.

The general approach to using the zoom box is: Frame an area using the keys described below, then `Enter` to expand what's in the frame to fill the whole screen (zoom in); or `Ctrl` + `Enter` to shrink the

current image into the framed area (zoom out). With a mouse, double-click the primary button to zoom in, double click the secondary button to zoom out.

`PageUp`, `PageDown` Use `PageUp` to initially bring up the zoom box. It starts at full screen size. Subsequent use of these keys makes the zoom box smaller or larger. Using `PageDown` to enlarge the zoom box when it is already at maximum size removes the zoom box from the display. Moving the mouse away from you or toward you while holding the primary button down performs the same functions as these keys.

Using the "arrow" keys (`Left`, `Right`, `Up`, `Down`) or moving the mouse without holding any buttons down, moves the zoom box.

Holding `Ctrl` while pressing arrow keys moves the box 5 times faster.

Panning: If you move a fullsize zoombox and don't change anything else before performing the zoom, Id just moves what's already on the screen and then fills in the new edges, to reduce drawing time. This feature applies to most fractal types but not all. A side effect is that while an image is incomplete, a full size zoom box moves in steps larger than one pixel. Id keeps the box on multiple pixel boundaries, to make panning possible. As a multi-pass (e.g. solid guessing) image approaches completion, the zoom box can move in smaller increments.

In addition to resizing the zoom box and moving it around, you can do some rather warped things with it. If you're a new Id user, we recommend skipping the rest of the zoom box functions for now and coming back to them when you're comfortable with the basic zoom box functions.

`Ctrl` + `-`, `Ctrl` + `+`, `Ctrl` + `Keypad-`, `Ctrl` + `Keypad+` Holding `Ctrl` and pressing the `+` or `-` keys, or the numeric keypad's `+` or `-` keys rotates the zoom box. Moving the mouse left or right while holding the secondary button down performs the same function.

`Ctrl` + `PageUp`, `Ctrl` + `PageDown` These commands change the zoom box's "aspect ratio", stretching or shrinking it vertically. Moving the mouse away from you or toward you while holding both buttons (or the middle button on a 3-button mouse) down performs the same function. There are no commands to directly stretch or shrink the zoom box horizontally - the same effect can be achieved by combining vertical stretching and resizing.

`Ctrl` + `Home`, `Ctrl` + `End` These commands "skew" the zoom box, moving the top and bottom edges in opposite directions. Moving the mouse left or right while holding both buttons (or the middle button on a 3-button mouse) down performs the same function. There are no commands to directly skew the left and right edges - the same effect can be achieved by using these functions combined with rotation.

`Ctrl` + `Insert`, `Ctrl` + `Delete` These commands change the zoom box color. This is useful when you're having trouble seeing the zoom box against the colors around it. Moving the mouse away from you or toward you while holding the secondary button down performs the same function.

You may find it difficult to figure out what combination of size, position rotation, stretch, and skew to use to get a particular result. (We do.)

A good way to get a feel for all these functions is to play with the Gingerbreadman fractal type. Gingerbreadman's shape makes it easy to see what you're doing to him. A warning though:

Gingerbreadman will run forever, he's never quite done! So, pre-empt with your next zoom when he's baked enough.

If you accidentally change your zoom box shape or rotate and forget which way is up, just use `PageDown` to make it bigger until it disappears, then `PageUp` to get a fresh one. With a mouse, after removing the old zoom box from the display release and re-press the primary button for a fresh one.

If your video mode does not have a 4:3 "aspect ratio" (i.e. if the visible display area on it is not 1.333 times as wide as it is high), rotating and zooming will have some odd effects - angles will change, including the zoom box's shape itself, circles (if you are so lucky as to see any with a non-standard aspect ratio) become non-circular, and so on.

Zooming is not implemented for the plasma and diffusion fractal types, nor for overlayed and 3D images. A few fractal types support zooming but do not support rotation and skewing - nothing happens when you try it.

Color Cycling Commands

Color-cycling mode is entered with the `C`, `+`, or `-` keys from an image, or with the `C` key from Palette-Editing mode.

You can also enter color-cycling while using a disk-video mode, to load or save a palette - other functions are not supported in disk-video.

Color cycling applies to the color numbers selected by the "cyclerange=" command line parameter (also changeable via the `Y` options screen and via the palette editor). By default, color numbers 1 to 255 inclusive are cycled. On some images you might want to set "inside=0" (`X` options or command line parameter) to exclude the "lake" from color cycling.

When you are in color-cycling mode, you will either see the screen colors cycling, or will see a white "overscan" border when paused, as a reminder that you are still in this mode. The keyboard commands available once you've entered color-cycling. are described below.

`F1` Bring up a help screen with commands specific to color cycling mode.

`Esc` Leave color-cycling mode.

`Home` Restore original palette.

`+` or `-` Begin cycling the palette by shifting each color to the next "contour." `+` cycles the colors in one direction, `-` in the other.

`<` or `>` Force a color-cycling pause, disable random colorizing, and single-step through a one color-cycle. For "fine-tuning" your image colors.

`Up`/`Down` Increase/decrease the cycling speed.

`F2` through `F10` Switches from simple rotation to color selection using randomly generated color bands of short (`F2`) to long (`F10`) duration.

`1` through `9` Causes the screen to be updated every 'n' color cycles (the default is 1). Handy for slower computers.

`Enter` Randomly selects a function key (`F2` through `F10`) and then updates ALL the screen colors prior to displaying them for instant, random colors. Hit this over and over again (we do).

`Space` Pause cycling with white overscan area. Cycling restarts with any command key (including another spacebar).

`Shift` + `F1` ... `Shift` + `F10` Pause cycling and reset the palette to a preset two color "straight" assignment, such as a spread from black to white

`Ctrl` + `F1` ... `Ctrl` + `F10` Pause & set a 2-color cyclical assignment, e.g. red→yellow→red.

`Alt` + `F1` ... `Alt` + `F10` Pause & set a 3-color cyclical assignment, e.g. green→white→blue.

`Shift` + `R`, `Shift` + `G`, `Shift` + `B` Pause and increase the red, green, or blue component of all colors by a small amount. Note the case distinction of this vs:

`R`, `G`, `B` Pause and decrease the red, green, or blue component of all colors by a small amount.

`D` or `A` Pause and load an external color map from the files default.map or altern.map, supplied with the program.

`L` Pause and load an external color map (.map file). Several .map files are supplied with Id. See Color Maps.

`S` Pause, prompt for a filename, and save the current color palette to the named file (.map assumed). See Color Maps.

Palette Editing Commands

Palette-editing mode provides a number of tools for modifying the colors in an image. Many thanks to Ethan Nagel for creating the palette editor.

Use the `E` key to enter palette-editing mode from a displayed image or from the main menu.

When this mode is entered, an empty palette frame is displayed. You can use the arrow keys to position the frame outline, and `PageUp` and `PageDown` to change its size. (The upper and lower limits on the size depend on the current video mode.) When the frame is positioned where you want it, hit `Enter` to display the current palette in the frame.

Note that the palette frame shows R(ed) G(reen) and B(lue) values for two color registers at the top. The active color register has a solid frame, the inactive register's frame is dotted. Within the active register, the active color component is framed.

With a video mode of 640x400 or higher, a status area appears between the two color registers. This status area shows:

```
nnn  = color number at the cursor location
```

```
A    = Auto mode
X, Y = exclusion modes
F    = freesyle mode
T    = stripe mode is waiting for #
```

Using the commands described below, you can assign particular colors to the registers and manipulate them. Note that at any given time there are two colors "X"d - these are pre-empted by the editor to display the palette frame. They can be edited but the results won't be visible. You can change which two colors are borrowed ("X"d out) by using the `V` command.

Once the palette frame is displayed and filled in, the following commands are available:

`F1` Bring up a help screen with commands specific to palette-editing mode.

`Esc` Leave palette-editing mode

`H` Hide the palette frame to see full image; the cross-hair remains visible and all functions remain enabled; hit `H` again to restore the palette display.

`Left`, `Right`, `Up`, `Down` Move the cross-hair cursor around. In 'auto' mode (the default) the color under the center of the cross-hair is automatically assigned to the active color register. Control-arrow keys move the cross-hair faster. A mouse can also be used to move around.

`R`, `G`, `B` Select the Red, Green, or Blue component of the active color register for subsequent commands

`Insert`, `Delete` Select previous or next color component in active register

`+`, `-` Increase or decrease the active color component value by 1. Numeric keypad `+` and `-` keys do the same.

`PageUp`, `PageDown` Increase or decrease the active color component value by 5; Moving the mouse up/down with primary button held is the same.

`0`, `1`, `2`, `3`, `4`, `5`, `6` Set the active color component's value to 0 10 20 ... 60

`Space` Select the other color register as the active one. In the default 'auto' mode this results in the now-inactive register being set to remember the color under the cursor, and the now-active register changing from whatever it had previously remembered to now follow the color.

`,` (comma), `.` (period) Rotate the palette one step. By default colors 1 through 255 inclusive are rotated. This range can be over-ridden with the "cyclerange" parameter, the `Y` options screen, or the `O` command described below.

`<`, `>` Rotate the palette continuously (until next keystroke)

`O` (oh, not zero) Set the color cycling range to the range of colors currently defined by the color registers.

`C` Enter Color-Cycling Mode. When you invoke color-cycling from here, it will subsequently return to palette-editing when you `Esc` from it. See Color Cycling Commands.

`=` Create a smoothly shaded range of colors between the colors selected by the two color registers.

`M` Specify a gamma value for the shading created by `=`.

`D` Duplicate the inactive color register's values to the active color register.

`T` Stripe-shade - create a smoothly shaded range of colors between the two color registers, setting only every Nth register. After hitting `T`, hit a numeric key from 2 to 9 to specify N. For example, if you press `T` `3`, smooth shading is done between the two color registers, affecting only every 3rd color between them. The other colors between them remain unchanged.

`W` Convert current palette to gray-scale. (If the `X` or `Y` exclude ranges described later are in force, only the active range of colors is converted to gray-scale.)

`Shift` + `F2` ... `Shift` + `F9` Store the current palette in a temporary save area associated with the function key. The temporary save palettes are useful for quickly comparing different palettes or the effect of some changes - see next command. The temporary palettes are only remembered until you exit from palette-editing mode. When palette editing mode is entered, the original palette is stored in the area associated with `Shift` + `F2`.

`F2` ... `F9` Restore the palette from a temporary save area. If you haven't previously saved a palette for the function key, you'll get a simple gray scale.

`L` Pause and load an external color map (.map file). See Color Maps.

`S` Pause, prompt for a filename, and save the current color palette to the named file (.map assumed). See Color Maps.

`I` Invert frame colors. With some colors the palette is easier to see when the frame colors are interchanged.

`\` Move or resize the palette frame. The frame outline is drawn - it can then be repositioned and sized with the arrow keys, `PageUp` and `PageDown`, just as was done when first entering palette-editing mode. Hit `Enter` when done moving/sizing.

`V` Use the colors currently selected by the two color registers for the palette editor's frame. When palette editing mode is entered, the last two colors are "X"d out for use by the palette editor; this command can be used to replace the default with two other color numbers.

`A` Toggle 'auto' mode on or off. When on (the default), the active color register follows the cursor; when off, `Enter` must be pressed to set the active register to the color under the cursor.

`Enter` Only useful when 'auto' is off, as described above; double clicking the primary mouse button is the same as `Enter`.

`X` Toggle 'exclude' mode on or off - when toggled on, only those image pixels which match the active color are displayed.

`Y` Toggle 'exclude' range on or off - similar to `X`, but all pixels matching colors in the range of the two color registers are displayed.

`N` Make a negative color palette - will convert only current color if in 'x' mode or range between editors in 'y' mode or entire palette if in "normal" mode.

`!`, `@`, `#` Swap R/G, G/B, and R/B columns. `!`, `@`, and `#` are `Shift` + `1`, `Shift` + `2`, and `Shift` + `3`, which you may find easier to remember.

`U` Undoes the last palette editor command. Will undo all the way to the beginning of the current session.

`E` Redoes the undone palette editor commands.

`F` Toggles "Freestyle mode" on and off (Freestyle mode changes a range of palette values smoothly from a center value outward). With your cursor inside the palette box, press the `F` key to enter Freestyle mode. A default range of colors will be selected for you centered at the cursor (the ends of the color range are noted by putting dashed lines around the corresponding palette values). While in Freestyle mode:

- Moving the mouse changes the location of the range of colors that are affected.
- `Ctrl` + `Insert`, `Ctrl` + `Delete` or the shifted-secondary-mouse-button changes the size of the affected palette range.
- The normal color editing keys (R,G,B,1-6, etc) set the central color of the affected palette range.
- Pressing `Enter` or double-clicking the primary mouse button makes the palette changes permanent (if you don't perform this step, any palette changes disappear when you press the `F` key again to exit freestyle mode).

For more details see Freestyle mode tutorial

Image Save/Restore Commands

`S` saves the current image to disk. All parameters required to recreate the image are saved with it. Progress is marked by colored lines moving down the screen's edges.

The default filename for the first image saved after starting Id is fract001.gif; subsequent saves in the same session are automatically incremented 002, 003... Use the "savename=" parameter or `X` options screen to change the name. By default, files left over from previous sessions are not overwritten - the first unused fractnnn name is used. Use the "overwrite=yes" parameter or `X` options screen) to overwrite existing files.

A save operation can be interrupted by pressing any key. If you interrupt, you'll be asked whether to keep or discard the partial file.

`R` restores an image previously saved with `S`, or an ordinary GIF file. After pressing `R` you are shown the file names in the current directory which match the current file mask. To select a file to restore, move the cursor to it (or type the first few letters of its name) and press `Enter`.

Directories are shown in the file list with a "\" at the end of the name. When you select a directory, the contents of that directory are shown. Or, you can type the name of a different directory and press `Enter` for a new display. You can also type a mask such as "*.xyz" and press `Enter` to display files whose name ends with the matching suffix (xyz).

You can use F6 to switch directories to the default Id directory or to your own directory which is specified through the environment variable "FRACTDIR".

Once you have selected a file to restore, a summary description of the file is shown, with a video mode selection list. Usually you can just press Enter to go past this screen and load the image. Other choices available at this point are: Arrow keys: select a different video mode Tab: display more information about the fractal F1: for help about the "err" column in displayed video modes If you restore a file into a video mode which does not have the same pixel dimensions as the file, Id will make some adjustments: The view window parameters (see V command) will automatically be set to an appropriate size, and if the image is larger than the screen dimensions, it will be reduced by using only every Nth pixel during the restore.

Parameter Save/Restore Commands

Parameter files can be used to save/restore all options and settings required to recreate particular images. The parameters required to describe an image require very little disk space, especially compared with saving the image itself.

@ or 2

The @ or 2 command loads a set of parameters describing an image. (Actually, it can also be used to set non-image parameters such as sound, but at this point we're interested in images. Other uses of parameter files are discussed in Parameter Files and the @ Command.)

When you hit @ or 2, Id displays the names of the entries in the currently selected parameter file. The default parameter file, id.par, is included with the Id release and contains parameters for some sample images.

After pressing @ or 2, highlight an entry and press Enter to load it, or press F6 to change to another parameter file.

Note that parameter file entries specify all calculation related parameters, but do not specify things like the video mode - the image will be plotted in your currently selected mode.

B

The B command saves the parameters required to describe the currently displayed image, which can subsequently be used with the @ or 2 command to recreate it.

After you press B, Id prompts for:

- Parameter file: The name of the file to store the parameters in. You should use some name like "myimages" instead of id.par, so that your images are kept separate from the ones released with new versions of Id. You can use the parmfile= command in sstools.ini to set the default parameter file name to "myimages" or whatever. (See Setting Defaults (sstools.ini File) and "parmfile=" in File Parameters.)
- Name: The name you want to assign to the entry, to be displayed when the @ or 2 command is used.
- Main comment: A comment to be shown beside the entry in the @ command display.

- Second, Third, and Fourth comment: Additional comments to store in the file with the entry. These comments go in the file only, and are not displayed by the `@` command. You can set these comments from the command line - see Comment= Command.
- Record colors?: Whether color information should be included in the entry. Usually the default value displayed by Id is what you want. Allowed values are: "no" - Don't record colors. "@mapfilename" - When these parameters are used, load colors from the named color map file. This is the default if you are currently using colors from a color map file. "yes" - Record the colors in detail. This is the default when you've changed the display colors by using the palette editor or by color cycling. The only reason that this isn't what Id always does for the `B` command is that color information can be bulky - up to nearly 3K of disk space per map - which adds up to a lot for many images. Smooth-shaded ranges of colors are compressed, so if that's used a lot in an image the color information won't be as bulky. "only" - Record only the colors in the par file, without any other parameters. This is useful for converting color maps to par entries.
- # of colors: This only matters if "Record colors?" is set to "yes". It specifies the number of colors to record. Recording less colors will take less space. Usually the default value displayed by Id is what you want. You might want to increase it in some cases, e.g. if you are using a 256 color mode with maxiter 150, and have used the palette editor to set all 256 possible colors for use with color cycling, then you'll want to set the "# of colors" to 256. See the recordcolors command, which controls when mapfiles are used and when compressed colors are written to par files.
- maxlinelength: This number controls the maximum width of a parameter entry in a par file. The default is 72 characters.
- At the bottom of the input screen are inputs for Id's "pieces" divide- and-conquer feature. You can create multiple par entries that break an image up into pieces so that you can generate the image pieces one by one. There are two reasons for doing this. The first is in case the fractal is very slow, and you want to generate parts of the image at the same time on several computers. The second is that you might want to make an image greater than 2048 x 2048. The parameters are: X Multiples - How many divisions of final image in the x direction Y Multiples - How many divisions of final image in the y direction Video mode - Id video mode for each piece (e.g. "F3")
- The last item defaults to the current video mode. If either X Multiples or Y Multiples are greater than 1, then multiple numbered par entries for the pieces are added to the par file, and a makemig.bat or makemig.sh script is created that builds all of the component pieces and then stitches them together into a "multi-image" GIF. The current limitations of the "divide and conquer" algorithm are 36 or fewer X and Y multiples (so you are limited to "only" 36x36=1296 component images), and a final resolution limit in both the X and Y directions of 65,535 (a limitation of "only" four billion pixels or so).
- The generated script writes the component images and final output to the image save library selected by savedir. It passes savedir=. on each command, so running the script from its own directory keeps those files with that run. The component images are removed after the final image is assembled.
- The final image generated by makemig is a "multi-image" GIF file called fractmig.gif. In case you have other software that can't handle multi-image GIF files, makemig includes a final (but commented out) call to simplgif, a companion program that reads a GIF file that may contain little tricks like multiple images and creates a simple GIF from it. Fair warning: simplgif needs room to build a composite image while it works, and it does that using a temporary disk file

equal to the size of the final image - and a 64Kx64K GIF image requires a 4GB temporary disk file!

`G`

The `G` command lets you give a startup parameter interactively.

"3D" Commands

See "3D" Images for details of these commands.

`3` Restore a saved image as a 3D "landscape", translating its color information into "height". You will be prompted for all KINDS of options.

`#` Restore in 3D and overlay the result on the current screen.

Interrupting and Resuming

Id command keys can be loosely grouped as:

- Keys which suspend calculation of the current image (if one is being calculated) and automatically resume after the function. `Tab` (display status information) and `F1` (display help), are the only keys in this group.
- Keys which automatically trigger calculation of a new image. Examples: selecting a video mode (e.g. `F3`); selecting a fractal type using `T`; using the `X` screen to change an option such as maximum iterations.
- Keys which do something, then wait for you to indicate what to do next. Examples: `M` to go to main menu; `C` to enter color cycling mode; `PageUp` to bring up a zoom box. After using a command in this group, calculation automatically resumes when you return from the function (e.g. `Esc` from color cycling, `PageDown` to clear zoom box). There are a few fractal types which cannot resume calculation, they are noted below. Note that after saving an image with `S`, you must press `Enter` to clear the "saved" message from the screen and resume.

An image which is `S`aved before it completes can later be `R`estored and continued. The calculation is automatically resumed when you restore such an image.

When a slow fractal type resumes after an interruption in the third category above, there may be a lag while nothing visible happens. This is because most cases of resume restart at the beginning of a screen line. If unsure, you can check whether calculation has resumed with the `Tab` key.

The following fractal types cannot (currently) be resumed: plasma, 3D transformations, julibrot, and 3D orbital types like lorenz3d. To check whether resuming an image is possible, use the `Tab` key while it is calculating. It is resumable unless there is a note under the fractal type saying it is not.

The Batch Mode section discusses how to resume in batch mode.

To `R`estore and resume a "formula", "lsystem", or "ifs" type fractal your "formulafile", "lfile", or "ifsfile" must contain the required name.

Orbits Window

The `O` key toggles the Orbit mode. When this mode is on, a cursor appears over the fractal. A window appears showing the orbit used in the calculation of the color at the point where the cursor is. Move the cursor around the fractal using the arrow keys or the mouse and watch the orbits change. Try entering the Orbits mode with View Windows (`V`) turned on. The following keys take effect in Orbits mode. `C` Circle toggle - makes little circles with radii inversely proportional to the iteration. Press <C> again to toggle back to point-by-point display of orbits. `L` Line toggle - connects orbits with lines (can use with `C`) `N` Numbers toggle - shows complex coordinates & color number of the cursor on the screen. Press <N> again to turn off numbers. `P` Enter pixel coordinates directly `H` Hide fractal toggle. Works only if View Windows is turned on and set for a small window (such as the default size.) Hides the fractal, allowing the orbit to take up the whole screen. Press `H` again to uncover the fractal. `S` Saves the fractal, cursor, orbits, and numbers as they appear on the screen. `<` or `,` Zoom orbits image smaller `>` or `.` Zoom orbits image larger `Z` Restore default zoom.

View Window

The `V` command is used to set the view window parameters described below. These parameters can be used to: * Define a small window on the screen which is to contain the generated images. Using a small window speeds up calculation time (there are fewer pixels to generate). You can use a small window to explore quickly, then turn the view window off to recalculate the image at full screen size. * Generate an image with a different "aspect ratio"; e.g. in a square window or in a tall skinny rectangle. * View saved GIF images which have pixel dimensions different from any mode supported by your hardware. This use of view windows occurs automatically when you restore such an image. * Define a disk video mode up to 65535x65535. First select a disk video mode using `Delete`. Then on the `V` screen enter both an X and a Y value at the Virtual Screen Total Pixels prompts. * Define a virtual video mode up to the size that fits in memory. First select a video mode using `Delete`. Then on the `V` screen enter both an X and a Y value at the Virtual Screen Total Pixels prompts. The Keep Aspect prompt is used if the asked for virtual screen is larger than video memory. If set, the X and Y values will both be reduced such that the ratio between them is maintained. If not set, just the Y value will be reduced.

"Preview display" Set this to "yes" to turn on view window, "no" for full screen display. While this is "no", the only view parameter which has any affect is "final media aspect ratio". When a view window is being used, all other Id functions continue to operate normally - you can zoom, color-cycle, and all the rest.

"Reduction factor" When an explicit size is not given, this determines the view window size, as a factor of the screen size. For example, a reduction factor of 2 makes the window 1/2 as big as the screen in both dimensions.

"Final media aspect ratio" This is the height of the final image you want, divided by the width. The default is 0.75 because typical video modes have a height:width ratio of 3:4. For example, set this to 2.0 for an image twice as high as it is wide. The effect of this parameter is visible only when "preview display" is enabled. If the explicit size of both x and y are set, setting this value to 0 will cause the appropriate value to be calculated based on x and y.

"Crop starting coordinates" This parameter affects what happens when you change the aspect ratio.

If set to "no", then when you change aspect ratio, the prior image will be squeezed or stretched to fit into the new shape. If set to "yes", the prior image is "cropped" to avoid squeezing or stretching.

"Explicit size" Setting these to non-zero values over-rides the "reduction factor" with explicit sizes in pixels. If only the "x pixels" size is specified, the "y pixels" size is calculated automatically based on x and the aspect ratio.

The following option is available when using disk video or virtual screen modes:

"Virtual screen" Setting these allow defining a virtual screen as large as the available video memory will permit.

The following options are available when using virtual screen modes:

"Keep aspect" If this is set, when the asked for virtual screen is larger than video memory the X and Y values will both be reduced such that the ratio between them is maintained. If not set, just the Y value will be reduced.

"Zoombox scrolling" The fixed setting tries to maintain the zoombox in the center of the screen by moving the virtual image. The relaxed setting moves the virtual image when the zoombox reached the edges of the screen.

More about final aspect ratio: If you want to produce a high quality hard-copy image which is say 8" high by 5" down, based on a vertical "slice" of an existing image, you could use a procedure like the following. You'll need some method of converting a GIF image to your final media (slide or whatever) - Id can only do the whole job with a PostScript printer, it does not preserve aspect ratio with other printers. * restore the existing image * set view parameters: preview to yes, reduction to anything (say 2), aspect ratio to 1.6, and crop to yes * zoom, rotate, whatever, till you get the desired final image * set preview display back to no * trigger final calculation in some high res disk video mode, using the appropriate video mode function key * print directly to a PostScript printer, or save the result as a GIF file and use external utilities to convert to hard copy.

Video Mode Function Keys

The concept of a "video mode" in Id is somewhat of a misnomer. Id is just another GUI program on your desktop and doesn't control the video display directly. The size of the video mode determines the pixel dimenions of any saved images. So you can think of the "video mode" as "image size". In a future release, the saved image size will be decoupled from the size of the displayed image.

The video modes that can be selected are determined entirely from the settings in the id.cfg file. See Customized Video Modes, id.cfg to customize the set of supported video modes.

Any supported video mode can be selected by going to the "Select Video Mode" screen (from main menu or by using `Delete`), then using the `Up` and `Down` arrow keys and/or `PageUp` and `PageDown` keys to highlight the desired mode, then pressing `Enter`.

Up to 39 modes can be assigned to the keys F2-F10, SF1-SF10 (`Shift` + `Fn`), CF1-CF10 (`Ctrl` + `Fn`), and AF1-AF10 (`Alt` + `Fn`). The modes assigned to function keys can be invoked directly by pressing the assigned key, without going to the video mode selection screen.

30 key combinations can be reassigned: `F1` to `F10` combined with any of `Shift`, `Ctrl`, or `Alt`. The video modes assigned to `F2` through `F10` can not be changed - these are assigned to the most common video modes, which might be used in demonstration files or batches.

To reassign a function key to a mode you often use, go to the "select video mode" screen, highlight the video mode, press the keypad `+` key, then press the desired function key or key combination. The new key assignment will be remembered for future runs.

To unassign a key (so that it doesn't invoke any video mode), highlight the mode currently selected by the key and press the keypad `-` key.

See "Disk-Video" Modes for a description of these non-display modes.

Browse Commands

The following keystrokes function while browsing an image:

Arrow Keys Step through the outlines on the screen. `Enter` Selects the image to display. `\`,`H` Recalls the last image selected. `Shift` + `D` Deletes the selected file. `Shift` + `R` Renames the selected file. `S` Saves the current image with the browser boxes displayed. `Esc`,`L` Toggles the browse mode off. `Ctrl` + `B` Brings up the Browser Parameters screen. `Ctrl` + `Insert`, Change the browser boxes color. `Ctrl` + `Delete`

This is a "visual directory" showing the relationship between saved images on disk and the region of the complex plane they represent.

When typing `L` from a fractal display, the current directory is searched for any saved files that are deeper zooms of the current image and their position shown on screen by a box (or crosshairs if the box would be too small). See also Browser Parameters for more on how this is done.

One outline flashes, the selected outline can be changed by using the arrow keys. At the moment the outlines are selected in the order that they appear in your directory, so don't worry if the flashing window jumps all over the place!

When enter is pressed, the selected image is loaded. In this mode a stack of the last sixteen selected filenames is maintained and the `\` or `H` key pops and loads the last image you were looking at. Using this it is possible to set up sequences of images that allow easy exploration of your favorite fractal without having to wait for recalc once the level of zoom gets too high, great for demos! (also useful for keeping track of just exactly where fract532.gif came from :-))

You can also use this facility to tidy up your disk: by typing `Shift` + `D` when a file is selected the browser will delete the file for you, after making sure that you really mean it, you must reply to the "are you sure" prompts with an `Shift` + `Y` and nothing else, otherwise the command is ignored. Just to make absolutely sure you don't accidentally wipe out the fruits of many hours of CPU time the default setting is to have the browser prompt you twice, you can disable the second prompt within the parameters screen, however, if you're feeling overconfident :-).

To complement the delete function there is a rename function, use the `Shift` + `R` key for this. You need to enter the full new file name, no .gif is implied.

It is possible to save the current image along with all of the displayed boxes indicating subimages by pressing the `S` key. This exits the browse mode to save the image and the boxes become a permanent part of the image. Currently, the screen image ends up with stray dots colored after it is saved.

`Esc` backs out of image selecting mode.

Here's a tip on how to zoom out beyond your starting point when browsing: Suppose you restore a fractal deeply-zoomed down in a directory of related zoomed images, and then bring up the browser. How do you zoom out? You can't use `\` because you started with the zoomed image, and there is no browser command to detect the next outer image. What you can do is exit the browser, press `PageUp` until the zoom box won't get any smaller, zoom out with `Ctrl` + `Enter`, and before any image starts to develop, call up the browser again, locate your zoomed image that you started with, and see if there is another image that contains it - if so, restore it with the browser. You can also use a view window `V` to load the first image, and then use the browser.

POSSIBLE ERRORS:

"Sorry..I can't find anything" The browser can't locate any files which match the file name mask. See Browser Parameters This is also displayed if you have less than 10K of far memory free when you run Id.

"Sorry.... no more space" At the moment the browser can only cope with 450 sub images at one time. Any subsequent images are ignored. Make sure that the minimum image size isn't set too small on the parameters screen.

"Sorry...it's a read only file, can't del [filename]" "Sorry....can't rename" The file which you were trying to delete or rename has the read only attribute set, you'll need to reset this with your operating system before you can get rid of it.

Evolver Commands

```
PageUp          When no Zoom Box is active, brings one up.
                When Zoom Box is active already, shrinks it.
PageDown        Expands the Zoom Box.
                Expanding past the screen size cancels the Zoom Box.
Arrow key       Pans (Moves) the Zoom Box.
Ctrl+Arrow key  Moves the Zoom Box to the next subimage.
Enter           Redraws the Screen or area inside the Zoom Box.
Ctrl+Enter      'Zoom-out' - expands the image so that your current image
                is positioned inside the current zoom-box location.
Ctrl+Pad+/Pad-  Rotates the inner Zoom Box.
Ctrl+PgUp/PgDn  Changes inner Zoom Box vertical size.
Ctrl+Home/End   Changes inner Zoom Box shape.
Ctrl+Ins/Del    Changes inner Zoom Box color.
Ctrl+E          Brings up the evolver screen.
Space           Brings up the evolver screen once in evolver mode.
B               Turns off evolver if in evolver mode.
F2              Halves the amount of mutation.
```

```
F3              Doubles the amount of mutation.
F4              Generates fewer, bigger images.
F5              Generates more, smaller images.
F6              Switches to/from 'spread' mode with fewer mutations around
                the middle.
```

RDS Commands

The following keystrokes function while viewing an RDS image:

`Enter` or `Space` — Toggle calibration bars on and off. `Ctrl` + `S` or `K` — Return to RDS Parameters Screen. `S` — Save RDS image, then restore original. `C`, `+`, `-` — Color cycle RDS image. Other keys — Exit RDS mode, restore original image, and pass keystroke on to main menu.

For more about RDS, see Random Dot Stereograms (RDS)

Hints

Remember, you do NOT have to wait for the program to finish a full screen display before entering a command. If you see an interesting spot you want to zoom in on while the screen is half-done, don't wait — do it! If you think after seeing the first few lines that another video mode would look better, go ahead — Id will shift modes and start the redraw at once. When it finishes a display, it beeps and waits for your next command.

In general, the most interesting areas are the "border" areas where the colors are changing rapidly. Zoom in on them for the best results. The first Mandelbrot-set (default) fractal image has a large, solid-colored interior that is the slowest to display; there's nothing to be seen by zooming there.

Plotting time is directly proportional to the number of pixels in a screen, and hence increases with the resolution of the video mode. You may want to start in a low-resolution mode for quick progress while zooming in, and switch to a higher-resolution mode when things get interesting. Or use the solid guessing mode and pre-empt with a zoom before it finishes. Plotting time also varies with the maximum iteration setting, the fractal type, and your choice of drawing mode. Solid-guessing (the default) is fastest, but it can be wrong: perfectionists will want to use dual-pass mode (its first-pass preview is handy if you might zoom pre-emptively) or single-pass mode.

When you start systematically exploring, you can save time (and hey, every little bit helps — these "objects" are INFINITE, remember!) by `S`aving your last screen in a session to a file, and then going straight to it the next time by using the command id fractxxx (the .gif extension is assumed), or by starting Id normally and then using the `R` command to reload the saved file. Or you could hit `B` to create a parameter file entry with the "recipe" for a given image, and next time use the `@` command to re-plot it.

Id on Linux and macOS

On Linux and macOS, Id uses the X11 driver. The driver uses an ordinary TrueColor X visual and keeps Id's 256-color image buffer internally, so a PseudoColor display is not required. On macOS,

the X11 display is provided by XQuartz.

On Linux and macOS, the executable file is named xid rather than id to avoid the standard system id utility. Command-line examples that start with id should be typed with xid on Linux and macOS.

The X11 driver uses the same id.cfg video mode table as the Win32 GDI driver. Video modes whose driver field is "gdi" are accepted by the X11 driver when the requested size fits on the current X screen and the mode uses 256 colors. Very large disk-video modes remain output modes, not display windows. The standard video= parameter and video mode selection screen are available.

The X11 window requests the selected text or graphics size from the window manager. If the window manager allows manual resizing, the video mode and image buffer remain the selected size. On Linux, the window position is remembered in the user's XDG configuration area. On macOS, it is remembered in ~/Library/Preferences as com.legalizeadulthood.iterated-dynamics.plist. If the saved position is not valid for the current X screen, Id lets the window manager place the window. A standard X geometry option such as "-geometry +100+100" can be used on the command line to choose the initial position for a run. An X11 font can be selected with "-fn <font>" or "--font <font>". The X11 driver chooses a fixed font from the display DPI.

Keyboard menus, help, text overlays, the text-mode file browser, mouse zoom, double-click zoom, image rendering, palette cycling, command shell return, and expose redraw are implemented. Output files use the normal savedir rules. On Linux the default save library follows XDG_DOCUMENTS_DIR or user-dirs.dirs and falls back to $HOME/Documents. On macOS the default save library uses the user's $HOME/Documents folder. See File Parameters and Libraries.

Continuous sound output is not implemented; the completion buzzer uses the X server bell when sound is enabled. Non-X11 desktops are not currently supported.

Fractal Types

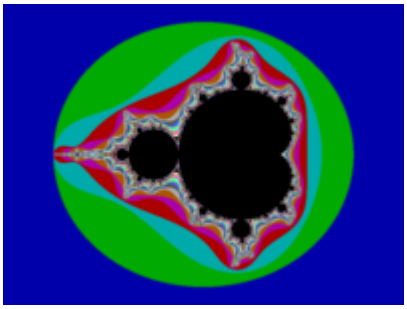
A list of the fractal types and their mathematics can be found in the Summary of Fractal Types. Some notes about how Id calculates them are in "A Little Code" in Fractals and the PC.

Id starts by default with the Mandelbrot set. You can change that by using the command-line argument "type=" followed by one of the fractal type names, or by using the `T` command and selecting the type - if parameters are needed, you will be prompted for them.

In the text that follows, due to the limitations of the ASCII character set, "a*b" means "a times b", and "a^b" means "a to the power b".

The Mandelbrot Set

(type=mandel)



This set is the classic: the only one implemented in many plotting programs, and the source of most of the printed fractal images published. Like most of the other types in Id, it is simply a graph: the x (horizontal) and y (vertical) coordinate axes represent ranges of two independent quantities, with various colors used to symbolize levels of a third quantity which depends on the first two. So far, so good: basic analytic geometry.

Now things get a bit hairier. The x axis is ordinary, vanilla real numbers. The y axis is an imaginary number, i.e. a real number times i, where i is the square root of -1. Every point on the plane — in this case, Id's main window — represents a complex number of the form:

```
x-coordinate + i * y-coordinate
```

If your math training stopped before you got to imaginary and complex numbers, this is not the place to catch up. Suffice it to say that they are just as "real" as the numbers you count fingers with (they're used every day by electrical engineers) and they can undergo the same kinds of algebraic operations.

OK, now pick any complex number — any point on the complex plane — and call it C, a constant. Pick another, this time one which can vary, and call it Z. Starting with Z=0 (i.e., at the origin, where the real and imaginary axes cross), calculate the value of the expression

$$Z^2 + C$$

Take the result, make it the new value of the variable Z, and calculate again. Take that result, make it Z, and do it again, and so on: in mathematical terms, iterate the function $Z_{n+1} = Z_n^2 + C$. For certain values of C, the result "levels off" after a while. For all others, it grows without limit. The Mandelbrot set you see at the start — the solid-colored lake (blue by default), the blue circles sprouting from it, and indeed every point of that color — is the set of all points C for which the magnitude of Z is less than 2 after 150 iterations (150 is the default setting, changeable via the `X` options screen or "maxiter=" parameter). All the surrounding "contours" of other colors represent points for which the magnitude of Z exceeds 2 after 149 iterations (the contour closest to the M-set itself), 148 iterations, (the next one out), and so on.

We actually don't test for the magnitude of Z exceeding 2 - we test the magnitude of Z squared against 4 instead because it is easier. This value (FOUR usually) is known as the "bailout" value for the calculation, because we stop iterating for the point when it is reached. The bailout value can be changed on the `Z` options screen but the default is usually best. See also Bailout Test.

Some features of interest:

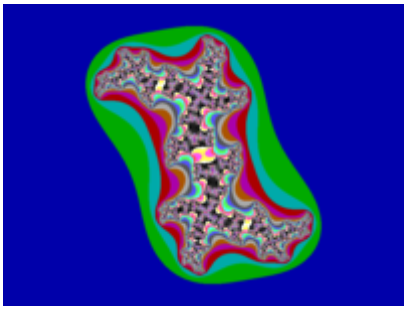
1. Use the `X` options screen to increase the maximum number of iterations. Notice that the

boundary of the M-set becomes more and more convoluted (the technical terms are "wiggly", "squiggly", and "utterly bizarre") as the Z-magnitudes for points that were still within the set after 150 iterations turn out to exceed 2 after 200, 500, or 1200. In fact, it can be proven that the true boundary is infinitely long: detail without limit.

2. Although there appear to be isolated "islands" of blue, zoom in — that is, plot for a smaller range of coordinates to show more detail — and you'll see that there are fine "causeways" of blue connecting them to the main set. As you zoomed, smaller islands became visible; the same is true for them. In fact, there are no isolated points in the M- set: it is "connected" in a strict mathematical sense.
3. The upper and lower halves of the first image are symmetric (a fact that Id makes use of here and in some other fractal types to speed plotting). But notice that the same general features — lobed discs, spirals, starbursts — tend to repeat themselves (although never exactly) at smaller and smaller scales, so that it can be impossible to judge by eye the scale of a given image.
4. In a sense, the contour colors are window-dressing: mathematically, it is the properties of the M-set itself that are interesting, and no information about it would be lost if all points outside the set were assigned the same color. If you're a serious, no-nonsense type, you may want to cycle the colors just once to see the kind of silliness that other people enjoy, and then never do it again. Go ahead. Just once, now. We trust you.

Julia Sets

(type=julia)



These sets were named for mathematician Gaston Julia, and can be generated by a simple change in the iteration process described for the Mandelbrot Set. Start with a specified value of C, "C-real + i * C- imaginary"; use as the initial value of Z "x-coordinate + i * y- coordinate"; and repeat the same iteration, $Z_{n+1} = Z_n^2 + C$.

There is a Julia set corresponding to every point on the complex plane — an infinite number of Julia sets. But the most visually interesting tend to be found for the same C values where the M-set image is busiest, i.e. points just outside the boundary. Go too far inside, and the corresponding Julia set is a circle; go too far outside, and it breaks up into scattered points. In fact, all Julia sets for C within the M- set share the "connected" property of the M-set, and all those for C outside lack it.

Id's spacebar toggle lets you "flip" between any view of the M-set and the Julia set for the point C at the center of that screen. You can then toggle back, or zoom your way into the Julia set for a while and then return to the M-set. So if the infinite complexity of the M-set palls, remember: each of its infinite points opens up a whole new Julia set.

Historically, the Julia sets came first: it was while looking at the M- set as an "index" of all the Julia sets' origins that Mandelbrot noticed its properties.

The relationship between the Mandelbrot set and Julia set can hold between other sets as well. Many of Id's types are "Mandelbrot/Julia" pairs — sometimes called "M-sets" or "J-sets". All these are generated by equations that are of the form $z_{k+1} = f(z_k, c)$, where the function orbit is the sequence z_0, z_1, ⋯, and the variable c is a complex parameter of the equation. The value c is fixed for "Julia" sets and is equal to the first two parameters entered with the "params=Creal/Cimag" command. The initial orbit value z(0) is the complex number corresponding to the screen pixel. For Mandelbrot sets, the parameter c is the complex number corresponding to the screen pixel. The value z_0 is c plus a perturbation equal to the values of the first two parameters. See the discussion of Mandellambda Sets. This approach may or may not be the "standard" way to create "Mandelbrot" sets out of "Julia" sets.

Some equations have additional parameters. These values are entered as the third or fourth params= value for both Julia and Mandelbrot sets. The variables x and y refer to the real and imaginary parts of z; similarly, cx and cy are the real and imaginary parts of the parameter c and fx(z) and fy(z) are the real and imaginary parts of f(z). The variable c is sometimes called lambda for historical reasons.

NOTE	if you use the "params=" argument to warp the M-set by starting with an initial value of Z other than 0, the M-set/J-sets correspondence breaks down and the spacebar toggle no longer works.

Julia Toggle Spacebar Commands

The spacebar toggle has been enhanced for the classic Mandelbrot and Julia types. When viewing the Mandelbrot, the spacebar turns on a window mode that displays the Inverse Julia corresponding to the cursor position in a window. Pressing the spacebar then causes the regular Julia escape time fractal corresponding to the cursor position to be generated. The following keys take effect in Inverse Julia mode.

`Space` Generate the escape-time Julia Set corresponding to the cursor position. Only works if fractal is a "Mandelbrot" type. `N` Numbers toggle - shows coordinates of the cursor on the screen. Press <N> again to turn off numbers. `P` Enter new pixel coordinates directly `H` Hide fractal toggle. Works only if View Windows is turned on and set for a small window (such as the default size.) Hides the fractal, allowing the orbit to take up the whole screen. Press <H> again to uncover the fractal. `S` Saves the fractal, cursor, orbits, and numbers. `<` or `,` Zoom inverse Julia image smaller. `>` or `.` Zoom inverse Julia image larger. `Z` Restore default zoom.

The Julia Inverse window is only implemented for the classic Mandelbrot (type=mandel). For other "Mandelbrot" types `Space` turns on the cursor without the Julia window, and allows you to select coordinates of the matching Julia set.

Inverse Julias

(type=julia_inverse)

Pick a function, such as the familiar $Z_n = Z_{n-1} + C$ (the defining function of the Mandelbrot Set). If you pick a point Z_0 at random from the complex plane, and repeatedly apply the function to it, you get a sequence of new points called an orbit, which usually either zips out toward infinity or zooms in toward one or more "attractor" points near the middle of the plane. The set of all points that are "attracted" to infinity is called the "basin of attraction" of infinity. Each of the other attractors also has its own basin of attraction. Why is it called a basin? Imagine a lake, and all the water in it "draining" into the attractor. The boundary between these basins is called the Julia Set of the function.

The boundary between the basins of attraction is sort of like a repeller; all orbits move away from it, toward one of the attractors. But if we define a new function as the inverse of the old one, as for instance $Z_n = \sqrt{Z_{n-1} - C}$, then the old attractors become repellers, and the former boundary itself becomes the attractor! Now, starting from any point, all orbits are drawn irresistibly to the Julia Set! In fact, once an orbit reaches the boundary, it will continue to hop about until it traces the entire Julia Set! This method for drawing Julia Sets is called the Inverse Iteration Method, or IIM for short.

Unfortunately, some parts of each Julia Set boundary are far more attractive to inverse orbits than others are, so that as an orbit traces out the set, it keeps coming back to these attractive parts again and again, only occasionally visiting the less attractive parts. Thus it may take an infinite length of time to draw the entire set. To hasten the process, we can keep track of how many times each pixel on our computer screen is visited by an orbit, and whenever an orbit reaches a pixel that has already been visited more than a certain number of times, we can consider that orbit finished and move on to another one. This "hit limit" thus becomes similar to the iteration limit used in the traditional escape-time fractal algorithm. This is called the Modified Inverse Iteration Method, or MIIM, and is much faster than the IIM.

Now, the inverse of Mandelbrot's classic function is a square root, and the square root actually has two solutions; one positive, one negative. Therefore at each step of each orbit of the inverse function there is a decision; whether to use the positive or the negative square root. Each one gives rise to a new point on the Julia Set, so each is a good choice. This series of choices defines a binary decision tree, each point on the Julia Set giving rise to two potential child points. There are many interesting ways to traverse a binary tree, among them breadth first, depth first (left or negative first), depth first (right or positive first), and completely at random. It turns out that most traversal methods lead to the same or similar pictures, but that how the image evolves as the orbits trace it out differs wildly depending on the traversal method chosen. As far as we know, this fact is an original discovery by Michael Snyder, and version 18.2 of Id's ancestor, Fractint, was its first publication.

Pick a Julia constant such as Z_0 = (-.74543, .11301), the popular Seahorse Julia, and try drawing it first breadth first, then depth first (right first), depth first (left first), and finally with random walk.

Caveats: the video memory is used in the algorithm, to keep track of how many times each pixel has been visited (by changing it's color). Therefore the algorithm will not work well if you zoom in far enough that part of the Julia Set is off the screen.

This method has the limitation that it does not work with disk video modes and it is not resumable.

The `J` key toggles between the inverse Julia orbit and the corresponding Julia escape time fractal.

Ant Automaton

(type=ant)

This fractal type is the generalized ant automaton described in the "Computer Recreations" column of the July 1994 Scientific American. The article attributes this automaton to Greg Turk of Stanford University, Leonid A. Bunivomitch of the Georgia Institute of Technology, and S. E. Troubetzkoy of the University of Bielefeld.

The ant wanders around the screen, starting at the middle. A rule string, which the user can input as Id's first parameter, determines the ant's direction. Only the digit 1 is significant — all other characters are treated as 0. When the type 1 ant leaves a cell (a pixel on the screen) of color k, it turns right if the kth symbol in the rule string is a 1, or left otherwise. Then the color in the abandoned cell is incremented. The type 2 ant uses only the rule string to move around. If the digit of the rule string is a 1, the ant turns right and puts a zero in the current cell, otherwise it turns left and puts a number in the current cell. An empty rule string causes the rule to be generated randomly.

When the first parameter is written as integer text, Id preserves that text as the rule string. This allows exact rule strings wider than Fractint's double precision numeric formatting can represent. When the first parameter uses decimal or exponent notation, Id follows Fractint compatibility and interprets the numeric value using Fractint's rule formatting.

The 2nd parameter is a maximum iteration to guarantee that the fractal will terminate.

The 3rd parameter is the number of ants (up to 256). If you select 0 ants, then the number of ants is random.

The 4th parameter allows you to select ant type 1 (the original), or type 2.

The 5th parameter determines whether the ant's progress stops when the edge of the screen is reaches (as in the original implementation), or whether the ant's path wraps to the opposite side of the screen. You can slow down the ant to see her better using the `P` screen orbit delay - try 10.

The 6th parameter accepts a random seed, allowing you to duplicate images using random values (empty rule string or 0 ants). The rseed= command is used when the 6th parameter is 0 or 1. Any other 6th parameter value is the Ant seed and overrides rseed= regardless of command-line or PAR order.

Try rule string 10. In this case, the ant moves in a seemingly random pattern, then suddenly marches off in a straight line. This happens for many other rule strings. The default 1100 produces symmetrical images.

If the screen initially contains an image, the path of the ant changes. To try this, generate a fractal, and press `Ctrl` + `A`. Note that images seeded with an image are not (yet) reproducible in PAR files. When started using the `Ctrl` + `A` keys, after the ant is finished the default fractal type reverts to that of the underlying fractal.

Special keystrokes are in effect during the ant's march. The `Space` key toggles a step-by-step mode. When in this mode, press `Enter` to see each step of the ant's progress. When orbit delay (on `P` screen)

is set to 1, the step mode is the default.

If you press the right or left arrow during the ant's journey, you can adjust the orbit delay factor with the arrow keys (increment by 10) or `Ctrl` + `Arrow` keys (increment by 100). Press any other key to get out of the orbit delay adjustment mode. Higher values cause slower motion. Changed values are not saved after the ant is finished, but you can set the orbit delay value in advance from the `P` screen.

Barnsley IFS Fractals

(type=ifs)

One of the most remarkable spin-offs of fractal geometry is the ability to "encode" realistic images in very small sets of numbers — parameters for a set of functions that map a region of two-dimensional space onto itself. In principle (and increasingly in practice), a scene of any level of complexity and detail can be stored as a handful of numbers, achieving amazing "compression" ratios... how about an image of a forest, more than 300,000 pixels at eight bits apiece, from a 1-KB "seed" file?

Again, Michael Barnsley and his co-workers at the Georgia Institute of Technology are to be thanked for pushing the development of these iterated function systems (IFS).

When you select this fractal type, Id scans the current IFS file (default is id.ifs, a set of definitions supplied with Id) for IFS definitions, then prompts you for the IFS name you wish to run. Fern and 3dfern are good ones to start with. You can press `F6` at the selection screen if you want to select a different .IFS file you've written.

Note that some Barnsley IFS values generate images quite a bit smaller than the initial (default) screen. Just bring up the zoom box, center it on the small image, and hit `Enter` to get a full-screen image.

To change the number of dots Id generates for an IFS image before stopping, you can change the "maximum iterations" parameter on the `X` options screen.

Id supports two types of IFS images: 2D and 3D. In order to fully appreciate 3D IFS images, since your monitor is presumably 2D, we have added rotation, translation, and perspective capabilities. These share values with the same variables used in Id's other 3D facilities; for their meaning see Rectangular Coordinate Transformation. You can enter these values from the command line using:

rotation=xrot/yrot/zrot (try 30/30/30) shift=xshift/yshift (shifts before applying perspective!) perspective=viewerposition (try 200)

Alternatively, entering `I` from main screen will allow you to modify these values. The defaults are the same as for regular 3D, and are not always optimum for 3D IFS. With the 3dfern IFS type, try rotation=30/30/30. Note that applying shift when using perspective changes the picture — your "point of view" is moved.

A truly wild variation of 3D may be seen by entering "2" for the stereo mode (see Stereo 3D Viewing), putting on red/blue "funny glasses", and watching the fern develop with full depth perception right there before your eyes!

This feature used to be dedicated to Bruce Goren, as a bribe to get him to send us more knockout stereo slides of 3D ferns, now that we have made it so easy! Bruce, what have you done for us **LATELY**?? (Just kidding, really!)

Each line in an IFS definition (look at id.ifs with your editor for examples) contains the parameters for one of the generating functions, e.g. in fern:

a	b	c	d	e	f	p
0	0	0	.16	0	0	.01
.85	.04	−.04	.85	0	1.6	.85
.2	−.26	.23	.22	0	1.6	.07
−.15	.28	.26	.24	0	.44	.07

The values on each line define a matrix, vector, and probability:

matrix	vector	prob
$\begin{bmatrix} a & b \\ c & d \end{bmatrix}$	$\begin{bmatrix} ef \end{bmatrix}$	p

The "p" values are the probabilities assigned to each function (how often it is used), which add up to one. Id supports up to 32 functions, although usually three or four are enough.

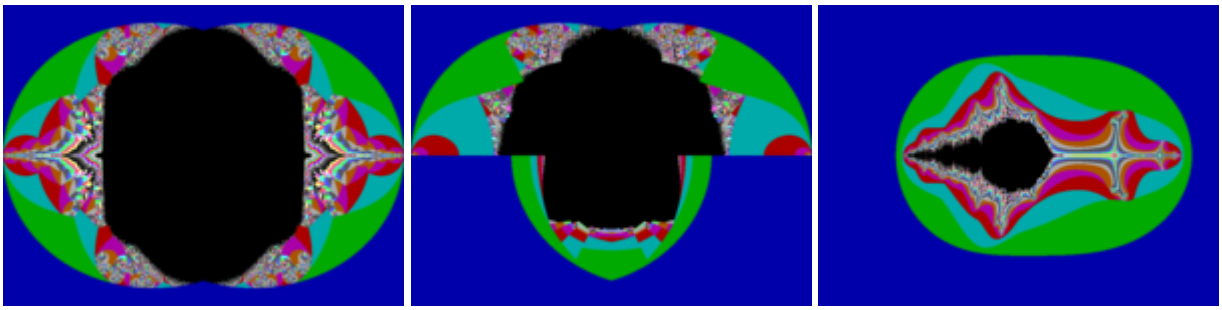
3D IFS definitions are a bit different. The name is followed by (3D) in the definition file, and each line of the definition contains 13 numbers: a b c d e f g h i j k l p, defining:

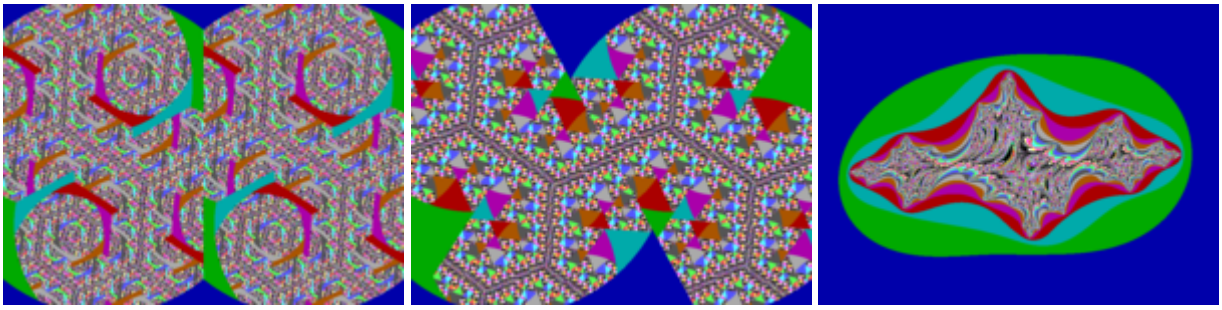
matrix	vector	prob
$\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$	$\begin{bmatrix} jkl \end{bmatrix}$	p

You can save the points in your IFS fractal in the file orbits.raw which is overwritten each time a fractal is generated.

Barnsley Mandelbrot/Julia Sets

(type=barnsleym1, barnsleym2, barnsleym3, barnsleyj1, barnsleyj2, barnsleyj3)





Michael Barnsley has written a fascinating college-level text, "Fractals Everywhere", on fractal geometry and its graphic applications. (See Bibliography.) In it, he applies the principle of the M and J sets to more general functions of two complex variables.

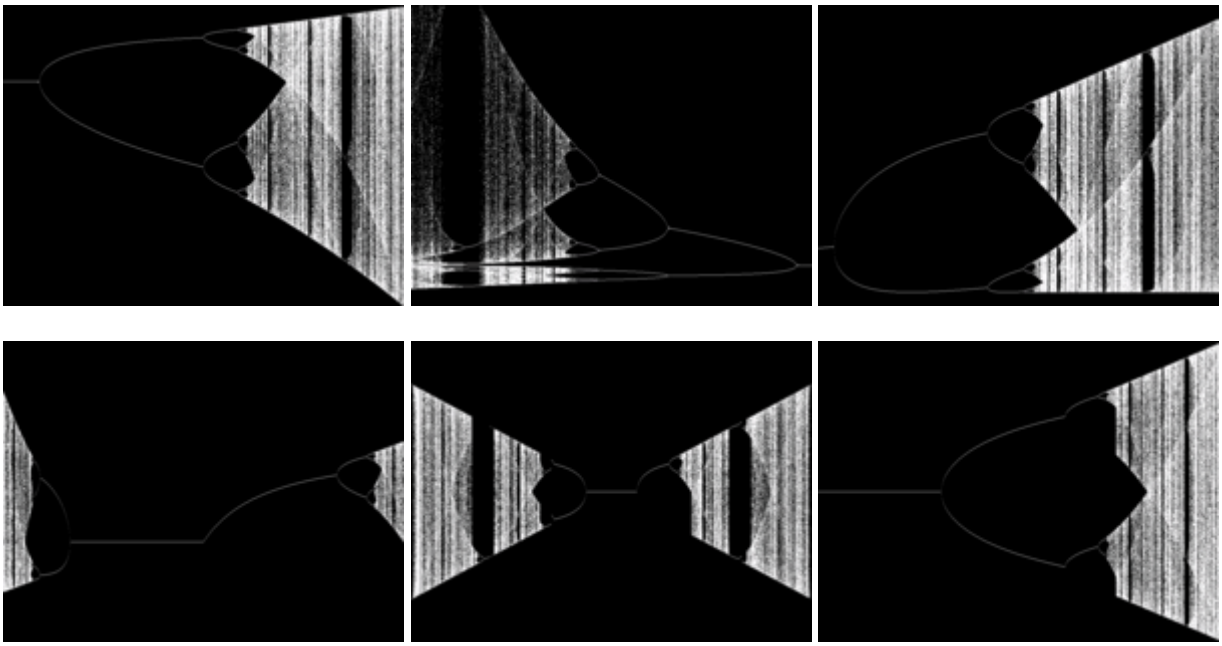
We have incorporated three of Barnsley's examples in Id. Their appearance suggests polarized-light microphotographs of minerals, with patterns that are less organic and more crystalline than those of the M/J sets. Each example has both a "Mandelbrot" and a "Julia" type. Toggle between them using the spacebar.

The parameters have the same meaning as they do for the "regular" Mandelbrot and Julia. For types M1, M2, and M3, they are used to "warp" the image by setting the initial value of Z. For the types J1 through J3, they are the values of C in the generating formulas.

Be sure to try the `O`rbit function while plotting these types.

Bifurcation

(type=bifurcation, bifmay, bifstewart, biflambda, bif=sinpi, bif+sinpi)



The wonder of fractal geometry is that such complex forms can arise from such simple generating processes. A parallel surprise has emerged in the study of dynamical systems: that simple, deterministic equations can yield chaotic behavior, in which the system never settles down to a steady state or even a periodic loop. Often such systems behave normally up to a certain level of some controlling parameter, then go through a transition in which there are two possible solutions,

then four, and finally a chaotic array of possibilities.

This emerged many years ago in biological models of population growth. Consider a (highly over-simplified) model in which the rate of growth is partly a function of the size of the current population:

$$P_{n+1} = rP_n(1 - P_n)$$

where population is normalized to be between 0 and 1. At growth rates less than 200 percent, this model is stable: for any starting value, after several generations the population settles down to a stable level. But for rates over 200 percent, the equation's curve splits or "bifurcates" into two discrete solutions, then four, and soon becomes chaotic.

Type=bifurcation illustrates this model. (Although it's now considered a poor one for real populations, it helped get people thinking about chaotic systems.) The horizontal axis represents growth rates, from 190 percent (far left) to 400 percent; the vertical axis normalized population values, from 0 to 4/3. Notice that within the chaotic region, there are narrow bands where there is a small, odd number of stable values. It turns out that the geometry of this branching is fractal; zoom in where changing pixel colors look suspicious, and see for yourself.

Three parameters apply to bifurcations: Filter Cycles, Seed Population, and Function or Beta.

Filter Cycles (default 1000) is the number of iterations to be done before plotting maxiter population values. This gives the iteration time to settle into the characteristic patterns that constitute the bifurcation diagram, and results in a clean-looking plot. However, using lower values produces interesting results too. Set Filter Cycles to 1 for an unfiltered map.

Seed Population (default 0.66) is the initial population value from which all others are calculated. For filtered maps the final image is independent of Seed Population value in the valid range (0.0 < Seed Population < 1.0). Seed Population becomes effective in unfiltered maps - try setting Filter Cycles to 1 (unfiltered) and Seed Population to 0.001 ("params=1/.001" on the command line). This results in a map overlaid with nice curves. Each Seed Population value results in a different set of curves.

Function (default "ident") is the function applied to the old population before the new population is determined. The "ident" function calculates the same bifurcation fractal that was generated before these formulae were generalized.

Beta is used in the bifmay bifurcations and is the power to which the denominator is raised.

Note that Id normally uses periodicity checking to speed up bifurcation computation. However, in some cases a better quality image will be obtained if you turn off periodicity checking with "periodicity=no"; for instance, if you use a high number of iterations and a smooth colormap.

Many formulae can be used to produce bifurcations. Mitchel Feigenbaum studied lots of bifurcations in the mid-70's, using a HP-65 calculator (Personal computers, fractals, and Id, were all science fiction then!). He studied where bifurcations occurred, for the formula $rP(1-P)$ the one described above. He found that the ratios of lengths of adjacent areas of bifurcation were four and a bit. These ratios vary, but, as the growth rate increases, they tend to a limit of 4.669+. This helped him guess where bifurcation points would be, and saved lots of time.

When he studied bifurcations of $r\sin(\pi P)$ he found a similar pattern, which is not surprising in itself. However, 4.669+ popped out, again. Different formulae, same number? Now, that's surprising! He tried many other formulae and always got 4.669+ - Hot Damn!!! So hot, in fact, that he phoned home and told his mom it would make him famous! He also went on to tell other scientists. The rest is history...

(It has been conjectured that if Feigenbaum had a copy of Id, and used it to study bifurcations, he may never have found his number, as it only became obvious from long perusal of hand-written lists of values, without the distraction of wild color-cycling effects!). We now know that this number is as universal as pi or e, the base of the natural logarithm. It appears in situations ranging from fluid-flow turbulence, electronic oscillators, chemical reactions, and even the Mandelbrot Set - yup, fraid so: "budding" of the Mandelbrot Set along the negative real axis occurs at intervals determined by Feigenbaum's constant, 4.669201660910.....

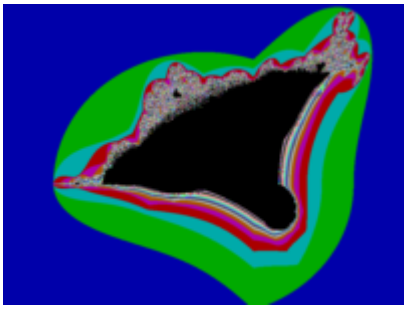
Id does not make direct use of the Feigenbaum constant (yet!). However, it does now reflect the fact that there is a whole sub-species of bifurcation-type fractals. Those implemented to date, and the related formulae,(writing P_{n+1} for pop[n+1] and P_n for pop[n]) are:

bifurcation	$P_{n+1} = P_n + r\,\mathrm{fn}(P_n)(1 - \mathrm{fn}(P_n))$	Verhulst Bifurcations
biflambda	$P_{n+1} = r\,\mathrm{fn}(P_n)(1 - \mathrm{fn}(P_n))$	Real equivalent of Lambda Sets.
bif+sinpi	$P_{n+1} = P_n + r\,\mathrm{fn}(\pi P_n)$	Population scenario based on...
bif=sinpi	$P_{n+1} = r\,\mathrm{fn}(\pi P_n)$	...Feigenbaum's second formula.
bifstewart	$P_{n+1} = r\,\mathrm{fn}(P_n)\mathrm{fn}(P_n) - 1$	Stewart Map.
bifmay	$P_{n+1} = r\frac{P_n}{(1+P_n)^b}$	May Map.

It took a while for bifurcations to appear here, despite them being over a century old, and intimately related to chaotic systems. However, they are now truly alive and well in Id!

Burning Ship

(type=burning-ship)



This fractal is a derivative of the classic Mandelbrot fractal with a twist. Some parts use absolute values in the equations and this gives an amazing angular quality to the fractal.

```
With stem:[z_n = x_n + i y_n] and degree stem:[d] from 2 through 5:
```

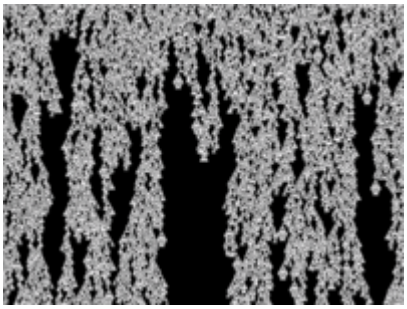
$$z_{n+1} = (|x_n| + i|y_n|)^d + c$$

The value c is the screen pixel. The first two parameters are added to z_0 as described for Mandelbrot-type sets; z_0 defaults to $(0, 0)$. Burning Ship images are commonly reflected vertically.

The parameters are perturbation of z_0 and d.

Cellular Automata

(type=cellular)



These fractals are generated by 1-dimensional cellular automata. Consider a 1-dimensional line of cells, where each cell can have the value 0 or 1. In each time step, the new value of a cell is computed from the old value of the cell and the values of its neighbors. On the screen, each horizontal row shows the value of the cells at any one time. The time axis proceeds down the screen, with each row computed from the row above.

Different classes of cellular automata can be described by how many different states a cell can have (k), and how many neighbors on each side are examined (r). Id implements the binary nearest neighbor cellular automata (k=2,r=1), the binary next-nearest neighbor cellular automata (k=2,r=2), and the ternary nearest neighbor cellular automata (k=3,r=1) and several others.

The rules used here determine the next state of a given cell by using the sum of the states in the cell's neighborhood. The sum of the cells in the neighborhood are mapped by rule to the new value of the cell. For the binary nearest neighbor cellular automata, only the closest neighbor on each side is used. This results in a 4 digit rule controlling the generation of each new line: if each of the cells in the neighborhood is 1, the maximum sum is 1+1+1 = 3 and the sum can range from 0 to 3, or 4 values. This results in a 4 digit rule. For instance, in the rule 1010, starting from the right we have 0→0, 1→1, 2→0, 3→1. If the cell's neighborhood sums to 2, the new cell value would be 0.

For the next-nearest cellular automata (kr = 22), each pixel is determined from the pixel value and the two neighbors on each side. This results in a 6 digit rule.

For the ternary nearest neighbor cellular automata (kr = 31), each cell can have the value 0, 1, or 2. A single neighbor on each side is examined, resulting in a 7 digit rule.

kr	#'s in rule	example rule	kr	#'s in rule	example rule
21	4	1010	42	16	2300331230331001
31	7	1211001	23	8	10011001
41	10	3311100320	33	15	021110101210010

kr	#'s in rule	example rule	kr	#'s in rule	example rule
51	13	2114220444030	24	10	0101001110
61	16	3452355321541340	25	12	110101011001
22	6	011010	26	14	00001100000110
32	11	21212002010	27	16	0010000000000110

The starting row of cells can be set to a pattern of up to 16 digits or to a random pattern. The borders are set to zeros if a pattern is entered or are set randomly if the starting row is set randomly.

A zero rule will randomly generate the rule to use.

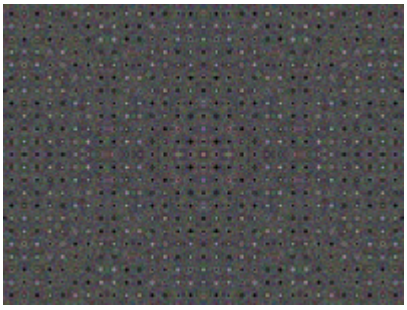
Random starting rows and randomly generated rules honor the rseed= command.

Hitting the space bar toggles between continuously generating the cellular automata and stopping at the end of the current screen.

Recommended reading: "Computer Software in Science and Mathematics", Stephen Wolfram, Scientific American, September, 1984. "Abstract Mathematical Art", Kenneth E. Perry, BYTE, December, 1986. "The Armchair Universe", A. K. Dewdney, W. H. Freeman and Company, 1988. "Complex Patterns Generated by Next Nearest Neighbors Cellular Automata", Wentian Li, Computers & Graphics, Volume 13, Number 4.

Circle

(type=circle)



This fractal types is from A. K. Dewdney's "Computer Recreations" column in "Scientific American". It is attributed to John Connett of the University of Minnesota.

(Don't tell anyone, but this fractal type is not really a fractal!)

Fascinating Moire patterns can be formed by calculating $x^2 + y^2$ for each pixel in a piece of the complex plane. After multiplication by a magnification factor (the parameter), the number is truncated to an integer and mapped to a color via $\text{color} = \text{value} \bmod \text{number-of-colors}$. That is, the integer is divided by the number of colors, and the remainder is the color index value used. The resulting image is not a fractal because all detail is lost after zooming in too far. Try it with different resolution video modes - the results may surprise you!

If inside=startrail is used, it will automatically be set to inside=norm by Id. This is because type

circle and inside=startrail locks up Id.

Diffusion Limited Aggregation

(type=diffusion)

Standard diffusion begins with a single point in the center of the screen. Subsequent points move around randomly until coming into contact with a point already on the screen, at which time their locations are fixed and they are drawn. This process repeats until the fractals reaches the edge of the screen. Use the show orbits function to see the points' random motion.

One unfortunate problem is that on a large screen, this process will tend to take eons. To speed things up, the points are restricted to a box around the initial point. The first parameter to diffusion contains the size of the border between the fractal and the edge of the box. If you make this number small, the fractal will look more solid and will be generated more quickly.

The second parameter to diffusion changes the type of growth. If you set it to 1, then the diffusion will start with a line along the bottom of the screen. Points will appear above this line and the fractal will grow upward. For this fractal, the points are restricted to a box which is as wide as the screen but whose distance from the fractal is given by the border size (the first parameter). Initial points are released from a centered segment along the top of this box which has a width equal to twice the border size.

If the second parameter is set to 2, then diffusion begins with a square box on the screen. Points appear on a circle inside the box whose distance from the box is equal to the border size. This fractal grows very slowly since the points are not restricted to a small box.

The third and last parameter for diffusion controls the color of the fractal. If it is set to zero then points are colored randomly. Otherwise, it tells how often to shift the color of the points being deposited. If you set it to 150, for example, then the color of the points will shift every 150 points leading to a radial color pattern if you are using the standards diffusion type.

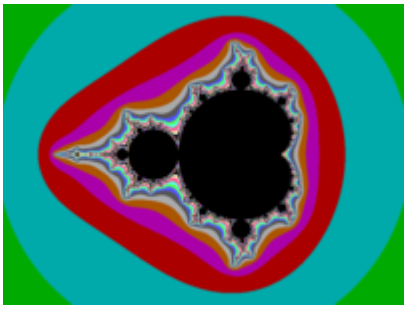
Diffusion’s random motion and optional random coloring honor rseed=, so a fixed seed makes the same image repeatable.

Diffusion was inspired by a Scientific American article a couple of years back which includes actual pictures of real physical phenomena that behave like this.

Thanks to Adrian Mariano for providing the diffusion code and documentation. Juan J. Buhler added additional options.

DivideBrot5

(type=dividebrot5)



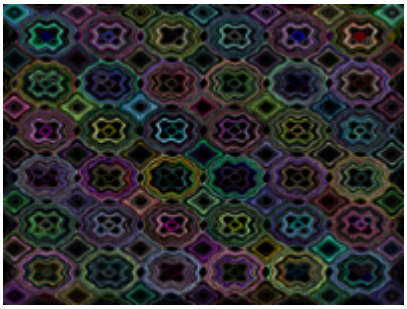
This is Jim Muth's fifth version of the DivideBrot formula.

The formula is:

```
DivideBrot5 { ; Jim Muth
   z = 0, c = pixel, a = real(p1) - 2,
   b = imag(p1) + 10^(-20) :
   z = sqr(z) / (z^(-a) + b) + c
   |z| < 16
}
```

Dynamic System

(type=dynamic)



These fractals are based on a cyclic system of differential equations:

$$x'(t) = -f(y(t))$$
$$y'(t) = f(x(t))$$

These equations are approximated by using a small time step dt, forming a time-discrete dynamic system:

$$x_{n+1} = x_n - \mathrm{d}t f(y_n)$$
$$y_{n+1} = y_n + \mathrm{d}t f(x_n)$$

The initial values x(0) and y(0) are set to various points in the plane, the dynamic system is iterated, and the resulting orbit points are plotted.

In Id, the function f is restricted to:

$$f(k) = \sin(k + a\mathrm{fn}1(bk))$$

The parameters are the spacing of the initial points, the time step dt, and the parameters (a,b,fn1) that affect the function f. Normally the orbit points are plotted individually, but for a negative spacing the points are connected.

This fractal is similar to the Pickover Popcorn. A variant is the implicit Euler approximation:

$$y_{n+1} = y_n + \mathrm{dt} f(x_n)$$
$$x_{n+1} = x_n - \mathrm{dt} f(y_{n+1})$$

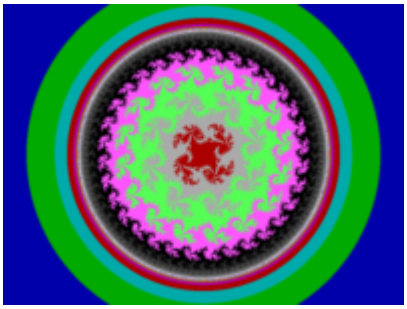
This variant results in complex orbits. The implicit Euler approximation is selected by entering dt < 0.

There are two options that have unusual effects on these fractals. The orbit delay value controls how many initial points are computed before the orbits are displayed on the screen. This allows the orbit to settle down. The outside=summ option causes each pixel to increment color every time an orbit touches it; the resulting display is a 2-d histogram.

These fractals are discussed in Chapter 14 of Pickover's "Computers, Pattern, Chaos, and Beauty".

Escher-Like Julia Sets

(type=escher_julia)



These unique variations on the Julia set theme, presented in "The Science of Fractal Images", challenge us to expand our pre-conceived notions of how fractals should be iterated. We start with a very basic Julia formula:

$$z_{n+1} = z_n^2 + (0,\ 0i)$$

The standard algorithm would test each iterated point to see if it "escapes to infinity". If its size or "modulus" (its distance from the origin) exceeds a preselected Bailout Test value, it is outside the Julia set, and it is banished to the world of multicolored level sets which color-cycle spectacularly. But another way of describing an escaped point is to say that it is "attracted" to infinity. We make this decision by calculating whether the point falls within the "target set" of all points closer to infinity than the boundary created by the bailout value. In this way, the "disk around infinity" is conceptually no different from the disks around Finite Attractors such as those used for Newton fractals.

In the above formula, with $c = (0,\ 0i)$, this standard algorithm yields a rather unexciting circle. But along comes Peitgen to tell us that "since T [the target set] can essentially be anything, this method has tremendous artistic potential. For example, T could be a so-called p-norm disk ... or a scaled

filled-in Julia set or something designed by hand. This method opens a simple [beware when he uses words like that] and systematic approach to Escher-like tilings."

So, what we do is iterate the above formula, scale each iteration, and plug it into a second Julia formula. This formula has a value of c selected by the user. If the point converges to this non-circular target set:

$$T = \{z : |(15.0z)^2 + c| \& < ; \text{BAILOUT}\}$$

we color it in proportion to the overall iteration count. If not, it will be attracted to infinity and can be colored with the usual outside coloring options. This formula uses an Id programming feature which allows the use of a customized coloring option for the points which converge to the target Julia set, yet allows the other points to be handled by the standard fractal engine with all of its options.

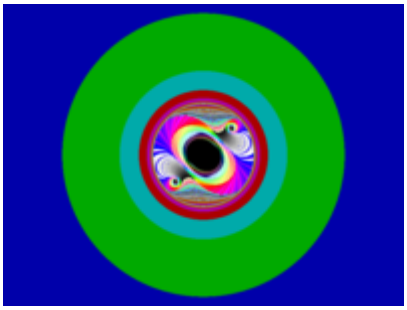
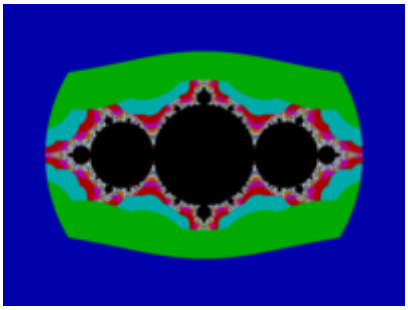
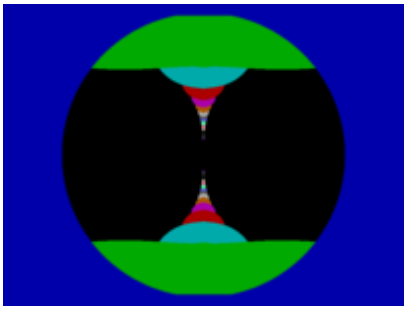
With the proper palette and parameters for c, and using the Inversion option and a solid outside color from the Color Parameters, you can create a solar eclipse, with the corona composed of Julia-shaped flames radiating from the sun's surface.

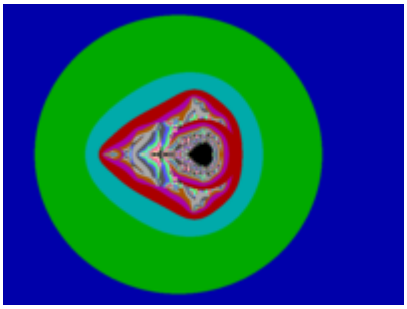
If you question the relevance of these images to Escher, check out his Circle Limit series (especially III and IV). In his own words: "It is to be doubted whether there exist today many ... artists of any kind, to whom the desire has come to penetrate to the depths of infinity.... There is only one possible way of ... obtaining an "infinity" entirely enclosed within a logical boundary line.... The largest ... shapes are now found in the center and the limit of infinite number and infinite smallness is reached at the circumference.... Not one single component ever reaches the edge. For beyond that there is "absolute nothingness." And yet this round world cannot exist without the emptiness around it, not simply because "within" presupposes "without", but also because it is out there in the "nothingness" that the center points of the arcs that go to build up the framework are fixed with such geometric exactitude."

References: Ernst, B. The Magic Mirror of M. C. Escher, Barnes & Noble, 1994, pp. 102-11. Peitgen, H.-O. and Saupe, D. The Science of Fractal Images, Springer- Verlag, 1988; pp. 185, 187.

fn||fn Fractals

(type=lambda(fn||fn), manlam(fn||fn), julia(fn||fn), mandel(fn||fn))



Two functions=[sin|cos|sinh|cosh|exp|log|sqr|...]) are specified with these types. The two functions are alternately used in the calculation based on a comparison between the modulus of the current Z and the shift value. The first function is used if the modulus of Z is less than the shift value and the second function is used otherwise.

The lambda(fn||fn) type calculates the Julia set of the formula $\lambda \mathrm{fn}(z)$, for various values of the function "fn", where lambda and z are both complex. Two values, the real and imaginary parts of lambda, should be given in the "params=" option. The third value is the shift value. The space bar will generate the corresponding "pseudo Mandelbrot" set, manlam(fn||fn).

The manlam(fn||fn) type calculates the "pseudo Mandelbrot" set of the formula $fn(z)c$ for various values of the function "fn", where c and z are both complex. Two values, the real and imaginary parts of z_(0) should be given in the "params=" option. The third value is the shift value. The space bar will generate the corresponding julia set, lamda(fn||fn).

The julia(fn||fn) type calculates the Julia set of the formula $fn(z)+c$, for various values of the function "fn", where c and z are both complex. Two values, the real and imaginary parts of c, should be given in the "params=" option. The third value is the shift value. The space bar will generate the corresponding mandelbrot set, mandel(fn||fn).

The mandel(fn||fn) type calculates the Mandelbrot set of the formula $fn(z)+c$, for various values of the function "fn", where c and z are both complex. Two values, the real and imaginary parts of z_0, should be given in the "params=" option. The third value is the shift value. The space bar will generate the corresponding julia set, julia(fn||fn).

Formula

(type=formula)

This is a "roll-your-own" fractal interpreter - you don't even need a compiler!

To run a "type=formula" fractal, you first need a text file containing formulas (there's a sample file - id.frm - included with this distribution). When you select the "formula" fractal type, Id scans the current formula file (default is id.frm) for formulas, then prompts you for the formula name you wish to run. After prompting for any parameters, the formula is parsed for syntax errors and then the fractal is generated. If you want to use a different formula file, press `F6` when you are prompted to select a formula name.

There are two command-line options that work with type=formula ("formulafile=" and "formulaname="), useful when you are using this fractal type in batch mode.

The following documentation is supplied by Mark Peterson, who wrote the formula interpreter:

Formula fractals allow you to create your own fractal formulas. The general format is:

```
Mandelbrot(XAXIS) { z = Pixel:  z = sqr(z) + pixel, |z| <= 4 }
   |         |          |                |              |
  Name     Symmetry    Initial         Iteration       Bailout
                       Condition                       Criteria
```

Initial conditions are set, then the iterations performed while the bailout criteria remains true or until 'z' turns into a periodic loop. All variables are created automatically by their usage and treated as complex. If you declare 'v = 2' then the variable 'v' is treated as a complex with an imaginary value of zero.

NOTE For periodicity checking, inside options, outside options, and the passes=o option to work correctly it is necessary to leave the result of the orbit calculation in the variable z.

Sequential processing of the formula can be altered with the flow control instructions:

```
if (expr1)
  statements
elseif (expr2)
   statements
.
.
elseif (exprn)
   statements
else
  statements
endif
```

where the expressions are evaluated and the statements executed are those immediately following the first "true" expression (the real part of the complex variable being nonzero). Nesting of if..endif blocks is permitted.

A single logical statement can be continued across multiple physical lines with the backslash (\) character. The backslash must be the last character on the physical line.

Table 1. Predefined Variables (x, y)

z	used for periodicity checking
p1	parameters 1 and 2
p2	parameters 3 and 4
p3	parameters 5 and 6
p4	parameters 7 and 8

`p5`	parameters 9 and 10
`pixel`	complex coordinates
`LastSqr`	Modulus from the last sqr() function
`rand`	Complex random number
`pi`	(3.14159..., 0.0)
`e`	(2.71828..., 0.0)
`maxit`	(maxit, 0) maximum iterations
`scrnmax`	(xdots, ydots) max horizontal/vertical resolution. e.g. for SF7 scrnmax = (1024,768)
`scrnpix`	(col, row) pixel screen coordinates. (col, row) ranges from (0,0) to (xdots-1, ydots-1)
`whitesq`	((col+row) modulo 2, 0) i.e. thinking of the screen coordinates as a large checker board, whitesq is (1,0) for the white squares and (0,0) for the black ones.
`ismand`	1 (true) by default, changes to 0 when the Mandelbrot/ Julia SPACE toggle is pressed. This allows writing formulas with both "Mandelbrot" and "Julia" behavior.
`center`	Zoom box (Xcenter, Ycenter) (see center-mag)
`magxmag`	Zoom box (Mag, Xmagnitude) (see center-mag)
`rotskew`	Zoom box (Rotation, Skew) (see center-mag)

Table 2. Operator Precedence

Precedence	Operator or Function
1	`sin()`, `cos()`, `sinh()`, `cosh()`, `cosxx()`, `tan()`, `cotan()`, `tanh()`, `cotanh()`, `sqr()`, `log()`, `exp()`, `abs()`, `conj()`, `real()`, `imag()`, `flip()`, `fn1()`, `fn2()`, `fn3()`, `fn4()`, `srand()`, `asin()`, `asinh()`, `acos()`, `acosh()`, `atan()`, `atanh()`, `sqrt()`, `cabs()`, `floor()`, `ceil()`, `trunc()`, `round()`
2	`-` (negation), `^` (power)
3	`*` (multiplication), `/` (division)
4	`+` (addition), `-` (subtraction)
5	`=` (assignment)
6	`<` (less than), `<=` (less than or equal to), `>` (greater than), `>=` (greater than or equal to), `==` (equal to), `!=` (not equal to)
7	`&&` (logical AND), `||` (logical OR)

Comparison operators compare only the real parts of their operands; imaginary parts are ignored. The result has real part 1 for true or 0 for false, and imaginary part 0.

Unlike the C or similar languages, the assignment operator '=' can only be used as an isolated statement. It cannot be used in an expression. So the exression q=3+(w=6) is a syntax error. Multiple variables can be assigned the same value in an assignment statement, e.g. z=q=6. Also unlike C-like languages the logical operators && and || do not have any "short cut" behavior; both operands to the operators are evaluated.

Precedence may be overridden by use of parenthesis. Note the modulus squared operator `|z|` is also parenthetic and always sets the imaginary component to zero. This means 'c * |z - 4|' first subtracts 4 from z, calculates the modulus squared then multiplies times 'c'. Nested modulus squared operators require overriding parenthesis:

```
c * |z + (|pixel|)|
```

The functions fn1(...) to fn4(...) are variable functions - when used, the user is prompted at run time (on the `Z` screen) to specify one of sin, cos, sinh, cosh, exp, log, sqr, etc. for each required variable function.

Most of the functions have their conventional meaning, here are a few notes on others that are not conventional.

Table 3. Function Meanings

`abs()`	returns `abs(x) + i*abs(y)`.
`|x + iy|`	returns `x*x + y*y`.
`cabs()`	returns `sqrt(x*x + y*y)`.
`conj()`	returns the complex conjugate of the argument. That is, changes sign of the imaginary component of argument: (x, y) becomes (x, -y).
`cosxx()`	duplicates a bug in the Fractint version 16 `cos()` function.
`flip()`	Swap the real and imaginary components of the complex number, e.g. (4, 5) would become (5, 4).
`ident()`	identity function. Leaves the value of the argument unchanged, acting like a "z" term in a formula.
`zero()`	returns 0.
`one()`	returns 1.
`floor()`	largest integer not greater than the argument `floor(x + iy) = floor(x) + i*floor(y)`.
`ceil()`	smallest integer not less than the argument.
`trunc()`	truncate fraction part toward zero.
`round()`	round to nearest integer or up, e.g. round(2.5, 3.4) = (3, 3).

The 'rand' predefined variable is changed with each iteration to a new random number with the real and imaginary components containing a value between zero and 1. Use the srand() function to initialize the random numbers to a consistent random number sequence. If a formula does not contain the srand() function, then the formula compiler will use the system time to initialize the sequence. This could cause a different fractal to be generated each time the formula is used depending on how the formula is written.

The rounding functions must be used cautiously; formulas that depend on exact values of numbers will not work reliably in all cases. For example, in floating point mode, trunc(6/3) returns 1 while trunc(6/real(3)) returns 2.

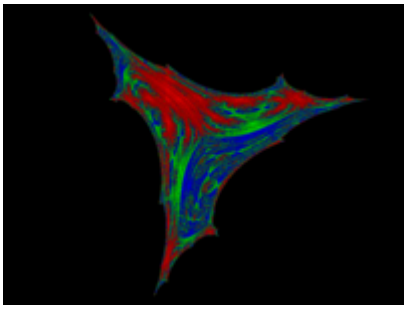
Note that if x is an integer, floor(x) = ceil(x) = trunc(x) = round(x) = x. The possible values for symmetry are: XAXIS, XAXIS_NOPARM YAXIS, YAXIS_NOPARM XYAXIS, XYAXIS_NOPARM ORIGIN, ORIGIN_NOPARM PI_SYM, PI_SYM_NOPARM XAXIS_NOREAL XAXIS_NOIMAG

These will force the symmetry even if no symmetry is actually present, so try your formulas without symmetry before you use these.

For mathematical formulas of functions used in the parser language, see Trig Identities

Frothy Basins

(type=frothybasin)



Frothy basins, or riddled basins, were discovered by James C. Alexander of the University of Maryland. The discussion below is derived from a two page article entitled "Basins of Froth" in Science News, November 14, 1992 and from correspondence with others, including Dr. Alexander.

The equations that generate this fractal are not very different from those that generate many other orbit fractals.

$$z_0 = \text{pixel}$$
$$z_{n+1} = z_n^2 - C\text{conj}(z_n)$$

One of the things that makes this fractal so interesting is the shape of the dynamical system's attractors. It is not at all uncommon for a dynamical system to have non-point attractors. Shapes such as circles are very common. Strange attractors are attractors which are themselves fractal. What is unusual about this system, however, is that the attractors intersect. This is the first case in which such a phenomenon has been observed. The attractors for this system are made up of line segments which overlap to form an equilateral triangle. This attractor triangle can be seen by using the "show orbits" option (the `O` key) or the "orbits window" option (the `Ctrl` + `O` key).

The number of attractors present is dependent on the value of A, the imaginary part of C. For values where A ⇐ 1.028713768218725..., there are three attractors. When A is larger than this critical value, two of attractors merge into one, leaving only two attractors. An interesting variation on this fractal can be generated by applying the above mapping twice per each iteration. The result is that some of the attractors are split into two parts, giving the system either six or three attractors, depending on whether A is less than or greater than the critical value.

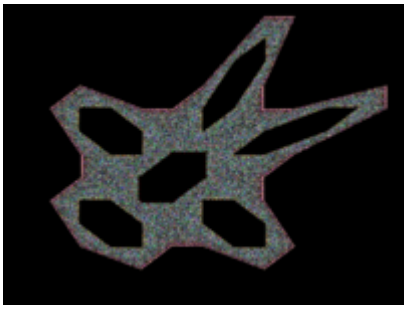
These are also called "riddled basins" because each basin is riddled with holes. Which attractor a point is eventually pulled into is extremely sensitive to its initial position. A very slight change in any direction may cause it to end up on a different attractor. As a result, the basins are thoroughly

intermingled. The effect appears to be a frothy mixture that has been subjected to lots of stirring and folding.

Pixel color is determined by which attractor captures the orbit. The shade of color is determined by the number of iterations required to capture the orbit. In Id, the actual shade of color used depends on how many colors are available in the video mode being used. If 256 colors are available, the default coloring scheme is determined by the number of iterations that were required to capture the orbit. An alternative coloring scheme can be used where the shade is determined by the iterations required divided by the maximum iterations. This method is especially useful on deeply zoomed images. If only 16 colors are available, then only the alternative coloring scheme is used. If fewer than 16 colors are available, then Id just colors the basins without any shading.

Gingerbreadman

(type=gingerbreadman)



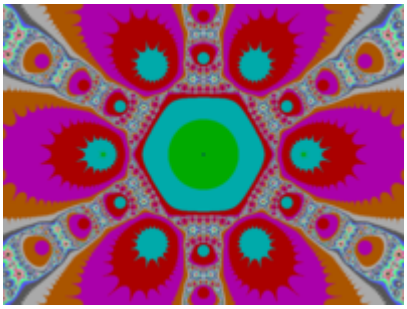
This simple fractal is a charming example stolen from "Science of Fractal Images", p. 149.

$$x_{\text{new}} = 1 - y + |x|$$
$$y_{\text{new}} = x$$

The initial x and y values are set by parameters, defaults x=-.1, y = 0.

Halley

(type=halley)

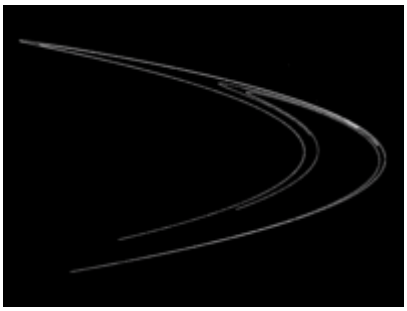


The Halley map is an algorithm used to find the roots of polynomial equations by successive "guesses" that converge on the correct value as you feed the results of each approximation back into the formula. It works very well — unless you are unlucky enough to pick a value that is on a line between two actual roots. In that case, the sequence explodes into chaos, with results that diverge more and more wildly as you continue the iteration.

This fractal type shows the results for the polynomial $z(z^a-1)$, which has $a+1$ roots in the complex plane. Use the Type command and enter "halley" in response to the prompt. You will be asked for a parameter, the "order" of the equation (an integer from 2 through 10 — 2 for $z(z^2-1)$, 7 for $z(z^7-1)$, etc.). A second parameter is the relaxation coefficient, and is used to control the convergence stability. A number greater than one increases the chaotic behavior and a number less than one decreases the chaotic behavior. The third parameter is the value used to determine when the formula has converged. The test for convergence is $\|z(n+1)|\hat{}2-|z(n)|\hat{}2|<\epsilon$. This convergence test produces the whisker-like projections which generally point to a root.

Henon Attractors

(type=henon)



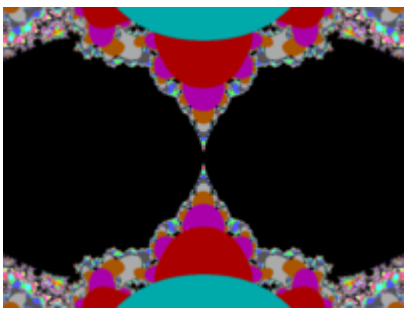
Michel Henon was an astronomer at Nice observatory in southern France. He came to the subject of fractals via investigations of the orbits of astronomical objects. The strange attractor most often linked with Henon's name comes not from a differential equation, but from the world of discrete mathematics - difference equations. The Henon map is an example of a very simple dynamic system that exhibits strange behavior. The orbit traces out a characteristic banana shape, but on close inspection, the shape is made up of thicker and thinner parts. Upon magnification, the thicker bands resolve to still other thick and thin components. And so it goes forever! The equations that generate this strange pattern perform the mathematical equivalent of repeated stretching and folding, over and over again.

$$x_{\text{new}} = 1+y-ax^2$$
$$y_{\text{new}} = bx$$

The default parameters are a=1.4 and b=.3.

HyperComplex

(type=hypercomplex,hypercomplexj)



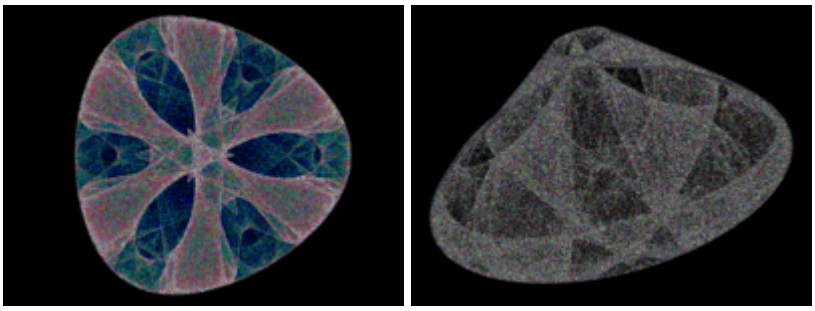

These fractals are based on hypercomplex numbers, which like quaternions are a four dimensional generalization of complex numbers. It is not possible to fully generalize the complex numbers to four dimensions without sacrificing some of the algebraic properties shared by real and complex numbers. Quaternions violate the commutative law of multiplication, which says $z_1 z_2 = z_2 z_1$. Hypercomplex numbers fail the rule that says all non-zero elements have multiplicative inverses - that is, if z is not 0, there should be a number $\frac{1}{z}$ such that $\left(\frac{1}{z}\right)z = 1$. This law holds most of the time but not all the time for hypercomplex numbers.

However hypercomplex numbers have a wonderful property for fractal purposes. Every function defined for complex numbers has a simple generalization to hypercomplex numbers. Id's implementation takes advantage of this by using "fn" variables - the iteration formula is $h_{n+1} = \mathrm{fn}(h_n) + c$. where "fn" is the hypercomplex generalization of sin, cos, log, sqr etc. You can see 3D versions of these fractals using fractal type Julibrot. Hypercomplex numbers were brought to our attention by Clyde Davenport, author of "A Hypercomplex Calculus with Applications to Relativity" (ISBN 0-9623837-0-8).

See also Quaternion and Quaternion and Hypercomplex Algebra

Icon

(type=icons, icons3d)



This fractal type was inspired by the book "Symmetry in Chaos" by Michael Field and Martin Golubitsky (ISBN 0-19-853689-5, Oxford Press).

To quote from the book's jacket,

> Field and Golubitsky describe how a chaotic process eventually can lead to symmetric patterns (in a river, for instance, photographs of the turbulent movement of eddies, taken over time, often reveal patterns on the average.

The icon type implemented here maps the classic population logistic map of bifurcation fractals onto the complex plane in Dn symmetry.

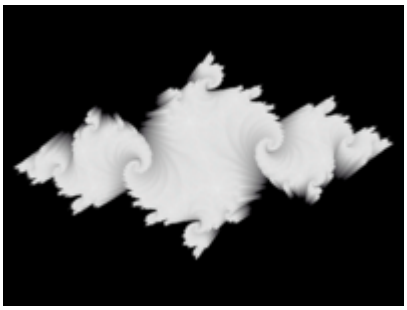
The initial points plotted are the more chaotic initial orbits, but as you wait, delicate webs will begin to form as the orbits settle into a more periodic pattern. Since pixels are colored by the number of times they are hit, the more periodic paths will become clarified with time. These fractals run continuously.

There are 6 parameters: Lambda, Alpha, Beta, Gamma, Omega, and Degree

Omega	0	D_n, or dihedral (rotation + reflectional) symmetry
	!0	Z_n, or cyclic (rotational) symmetry
Degree	n	Degree of symmetry

Julibrots

(type=julibrot)



The Julibrot fractal type uses a general-purpose renderer for visualizing three dimensional solid fractals. Originally Mark Peterson developed this rendering mechanism to view a 3-D sections of a 4-D structure he called a "Julibrot". This structure, also called "layered Julia set" in the fractal literature, hinges on the relationship between the Mandelbrot and Julia sets. Each Julia set is created using a fixed value c in the iterated formula $z^2 + c$. The Julibrot is created by layering Julia sets in the x-y plane and continuously varying c, creating new Julia sets as z is incremented. The solid shape thus created is rendered by shading the surface using a brightness inversely proportional to the virtual viewer's eye.

The Julibrot engine can be used with other Julia formulas besides the classic $z^2 + c$. The first field on the Julibrot parameter screen lets you select which orbit formula to use.

You can also use the Julibrot renderer to visualize 3D cross sections of true four dimensional Quaternion and Hypercomplex fractals.

The Julibrot Parameter Screens

Orbit Algorithm - select the orbit algorithm to use. The available possibilities include 2-D Julia and both mandelbrot and Julia variants of the 4-D Quaternion and Hypercomplex fractals.

Orbit parameters - the next screen lets you fill in any parameters belonging to the orbit algorithm. This list of parameters is not necessarily the same as the list normally presented for the orbit algorithm, because some of these parameters are used in the Julibrot layering process.

```
From/To Parameters These parameters allow you to specify the
"Mandelbrot" values used to generate the layered Julias.  The
parameter c in the Julia formulas will be incremented in steps
ranging from the "from" x and y values to the "to" x and y values.
If the orbit formula is one of the "true" four dimensional fractal
types quat, quatj, hypercomplex, or hypercomplexj, then these numbers
are used with the 3rd and 4th dimensional values.
```

```
The "from/to" variables are different for the different kinds of
orbit algorithm.
```

```
2D Julia sets - complex number formula z' = f(z) + c
   The "from/to" parameters change the values of c.
4D Julia sets - Quaternion or Hypercomplex formula z' = f(z) + c
   The four dimensions of c are set by the orbit parameters.
   The first two dimensions of z are determined by the corners values.
   The third and fourth dimensions of z are the "to/from" variables.
4D Mandelbrot sets - Quaternion or Hypercomplex formula z' = f(z) + c
   The first two dimensions of c are determined by the corners values.
```

```
The third and fourth dimensions of c are the "to/from" variables.
```

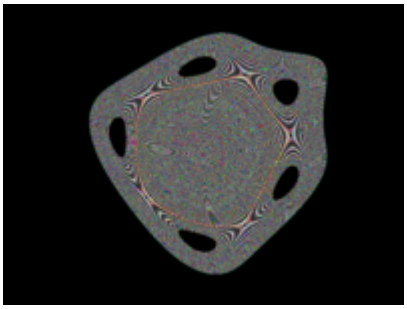
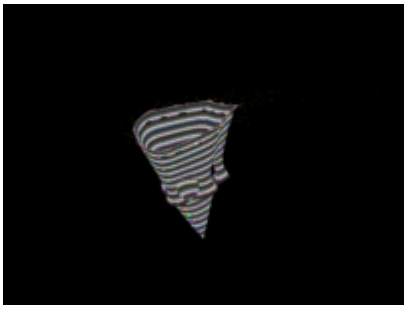
Distance between the eyes - set this to 2.5 if you want a red/blue anaglyph image, 0 for a normal grayscale image.

Number of z pixels - this sets how many layers are rendered in the screen z-axis. Use a higher value with higher resolution video modes.

The remainder of the parameters are needed to construct the red/blue picture so that the fractal appears with the desired depth and proper 'z' location. With the origin set to 8 inches beyond the screen plane and the depth of the fractal at 8 inches the default fractal will appear to start at 4 inches beyond the screen and extend to 12 inches if your eyeballs are 2.5 inches apart and located at a distance of 24 inches from the screen. The screen dimensions provide the reference frame.

Kam Torus

(type=kamtorus, kamtorus3d)

This type is created by superimposing orbits generated by a set of equations, with a variable incremented each time.

$$x_0 = y_0 = \frac{\text{orbit}}{3}$$
$$x_{n+1} = x_n \cos(a) + (x_n^2 - y_n)\sin(a)$$
$$y_{n+1} = x_n \sin(a) - (x_n^2 - y_n)\cos(a)$$

After each orbit, 'orbit' is incremented by a step size. The parameters are angle "a", step size for incrementing 'orbit', stop value for 'orbit', and points per orbit. Try this with a stop value of 5 with sound=x for some weird fractal music (ok, ok, fractal noise)! You will also see the KAM Torus head into some chaotic territory that Scott Taylor wanted to hide from you by setting the defaults the way he did, but now we have revealed all!

The 3D variant is created by treating 'orbit' as the z coordinate.

With both variants, you can adjust the "maxiter" value (`X` options screen or parameter maxiter=) to change the number of orbits plotted.

If you're wondering where KAM comes from in the name of this fractal, it comes from the Kolmogorov-Arnold-Moser theorem. This theorem is a result in dynamical systems about the persistence of quasiperiodic motions under small perturbations. The problem is whether or not a small perturbation of a conservative dynamical system results in a lasting quasiperiodic orbit. The original breakthrough to this problem was given by Andrey Kolmogorov in 1954. This was rigorously proved and extended by Jurgen Moser in 1962 (for smooth twist maps) and Vladimir Arnold in 1963 (for analytic Hamiltonian systems), and the general result is known as the KAM theorem. Quasiperiodic orbits are more affectionatly known as chaotic orbits! (Thanks, Wikipedia!)

L-Systems

(type=lsystem)

In 1968, the Hungarian biologist Aristid Lindenmayer developed a formal language for describing the growth of plant cell structures. Lindenmayer used his formal language to describe the growth patterns of various types of algae. Later, the formal language was used to describe the branching structures of higher plants. The work was popularized among fractal enthusiasts in the book "The Algorithmic Beauty of Plants" (1990), by Przemyslaw Prusinkiewicz and Aristid Lindenmayer. Lindenmayer died in 1989, but his work was continued and extended by Prusinkiewicz whose main work consists of using Lindenmayer grammars to model plant growth.

A Lindenmayer system, or L-system for short, consists of a set of symbols, a grammar, an interpretation scheme for the symbols and an initial string of symbols, called the axiom. The interpretation scheme operates on a string of symbols to create a picture. The grammar says how symbols in the string are replaced in each generation of the system. Starting with the initial string of symbols, the rules of the grammar are applied to all the symbols in the string to produce a new string of symbols. Each application of the grammar rules is called a generation. Well, that's enough theory!

These fractals are constructed from line segments using rules specified in drawing commands. Starting with an initial string, the axiom, transformation rules are applied a specified number of times, to produce the final command string which is used to draw the image.

Like the type=formula fractals, this type requires a separate data file. A sample file, id.l, is included with this distribution. When you select type lsystem, the current L-system file is read and you are asked for the L-system name you wish to run. Press `F6` at this point if you wish to use a different L-system file. After selecting an L-system, you are asked for one parameter - the "order", or number of times to execute all the transformation rules. It is wise to start with small orders, because the size of

the substituted command string grows exponentially and it is very easy to exceed your resolution. (Higher orders take longer to generate too.) The command line options "lname=" and "lfile=" can be used to over-ride the default file name and L-system name.

Each L-system entry in the file contains a specification of the angle, the axiom, and the transformation rules. Each item must appear on its own line and each line must be less than 160 characters long.

The statement "angle n" sets the angle to 360/n degrees; n must be an integer greater than two and less than fifty.

"Axiom string" defines the axiom.

Transformation rules are specified as "a=string" and convert the single character 'a' into "string." If more than one rule is specified for a single character all of the strings will be added together. This allows specifying transformations longer than the 160 character limit. Transformation rules may operate on any characters except space, tab or '}'.

Any information after a ; (semi-colon) on a line is treated as a comment.

Here is a sample L-system:

```
Dragon {         ; Name of L-system, { indicates start
  Angle 8        ; Specify the angle increment to 45 degrees
  Axiom FX       ; Starting character string
  F=             ; First rule:  Delete 'F'
  y=+FX--FY+     ; Change 'y' into  "+fx--fy+"
  x=-FX++FY-     ; Similar transformation on 'x'
}                ; final } indicates end
```

```
The standard drawing commands are:
  F Draw forward
  G Move forward (without drawing)
  + Increase angle
  - Decrease angle
  | Try to turn 180 degrees. (If angle is odd, the turn
    will be the largest possible turn less than 180 degrees.)
```

These commands increment angle by the user specified angle value. They should be used when possible because they are fast. If greater flexibility is needed, use the following commands which keep a completely separate angle pointer which is specified in degrees.

```
D   Draw forward
M   Move forward
\nn Increase angle nn degrees
/nn Decrease angle nn degrees
```

```
Color control:
  Cnn Select color nn
  <nn Increment color by nn
  >nn decrement color by nn
```

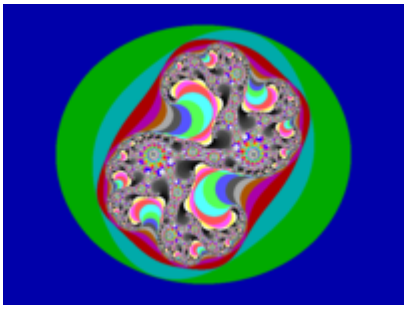
```
Advanced commands:
  !     Reverse directions (Switch meanings of +, - and , /)
  @nnn  Multiply line segment size by nnn
        nnn may be a plain number, or may be preceded by
            I for inverse, or Q for square root.
            (e.g.  @IQ2 divides size by the square root of 2)
  [     Push.  Stores current angle and position on a stack
  ]     Pop.  Return to location of last push
```

Other characters are perfectly legal in command strings. They are ignored for drawing purposes, but can be used to achieve complex translations.

The characters '+', '-', '<', '>', '[', ']', '|', '!', '@', '/', '\', and 'c' are reserved symbols and cannot be redefined. For example, c=f+f and ⇐, are syntax errors.

Lambda Sets

(type=lambda)



This type calculates the Julia set of the formula $\lambda z(1-z)$. That is, the value z_0 is initialized with the value corresponding to each pixel position, and the formula iterated. The pixel is colored according to the iteration when the sum of the squares of the real and imaginary parts exceeds 4.

Two parameters, the real and imaginary parts of lambda, are required. Try 0 and 1 to see the classical fractal "dragon". Then try 0.2 and 1 for a lot more detail to zoom in on.

It turns out that all quadratic Julia-type sets can be calculated using just the formula z^2+c (the "classic" Julia"), so that this type is redundant, but we include it for reason of it's prominence in the history of fractals.

Lambdafn

(type=lambdafn)



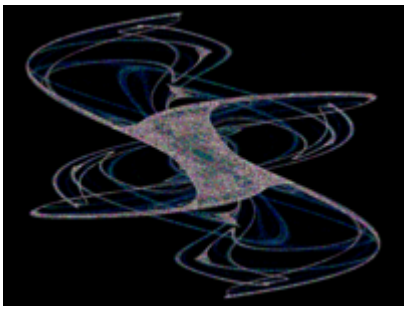
Function=[sin|cos|sinh|cosh|exp|log|sqr|...]) is specified with this type. In the following description, we will use the shorthand "lambdasine" for "lambdafn with function=sin".

These types calculate the Julia set of the formula lambda*fn(Z), for various values of the function "fn", where lambda and Z are both complex. Two values, the real and imaginary parts of lambda, should be given in the "params=" option. For the feathery, nested spirals of LambdaSines and the frost-on-glass patterns of LambdaCosines, make the real part = 1, and try values for the imaginary part ranging from 0.1 to 0.4 (hint: values near 0.4 have the best patterns). In these ranges the Julia set "explodes". For the tongues and blobs of LambdaExponents, try a real part of 0.379 and an imaginary part of 0.479.

Each LambdaSine/Cosine iteration calculates a hyperbolic sine, hyperbolic cosine, a sine, and a cosine (the LambdaExponent iteration "only" requires an exponent, sine, and cosine operation)!

Latoocarfian

(type=latoocarfian)



This fractal type first appeared in the book "Chaos in Wonderland" by Clifford Pickover (ISBN 0-312-10743-9 St. Martin’s Press).

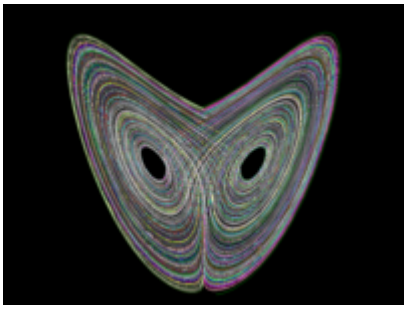
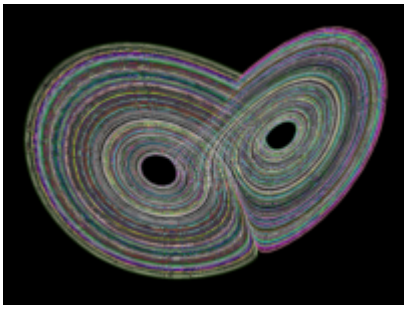
The Latoocarfians are beings that inhabit the moon Ganymede (of Jupiter) and have their forms generated by these formulas.

The initial points plotted are the more chaotic initial orbits, but as you wait, delicate webs will begin to form as the orbits settle into a more periodic pattern. Since pixels are colored by the number of times they are hit, the more periodic paths will become clarified with time.

There are 4 parameters: a, b, c, d and we recomend: a > -3, b < 3, c > 0.5, d < 1.5

Lorenz Attractors

(type=lorenz, lorenz3d)

The "Lorenz attractor" is a "simple" set of three deterministic equations developed by Edward Lorenz while studying the non- repeatability of weather patterns. The weather forecaster's basic problem is that even very tiny changes in initial patterns ("the beating of a butterfly's wings" - the official term is "sensitive dependence on initial conditions") eventually reduces the best weather forecast to rubble.

The Lorenz attractor is the plot of the orbit of a dynamic system consisting of three first order non-linear differential equations. The solution to the differential equation is vector-valued function of one variable. If you think of the variable as time, the solution traces an orbit. The orbit is made up of two spirals at an angle to each other in three dimensions. We change the orbit color as time goes on to add a little dazzle to the image. The equations are:

$$\frac{dx}{dt} = -ax + ay$$
$$\frac{dy}{dt} = bx - y - zx$$
$$\frac{dz}{dt} = -cz + xy$$

We solve these differential equations approximately using a method known as the first order taylor series. Calculus teachers everywhere will kill us for saying this, but you treat the notation for the derivative dx/dt as though it really is a fraction, with "dx" the small change in x that happens when the time changes "dt". So multiply through the above equations by dt, and you will have the change in the orbit for a small time step. We add these changes to the old vector to get the new vector after one step. This gives us:

$$x_{\text{new}} = x - axdt + aydt$$
$$y_{\text{new}} = y + bxdt - ydt - xzdt$$

```
(default values: dt = .02, a = 5, b = 15, c = 1)
```

We connect the successive points with a line, project the resulting 3D orbit onto the screen, and voila! The Lorenz attractor!

We have added two versions of the Lorenz attractor. "Type=lorenz" is the Lorenz attractor as seen

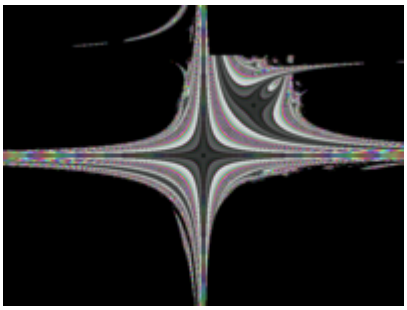
in everyday 2D. "Type=lorenz3d" is the same set of equations with the added twist that the results are run through our perspective 3D routines, so that you get to view it from different angles (you can modify your perspective "on the fly" by using the `I` command.) If you set the "stereo" option to "2", and have red/blue funny glasses on, you will see the attractor orbit with depth perception.

Hint: the default perspective values (x = 60, y = 30, z = 0) aren't the best ones to use for fun Lorenz attractor viewing. Experiment a bit - start with rotation values of 0/0/0 and then change to 20/0/0 and 40/0/0 to see the attractor from different angles.- and while you're at it, use a non-zero perspective point. Try 100 and see what happens when you get **inside** the Lorenz orbits. Here comes one - Duck! While you are at it, turn on the sound with the "X". This way you'll at least hear it coming!

Different Lorenz attractors can be created using different parameters. Four parameters are used. The first is the time-step (dt). The default value is .02. A smaller value makes the plotting go slower; a larger value is faster but rougher. A line is drawn to connect successive orbit values. The 2nd, third, and fourth parameters are coefficients used in the differential equation (a, b, and c). The default values are 5, 15, and 1. Try changing these a little at a time to see the result.

Lyapunov Fractals

(type=lyapunov)



The bifurcation fractal illustrates what happens in a simple population model as the growth rate increases. The Lyapunov fractal expands that model into two dimensions by letting the growth rate vary in a periodic fashion between two values. Each pair of growth rates is run through a logistic population model and a value called the Lyapunov exponent is calculated for each pair and is plotted. The Lyapunov exponent is calculated by adding up $\log(|r - 2rx|)$ over many cycles of the population model and dividing by the number of cycles. Negative Lyapunov exponents indicate a stable, periodic behavior and are plotted in color. Positive Lyapunov exponents indicate chaos (or a diverging model) and are colored black.

Order parameter: Each possible periodic sequence yields a two dimensional space to explore. The order parameter selects a sequence. The default value 0 represents the sequence ab which alternates between the two values of the growth parameter. On the screen, the a values run vertically and the b values run horizontally. Here is how to calculate the space parameter for any desired sequence. Take your sequence of a's and b's and arrange it so that it starts with at least 2 a's and ends with a b. It may be necessary to rotate the sequence or swap a's and b's. Strike the first a and the last b off the list and replace each remaining a with a 1 and each remaining b with a zero. Interpret this as a binary number and convert it into decimal.

An example: I like sonnets. A sonnet is a poem with fourteen lines that has the following rhyming

sequence: abba abba abab cc. Ignoring the rhyming couplet at the end, let's calculate the order parameter for this pattern.

```
abbaabbaabab          doesn't start with at least 2 a's
aabbaababab           rotate it
1001101010            drop the first and last, replace with 0's and 1's
```

```
512+64+32+8+2 = 618
```

An order parameter of 618 gives the Lyapunov equivalent of a sonnet. "How do I make thee? Let me count the ways..."

Population Seed: When two parts of a Lyapunov overlap, which spike overlaps which is strongly dependent on the initial value of the population model. Any changes from using a different starting value between 0 and 1 may be subtle. The values 0 and 1 are interpreted in a special manner. A seed of 1 will choose a random number between 0 and 1 at the start of each pixel. A seed of 0 will suppress resetting the seed value between pixels unless the population model diverges in which case a random seed will be used on the next pixel.

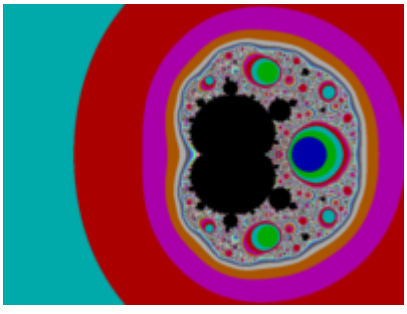
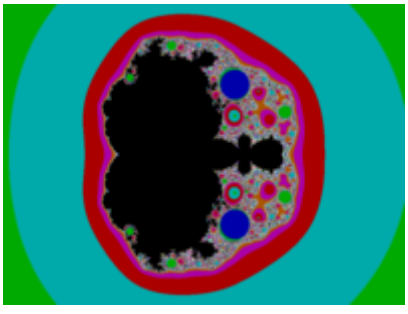
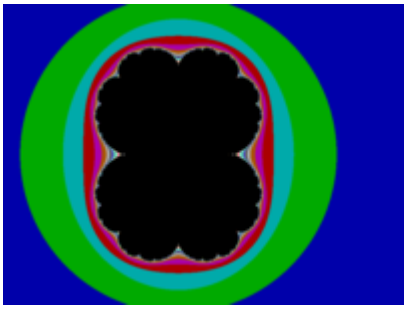
These random population choices honor rseed=, so a fixed rseed= makes the random population sequence repeatable.

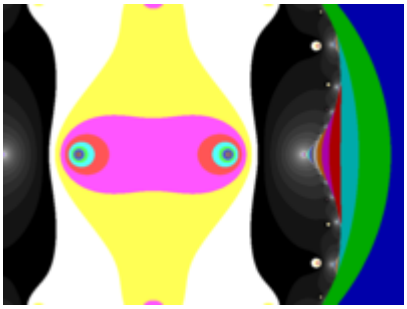
Filter Cycles: Like the Bifurcation model, the Lyapunov allow you to set the number of cycles that will be run to allow the model to approach equilibrium before the lyapunov exponent calculation is begun. The default value of 0 uses one half of the iterations before beginning the calculation of the exponent.

Reference: A.K. Dewdney, Mathematical Recreations, Scientific American, Sept. 1991

Magnetic Fractals

(type=magnet1m, magnet2m, magnet1j, magnet2j)



These fractals use formulae derived from the study of hierarchical lattices, in the context of magnetic renormalisation transformations. This kinda stuff is useful in an area of theoretical physics that deals with magnetic phase-transitions (predicting at which temperatures a given substance will be magnetic, or non-magnetic). In an attempt to clarify the results obtained for real temperatures (the kind that you and I can feel), the study moved into the realm of complex numbers, aiming to spot real phase-transitions by finding the intersections of lines representing complex phase-transitions with the real axis. The first people to try this were two physicists called Yang and Lee, who found the situation a bit more complex than first expected, as the phase boundaries for complex temperatures are (surprise!) fractals.

And that's all the technical (?) background you're getting here! For more details (are you serious?!) read "The Beauty of Fractals". When you understand it all, you might like to rewrite this section, before you start your new job as a professor of theoretical physics...

In Id terms, the important bits of the above are "fractals", "complex numbers", "formulae", and "The Beauty of Fractals". Lifting the formulae straight out of the book and iterating them over the complex plane (just like the Mandelbrot set) produces fractals.

The formulae are a bit more complicated than the $z^2 + c$ used for the Mandelbrot set, that's all. They are :

$$\text{magnet1} : \left(\frac{z^2 + c - 1}{2z + c - 2} \right)^2$$

$$\text{magnet2} : \left(\frac{z^3 + 3(c-1)z + (c-1)(c-2)}{3z^2 + 3(c-2)z + (c-1)(c-2) + 1} \right)^2$$

These aren't quite as horrific as they look (oh yeah ?!) as they only involve two variables (z and c), but cubing things, doing division, and eventually squaring the result (all in complex numbers) don't exactly spell s-p-e-e-d ! These are not the fastest fractals in Id!

As you might expect, for both formulae there is a single related Mandelbrot-type set (magnet1m, magnet2m) and an infinite number of related Julia-type sets (magnet1j, magnet2j), with the usual toggle between the corresponding Ms and Js via the spacebar.

If you fancy delving into the Julia-types by hand, you will be prompted for the real and imaginary parts of the parameter denoted by C. The result is symmetrical about the real axis (and therefore the initial image gets drawn in half the usual time) if you specify a value of zero for the imaginary part of C.

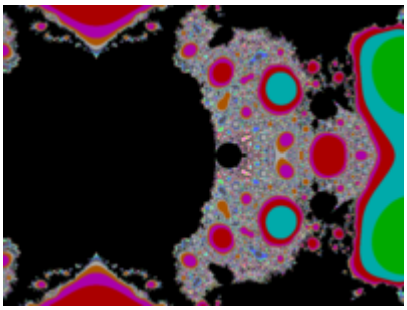
Id historical note: Another complication (besides the formulae) in implementing these fractal types

was that they all have a finite attractor (1.0 + 0.0i), as well as the usual one at infinity. This fact spurred the development of finite attractor logic in Id. Without this code you can still generate these fractals, but you usually end up with a pretty boring image that is mostly deep blue "lake", courtesy of Id's standard Periodicity Logic. See Finite Attractors for more information on this aspect of Id internals.

(Thanks to Kevin Allen for magnetic type documentation above).

Mandelbrot Mix 4

(type=mandelbrotmix4)



Jim Muth published a "Fractal of the Day" on the Fractint mailing list for many years. As often as not Jim picks the formula Mandelbrotmix4 as the fractal continent to explore. To honor Jim, the Id authors have provided this fractal as a built-in type.

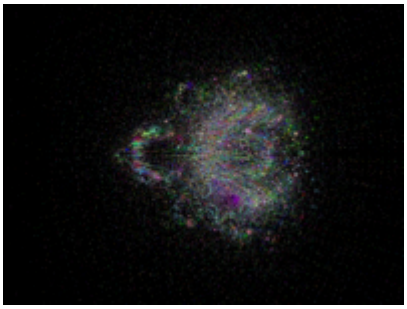
The formula is:

```
MandelbrotMix4 {; Jim Muth
  a=real(p1), b=imag(p1), d=real(p2), f=imag(p2),
    g=1/f, h=1/d, j=1/(f-b), z=(-a*b*g*h)^j,
    k=real(p3)+1, l=imag(p3)+100, c=fn1(pixel) :
  z=k*((a*(z^b))+(d*(z^f)))+c,
  |z| < l
}
```

Note that Jim uses l=imag(p3)+100, which is to say, the sixth scalar parameter, as the bailout. Our implementation follows Jim if the user requests the default bailout.

Mandelcloud

(type=mandelcloud)



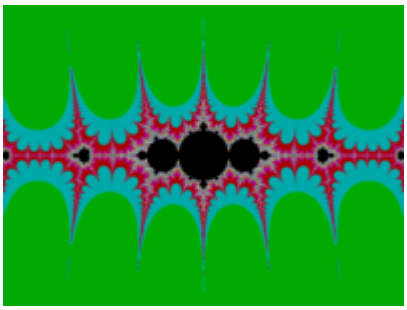
This fractal computes the Mandelbrot function, but displays it differently. It starts with regularly spaced initial pixels and displays the resulting orbits. This idea is somewhat similar to the Dynamic System.

There are two options that have unusual effects on this fractal. The orbit delay value controls how many initial points are computed before the orbits are displayed on the screen. This allows the orbit to settle down. The outside=summ option causes each pixel to increment color every time an orbit touches it; the resulting display is a 2-d histogram.

This fractal was invented by Noel Giffin.

Mandelfn

(type=mandelfn)



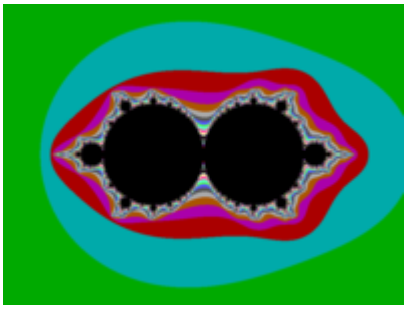
Function=[sin|cos|sinh|cosh|exp|log|sqr|...]) is specified with this type.

These are "pseudo-Mandelbrot" mappings for the LambdaFn Julia functions. They map to their corresponding Julia sets via the spacebar command in exactly the same fashion as the original M/J sets. In general, they are interesting mainly because of that property (the function=exp set in particular is rather boring). Generate the appropriate "Mandelfn" set, zoom on a likely spot where the colors are changing rapidly, and hit the spacebar key to plot the Julia set for that particular point.

Try "type=mandelfn corners=4.68/4.76/-.03/.03 function=cos" with the `G` command for a graphic demonstration that we're not taking Mandelbrot's name in vain here. We didn't even know these little buggers were here until Mark Peterson found them!

Mandellambda Sets

(type=mandellambda)



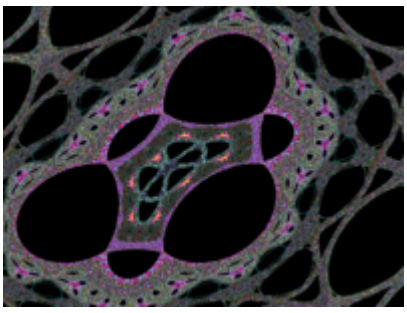
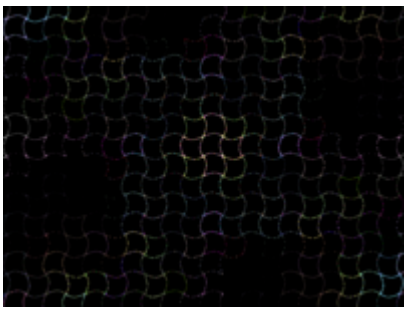
This type is the "Mandelbrot equivalent" of the lambda set. A comment is in order here. Almost all the Id "Mandelbrot" sets are created from orbits generated using formulas like $z_{n+1} = f(z_n, c)$, with z_0 and c initialized to the complex value corresponding to the current pixel. Our reasoning was that "Mandelbrots" are maps of the corresponding "Julias". Using this scheme each pixel of a "Mandelbrot" is colored the same as the Julia set corresponding to that pixel. However, Kevin Allen informs us that the mandellambda set appears in the literature with z_0 initialized to a critical point (a point where the derivative of the formula is zero), which in this case happens to be the point (.5,0). Since Kevin knows more about Dr. Mandelbrot than we do, and Dr. Mandelbrot knows more about fractals than we do, we defer! Id calculates mandelambda Dr. Mandelbrot's way instead of our way. But all the other "Mandelbrot" sets in Id are still calculated our way! (Fortunately for us, for the classic Mandelbrot Set these two methods are the same!)

Well now, folks, apart from questions of faithfulness to fractals named in the literature (which we do take seriously!), if a formula makes a beautiful fractal, it is not wrong. In fact some of the best fractals in Id are the results of mistakes! Nevertheless, thanks to Kevin for keeping us accurate!

(See description of "initorbit=" command in Image Calculation Parameters for a way to experiment with different orbit initializations).

Martin Attractors

(type=hopalong, martin, chip, quadruptwo, threeply)

These fractal types are from A. K. Dewdney's "Computer Recreations" column in "Scientific American". They are attributed to Barry Martin of Aston University in Birmingham, England.

Hopalong is an "orbit" type fractal like lorenz. The image is obtained by iterating this formula after setting $z_0 = y_0 = 0$:

$$x_{n+1} = y_n - \text{sign}(x_n)\sqrt{\text{abs}\, bx_n - c}$$
$$y_{n+1} = a - x_n$$

Parameters are a, b, and c. The function "sign()" returns 1 if the argument is positive, -1 if argument is negative.

This fractal continues to develop in surprising ways after many iterations.

Another Martin fractal is simpler. The iterated formula is:

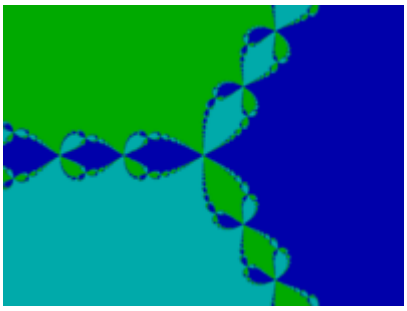
$$x_{n+1} = y_n - \sin(x_n)$$
$$y_{n+1} = a - x_n$$

The parameter is "a". Try values near the number π.

Michael Peters has based the HOP program on variations of these Martin types. You will find three of these here: chip, quadruptwo, and threeply.

Newton Domains of Attraction

(type=newtbasin)



The Newton formula is an algorithm used to find the roots of polynomial equations by successive "guesses" that converge on the correct value as you feed the results of each approximation back into the formula. It works very well — unless you are unlucky enough to pick a value that is on a line between two actual roots. In that case, the sequence explodes into chaos, with results that diverge more and more wildly as you continue the iteration.

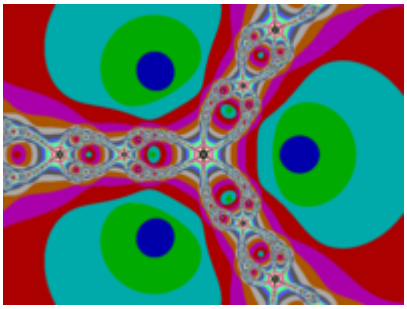
This fractal type shows the results for the polynomial $Z^n - 1$, which has n roots in the complex plane. Use the Type command and enter "newtbasin" in response to the prompt. You will be asked for a parameter, the "order" of the equation (an integer from 3 through 10 — 3 for $x^3 - 1$, 7 for $x^7 - 1$, etc.). A second parameter is a flag to turn on alternating shades showing changes in the number of iterations needed to attract an orbit. Some people like stripes and some don't, as always, Id gives you a choice!

The coloring of the plot shows the "basins of attraction" for each root of the polynomial — i.e., an initial guess within any area of a given color would lead you to one of the roots. As you can see, things get a bit weird along certain radial lines or "spokes", those being the lines between actual roots. By "weird", we mean infinitely complex in the good old fractal sense. Zoom in and see for yourself.

This fractal type is symmetric about the origin, with the number of "spokes" depending on the order you select.

Newton

(type=newton)



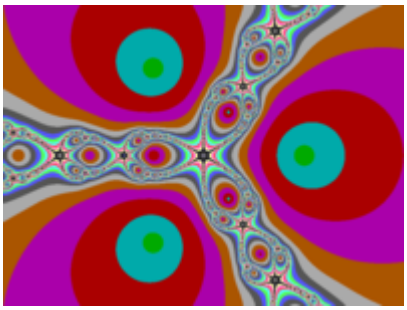
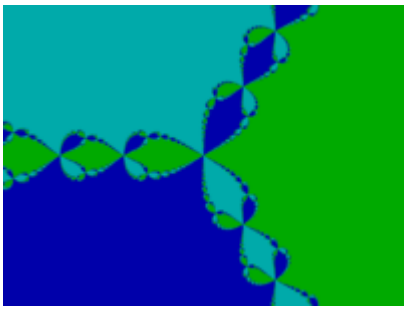
The generating formula here is identical to that for newtbasin, but the coloring scheme is different. Pixels are colored not according to the root that would be "converged on" if you started using Newton's formula from that point, but according to the iteration when the value is close to a root. For example, if the calculations for a particular pixel converge to the 7th root on the 23rd iteration, newtbasin will color that pixel using color #7, but newton will color it using color #23.

If you have a 256-color mode, use it: the effects can be much livelier than those you get with type=newtbasin, and color cycling becomes, like, downright cosmic. If your "corners" choice is symmetrical, Id exploits the symmetry for faster display.

The applicable "params=" values are the same as newtbasin. Try "params=4." Other values are 3 through 10. 8 has twice the symmetry and is faster.

Complex Newton

(type=complexnewton, complexbasin)

Well, hey, "$Z^n - 1$" is so boring when you can use "$Z^a - b$" where "a" and "b" are complex numbers! The "complexnewton" and "complexbasin" fractal types are just the "newton" and "newtbasin" fractal types with this little added twist. When you select these fractal types, you are prompted for four values (the real and imaginary portions of "a" and "b"). If "a" has a complex portion, the fractal has a discontinuity along the negative axis - relax, we finally figured out that it's **supposed** to be there!

Orbit Fractals

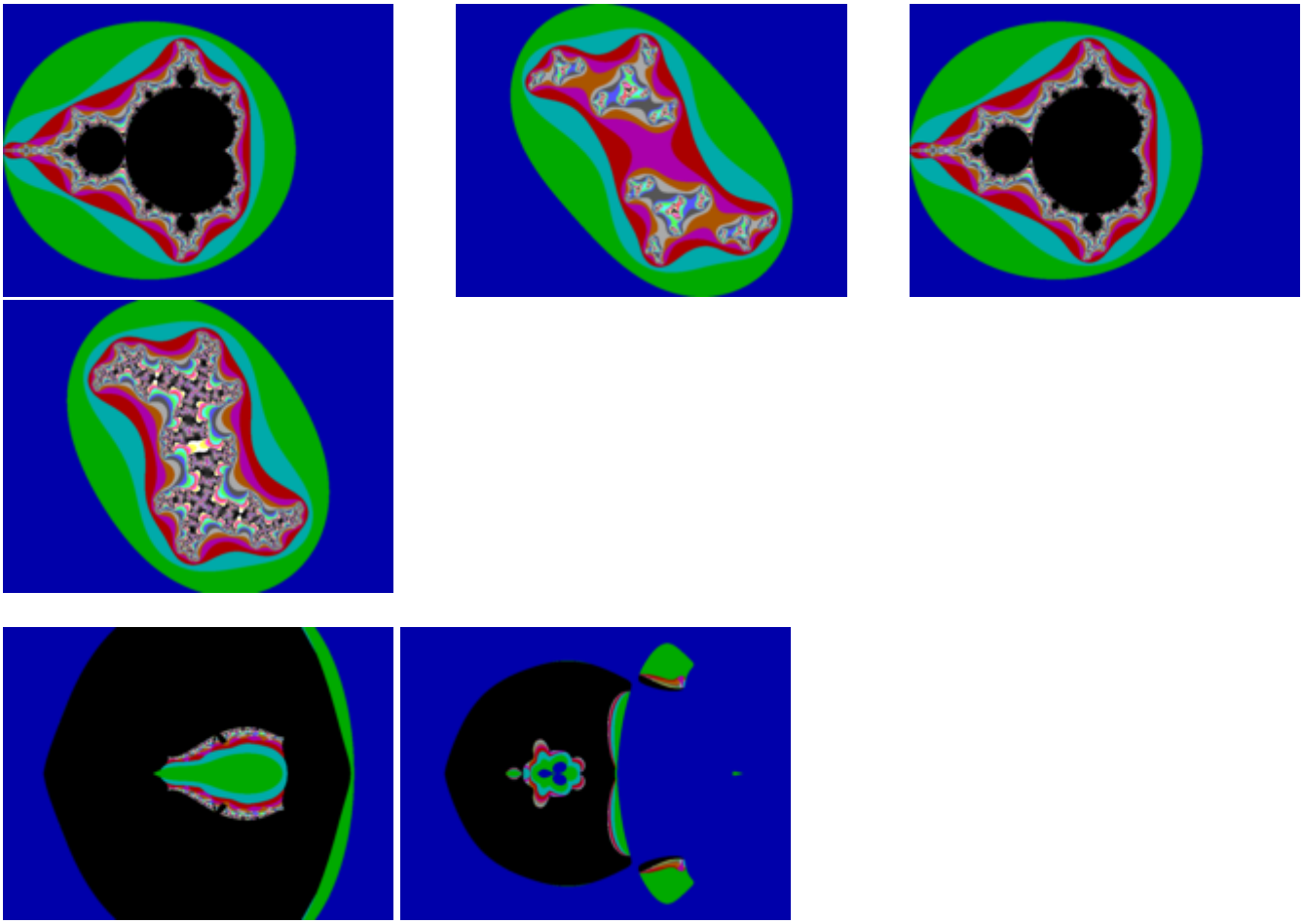
Orbit fractals are generated by plotting an orbit path in two or three dimensional space.

See Lorenz Attractors, Rossler Attractors, Henon Attractors, Pickover Attractors, Gingerbreadman, and Martin Attractors.

The orbit trajectory for these types can be saved in the file orbits.raw by invoking Id with the "orbitsave=yes" command-line option. This file will be overwritten each time you generate a new fractal, so rename it if you want to save it.

Peterson Variations

(type=marksmandel, marksjulia, cmplxmarksmand, cmplxmarksjul, marksmandelpwr, tim's_error)



These fractal types are contributions of Mark Peterson. MarksMandel and MarksJulia are two families of fractal types that are linked in the same manner as the classic Mandelbrot/Julia sets: each MarksMandel set can be considered as a mapping into the MarksJulia sets, and is linked with the spacebar toggle. The basic equation for these sets is:

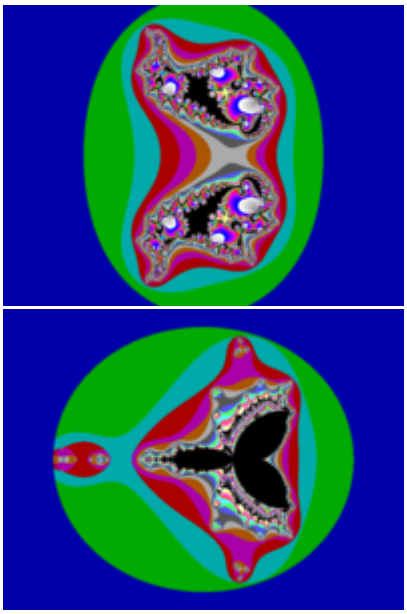
$$z_{n+1} = \lambda^{\exp-1} z_n^2 + \lambda$$

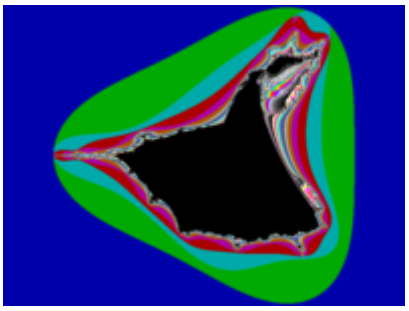
where z_0 = 0.0 and λ is $(x + iy)$ for MarksMandel. For MarksJulia, $z_0 = (x + iy)$ and λ is a constant (taken from the MarksMandel spacebar toggle, if that method is used). The exponent is a positive integer or a complex number. We call these "families" because each value of the exponent yields a different MarksMandel set, which turns out to be a kinda-polygon with (exponent) sides. The exponent value is the third parameter, after the "initialization warping" values. Typically one would use null warping values, and specify the exponent with something like "params=0/0/5", which creates an unwarped, pentagonal MarksMandel set.

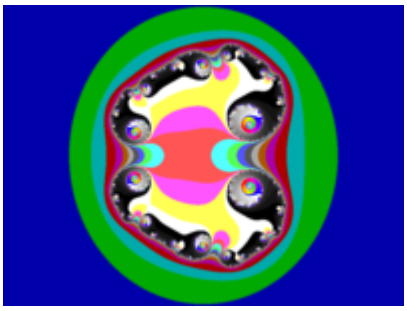
In the process of coding MarksMandelPwr formula type, Tim Wegner created the type "tim's_error" after making an interesting coding mistake.

Phoenix

(type=phoenix, mandphoenix, phoenixcplx, mandphoenixclx)







The phoenix type defaults to the original phoenix curve discovered by Shigehiro Ushiki, "Phoenix", IEEE Transactions on Circuits and Systems, Vol. 35, No. 7, July 1988, pp. 788-789. These images do not have the X and Y axis swapped as is normal for this type.

The mandphoenix type is the corresponding Mandelbrot set image of the phoenix type. The spacebar toggles between the two as long as the mandphoenix type has an initial z_0 of (0,0). The mandphoenix is not an effective index to the phoenix type, so explore the wild blue yonder.

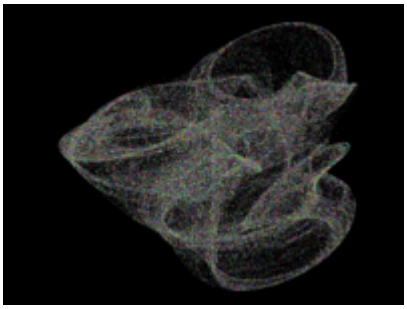
To reproduce the Mandelbrot set image of the phoenix type as shown in Stevens' book, "Fractal Programming in C", set initorbit=0/0 on the command line or with the `G` key. The colors need to be rotated one position because Stevens uses the values from the previous calculation instead of the current calculation to determine when to bailout.

The phoenixcplx type is implemented using complex constants instead of the real constants that Stevens used. This recreates the mapping as originally presented by Ushiki.

The mandphoenixclx type is the corresponding Mandelbrot set image of the phoenixcplx type. The spacebar toggles between the two as long as the mandphoenixclx type has a perturbation of z_0 of (0,0). The mandphoenixclx is an effective index to the phoenixcplx type.

Pickover Attractors

(type=pickover)



Clifford A. Pickover of the IBM Thomas J. Watson Research center is such a creative source for fractals that we attach his name to this one only with great trepidation. Probably tomorrow he'll come up with another one and we'll be back to square one trying to figure out a name!

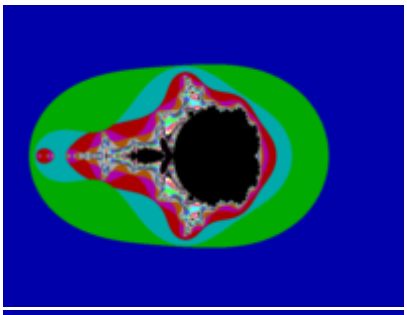
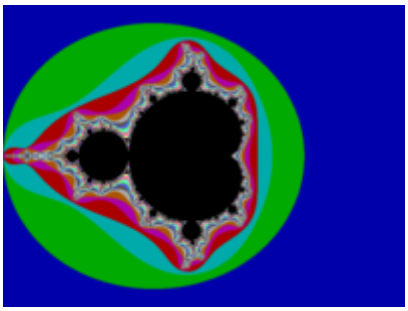
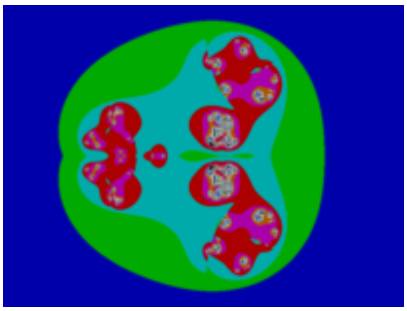
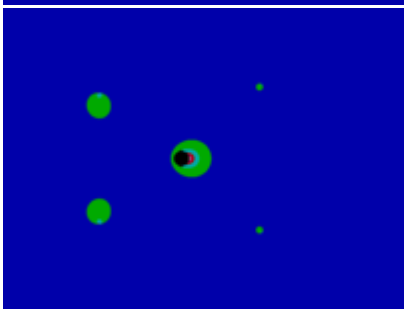
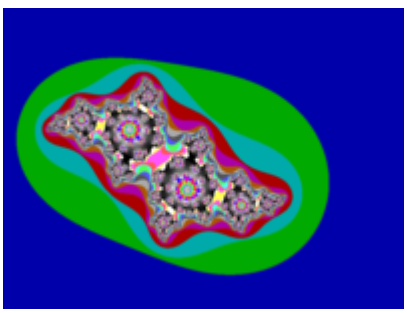
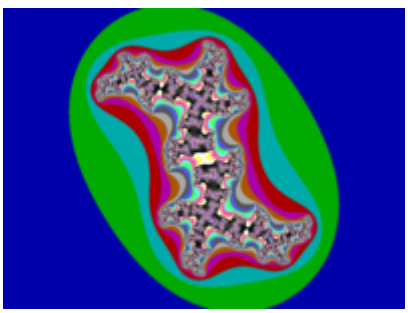
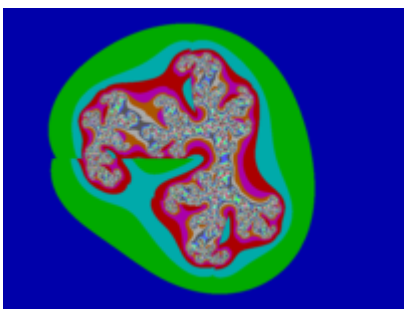
This one is the three dimensional orbit defined by:

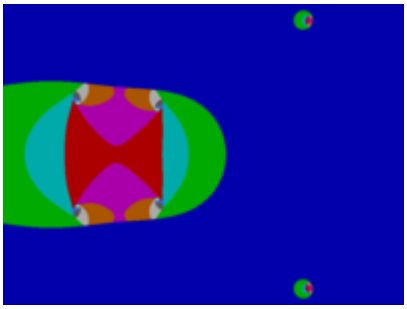
$$x_{\text{new}} = \sin(a\,y) - z\cos(b\,x)$$
$$y_{\text{new}} = z\sin(c\,x) - \cos(d\,y)$$
$$z_{\text{new}} = \sin(x)$$

Default parameters are: a = 2.24, b = .43, c = -.65, d = -2.43

Pickover Mandelbrot/Julia Types

(type=manfn+zsqrd, julfn+zsqrd, manzpower, julzpower, manzzpwr, julzzpwr, manfn+exp, julfn+exp)



These types have been explored by Clifford A. Pickover, of the IBM Thomas J. Watson Research center. As implemented in Id, they are regular Mandelbrot/Julia set pairs that may be plotted with or without the "biomorph" option Pickover used to create organic-looking beasties (see below).

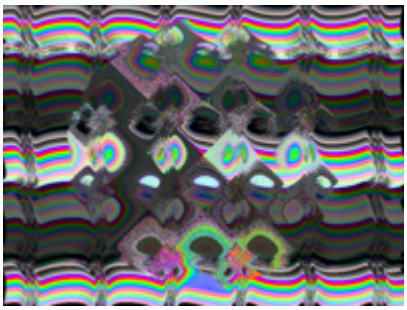
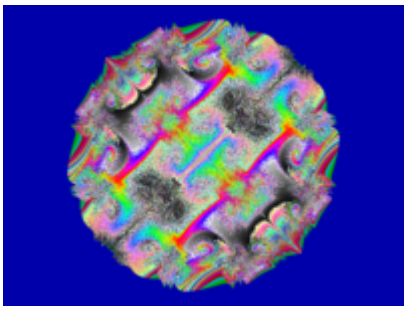
These types are produced with formulas built from the functions z^z, z^n, $\sin(z)$, and e^z for complex z. Types with "power" or "pwr" in their name have an exponent value as a third parameter. For example, type=manzpower params=0/0/2 is our old friend the classical Mandelbrot, and type=manzpower params=0/0/4 is the Quartic Mandelbrot. Other values of the exponent give still other fractals. Since these were the original "biomorph" types, we should give an example. Try:

```
id type=manfn+zsqrd biomorph=0 corners=-8/8/-6/6 function=sin
```

to see a big biomorph digesting little biomorphs!

Pickover Popcorn

(type=popcorn, popcornjul)

Here is another Pickover idea. This one computes and plots the orbits of the dynamic system defined by:

$$x_{n+1} = x_n - \text{real}(h\text{fn1}(y_n + \text{fn2}(c\,y_n))) - \text{imag}(h\text{fn3}(x_n + \text{fn4}(c\,x_n)))$$
$$y_{n+1} = y_n - \text{real}(h\text{fn3}(x_n + \text{fn4}(c\,x_n))) - \text{imag}(h\text{fn1}(y_n + \text{fn2}(c\,y_n)))$$

In the original the functions were: sin, tan, sin, tan, and c was 3.

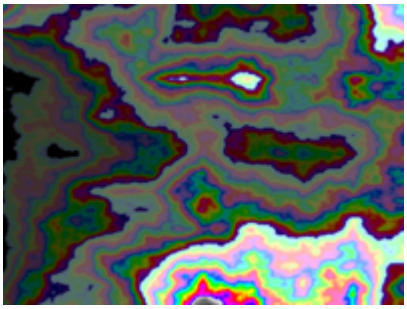
The initializers x_0 and y_0 equal to all the complex values within the "corners" values, and h=.01. All these orbits are superimposed, resulting in "popcorn" effect. You may want to use a maxiter value less than normal - Pickover recommends a value of 50. Although you can zoom and rotate popcorn, the results may not be what you'd expect, due to the superimposing of orbits and arbitrary use of color. The orbits frequently occur outside of the screen boundaries. To view the fractal in its

entirety, set the preview display to "yes" using the `V` command.

As a bonus, type=popcornjul shows the Julia set generated by these same equations with the usual escape-time coloring. Turn on orbit viewing with the `O` command, and as you watch the orbit pattern you may get some insight as to where the popcorn comes from.

Plasma Clouds

(type=plasma)



Plasma clouds are real live fractals, even though we didn't know it at first. They are generated by a recursive algorithm that randomly picks colors of the corner of a rectangle, and then continues recursively quartering previous rectangles. Random colors are averaged with those of the outer rectangles so that small neighborhoods do not show much change, for a smoothed-out, cloud-like effect. The more colors your video mode supports, the better. The result, believe it or not, is a fractal landscape viewed as a contour map, with colors indicating constant elevation. To see this, save and view with the `3` command (see "3D" Images) and your "cloud" will be converted to a mountain!

You've got to try color cycling on these (hit "+" or "-"). If you haven't been hypnotized by the drawing process, the writhing colors will do it for sure. We have now implemented subliminal messages to exploit the user's vulnerable state; their content varies with your bank balance, politics, gender, accessibility to a Id programmer, and so on. A free copy of "The Millionaire Next Door" to the first person who spots them!

This type accepts four parameters.

The first determines how abruptly the colors change. A value of .5 yields bland clouds, while 50 yields very grainy ones. The default value is 2.

The second determines whether to use the original algorithm (0) or a modified one (1). The new one gives the same type of images but draws the dots in a different order. It will let you see what the final image will look like much sooner than the old one.

The third determines whether to use a new seed for generating the next plasma cloud (0) or to use the previous seed (1).

The fourth parameter turns on 16-bit .POT output which provides much smoother height gradations. This is especially useful for creating mountain landscapes when using the plasma output with a ray tracer such as POV-Ray.

With parameter three set to 1, the next plasma cloud generated will be identical to the previous but

at whatever new resolution is desired.

Zooming is ignored, as each plasma-cloud screen is generated randomly.

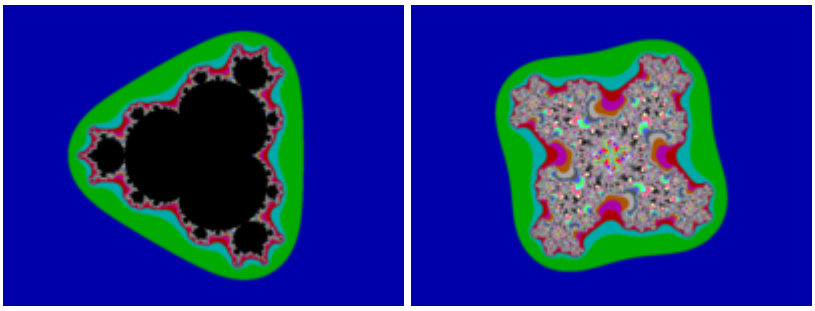
The random number seed used for each plasma image is displayed on the `Tab` information screen, and can be entered with the command line parameter "rseed=" to recreate a particular image.

The algorithm is based on the Pascal program distributed by Bret Mulvey. We have ported it to C and integrated it with Id's graphics and animation facilities. This implementation does not use floating-point math. The algorithm was modified so that the plasma effect is independent of screen resolution.

Saved plasma-cloud screens are excellent starting images for fractal "landscapes" created with the "3D" commands.

Quartic Mandelbrot/Julia

(type=mandel4, julia4)

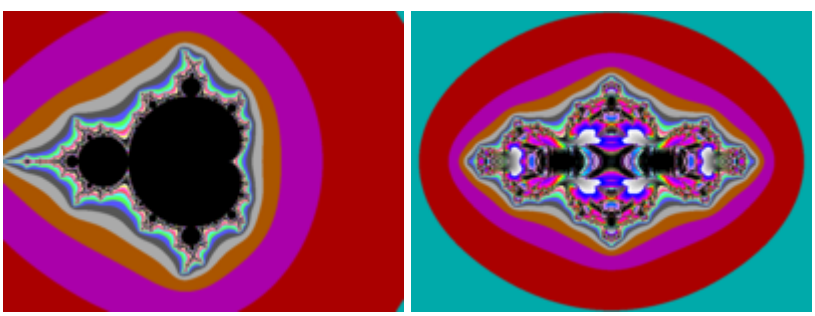


These fractal types are the moral equivalent of the original M and J sets, except that they use the formula $z_{n+1} = z_n^4 + c$, which adds additional pseudo-symmetries to the plots. The "Mandel4" set maps to the "Julia4" set via — surprise! — the spacebar toggle. The M4 set is kind of boring at first (the area between the "inside" and the "outside" of the set is pretty thin, and it tends to take a few zooms to get to any interesting sections), but it looks nice once you get there. The Julia sets look nice right from the start.

Other powers, like z_n^3 or z_n^7, work in exactly the same fashion. We used this one only because we're lazy, and $z_n^4 = (z_n^2)^2$.

Quaternion

(type=quat,quatjul)



These fractals are based on quaternions. Quaternions are an extension of complex numbers, with 4 parts instead of 2. That is, a quaternion Q equals a+ib+jc+kd, where a,b,c,d are reals. Quaternions have rules for addition and multiplication. The normal Mandelbrot and Julia formulas can be generalized to use quaternions instead of complex numbers.

There is one complication. Complex numbers have 2 parts, so they can be displayed on a plane. Quaternions have 4 parts, so they require 4 dimensions to view. That is, the quaternion Mandelbrot set is actually a 4-dimensional object. Each quaternion C generates a 4-dimensional Julia set.

One method of displaying the 4-dimensional object is to take a 3- dimensional slice and render the resulting object in 3-dimensional perspective. Id isn't that sophisticated, so it merely displays a 2-dimensional slice of the resulting object. (Note: Now Id is that sophisticated! See the Julibrot type!)

In Id, for the Julia set, you can specify the four parameters of the quaternion constant: c=(c1,ci,cj,ck), but the 2-dimensional slice of the z-plane Julia set is fixed to (xpixel,ypixel,0,0).

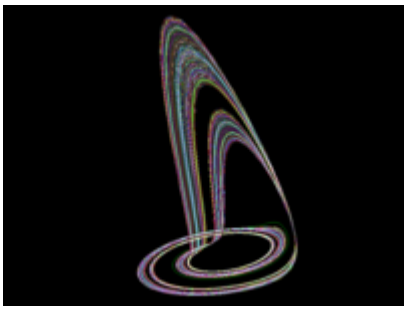
For the Mandelbrot set, you can specify the position of the c-plane slice: (xpixel,ypixel,cj,ck).

These fractals are discussed in Chapter 10 of Pickover's "Computers, Pattern, Chaos, and Beauty".

See also HyperComplex and Quaternion and Hypercomplex Algebra

Rossler Attractors

(type=rossler3d)



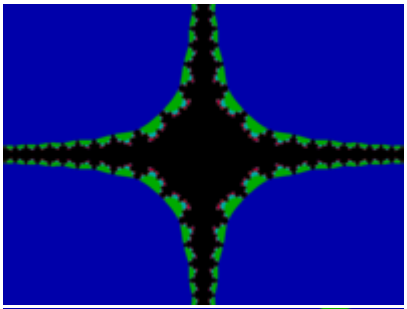
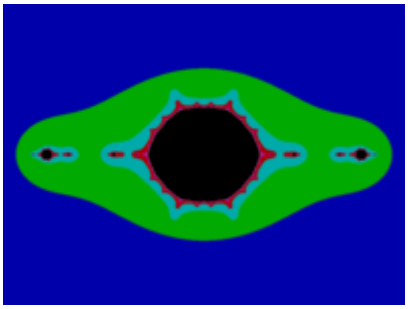
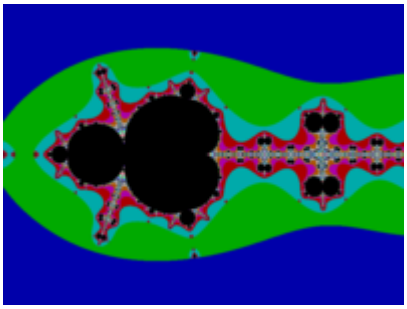
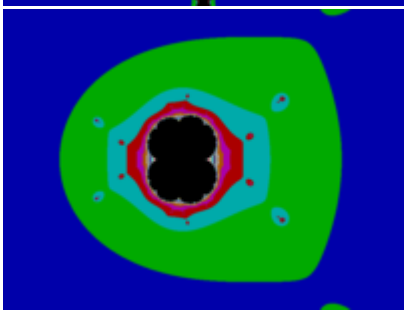
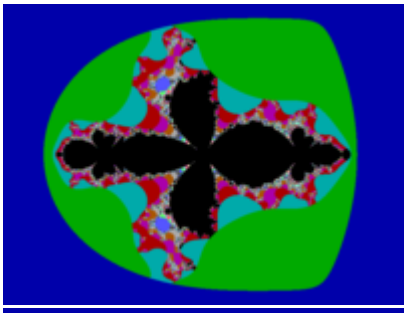
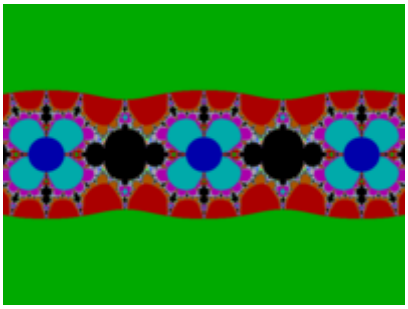
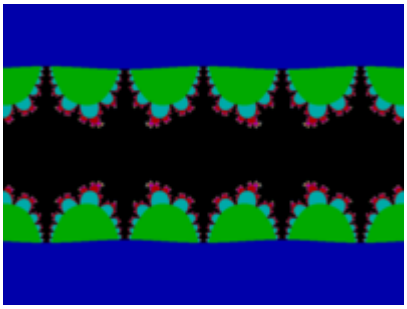
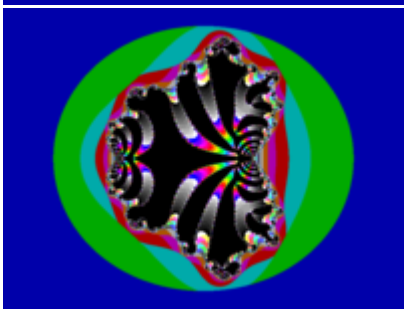
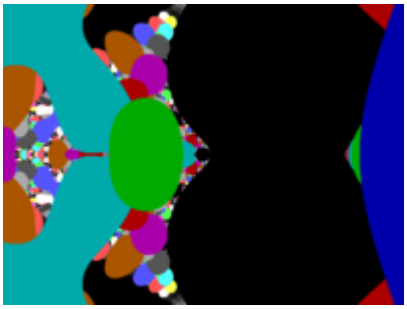
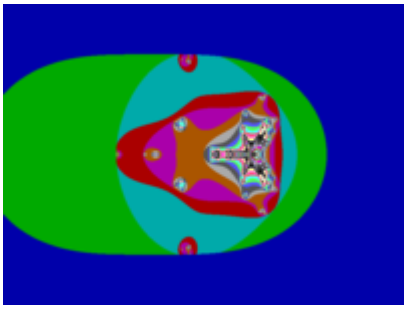
This fractal is named after the German Otto Rossler, a non-practicing medical doctor who approached chaos with a bemusedly philosophical attitude. He would see strange attractors as philosophical objects. His fractal namesake looks like a band of ribbon with a fold in it. All we can say is we used the same calculus-teacher-defeating trick of multiplying the equations by "dt" to solve the differential equation and generate the orbit. This time we will skip straight to the orbit generator - if you followed what we did above with type Lorenz you can easily reverse engineer the differential equations.

$$x_{\text{new}} = x - ydt - zdt$$
$$y_{\text{new}} = y + xdt + aydt$$
$$z_{\text{new}} = z + bdt + xzdt - czdt$$

Default parameters are dt = .04, a = .2, b = .2, c = 5.7

Scott Taylor / Lee Skinner Variations

(type=fn(z*z), fn*fn, fn*z+z, fn+fn, fn(z)+fn(pix), sqr(1/fn), sqr(fn), spider, tetrate, manowar)

Two of Id's faithful users went bonkers when we introduced the "formula" type, and came up with all kinds of variations on escape-time fractals using trig functions. We decided to put them in as regular types, but there were just too many! So we defined the types with variable functions and let you, the overwhelmed user, specify what the functions should be! Thus Scott Taylor's "$z = \sin(z) + z^2$" formula type is now the "fn+fn" regular type, and EITHER function can be one of sin,

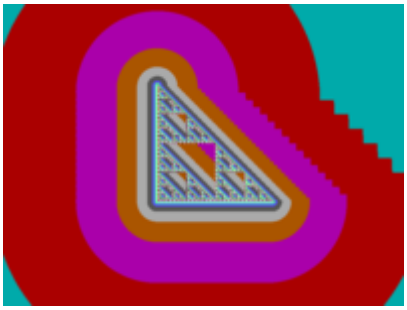
cos, tan, cotan, sinh, cosh, tanh, cotanh, exp, log, sqr, recip, ident, zero, one, conj, flip, cosxx, asin, asinh, acos, acosh, atan, atanh, sqrt, abs, or cabs.

Plus we give you 4 parameters to set, the complex coefficients of the two functions! Thus the innocent-looking "fn+fn" type is really 729 different types in disguise, not counting the damage done by the parameters!

Lee informs us that you should not judge fractals by their "outer" appearance. For example, the images produced by $z = \sin(z) + z^2$ and $z = \sin(z) - z^2$ look very similar, but are different when you zoom in.

Sierpinski Gasket

(type=sierpinski)



Another pre-Mandelbrot classic, this one found by W. Sierpinski around World War I. It is generated by dividing a triangle into four congruent smaller triangles, doing the same to each of them, and so on, yea, even unto infinity. (Notice how hard we try to avoid reiterating "iterating"?)

If you think of the interior triangles as "holes", they occupy more and more of the total area, while the "solid" portion becomes as hopelessly fragile as that gasket you had to remove without damaging it — you remember, that Sunday afternoon when all the parts stores were closed? There's a three-dimensional equivalent using nested tetrahedrons instead of triangles, but it generates too much pyramid power to be safely unleashed yet.

There are no parameters for this type.

Test Stub

(type=test)

This is a stub that we (and you!) use for trying out new fractal types. "Type=test" fractals make use of Id's structure and features for whatever code is in the routine 'testpt()' (located in the small source file testpt.cpp) to determine the color of a particular pixel.

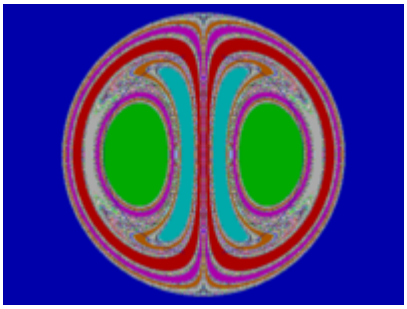
If you have a favorite fractal type that you believe would fit nicely into Id, just rewrite the function in testpt.cpp (or use the prototype function there, which is a simple M-set implementation) with an algorithm that computes a color based on a point in the complex plane.

After you get it working, send your code as a pull request on our github project and we might just add it to the next release of Id, with full credit to you. Our criteria are: 1) an interesting image and

2) a formula significantly different from types already supported. (Bribery may also work. This author is completely honest, but I don't trust those other guys.) Be sure to include an explanation of your algorithm and the parameters supported, preferably formatted as you see here to simplify folding it into the documentation.

Unity

(type=unity)



This Peterson variation began with curiosity about other "Newton-style" approximation processes. A simple one,

$$\begin{aligned} \mathrm{One} &= x^2 + y^2 \\ y &= (2 - \mathrm{One})x \\ x &= (2 - \mathrm{One})y \end{aligned}$$

produces the fractal called Unity.

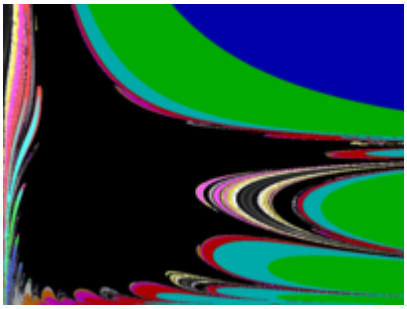
One of its interesting features is the "ghost lines." The iteration loop bails out when it reaches the number 1 to within the resolution of a screen pixel. When you zoom a section of the image, the bailout criterion is adjusted, causing some lines to become thinner and others thicker.

Only one line in Unity that forms a perfect circle: the one at a radius of 1 from the origin. This line is actually infinitely thin. Zooming on it reveals only a thinner line, up (down?) to the limit of accuracy for the algorithm. The same thing happens with other lines in the fractal, such as those around $|x| = |y| = \left(\frac{1}{2}\right)^{\frac{1}{2}} \approx .7071$

Try some other tortuous approximations using the Test Stub and let us know what you come up with!

Volterra-Lotka Fractals

(type=volterra-lotka)



In the book "The Beauty of Fractals", these images are offered as an example of "how an apparently innocent system of differential equations gives rise to unimaginably rich and complex behavior after discretization." The Volterra-Lotka equations are a refinement of attempts to model predator-prey systems.

If x represents the prey population and y represents the predator population, their relationship can be expressed as:

$$\frac{dx}{dt} = Ax - Bxy = f(x, y)$$
$$\frac{dy}{dt} = -Cy + Dxy = g(x, y)$$

According to Peitgen and Richter, "Hence, x grows at a constant rate in the absence of y, and y decays at a constant rate in the absence of x. The prey is consumed in proportion to y, and the predators expand in proportion to x." They proceed to "discretize" this system, by "mating" the Euler and Heun methods. For purposes of image computation, their formula (equation 8.3 on page 125) can be interpreted as:

$$x_{\text{new}} = x + \frac{h}{2}[f(x, y) + f(x + pf(x, y), y + pg(x, y))]$$
$$y_{\text{new}} = y + \frac{h}{2}[g(x, y) + g(x + pf(x, y), y + pg(x, y))]$$

This formula can be used to plot or connect single points, starting with arbitrary values of x(0) and y(0), to produce typical "strange attractor" images such as the ones commonly derived from the Henon or Lorenz formulae. But to produce an escape-time fractal, we iterate this formula for all (x, y) pairs that we can associate with pixels on our monitor screen. The standard window is: 0.0 < x < 6.0; 0.0 < y < 4.5. Since the "unimaginably rich and complex behavior" occurs with the points that do not escape, the inside coloring method assumes considerable importance.

The parameters h and p can be selected between 0.0 and 1.0, and this determines the types of attractors that will result. Infinity and (1, 1) are predictable attractors. For certain combinations, an "invariant circle" (which is not strictly circular) and/or an orbit of period 9 also are attractive.

The invariant circle and periodic orbit change with each (h, p) pair, and they must be determined empirically. That process would be thoroughly impractical to implement through any kind of fixed formula. This is especially true because even when these attractors are chosen, the threshold for determining when a point is "close enough" is quite arbitrary, and yet it affects the image considerably. The best compromise in the context of a generalized formula is to use either the "zmag" or "bof60" inside coloring options. See Inside=bof60|bof61|zmag|fmod|period|atan for

details. This formula performs best with a relatively high bailout value; the default is set at 256, rather than the standard default of 4.

Reference: Peitgen, H.-O. and Richter, P.H. The Beauty of Fractals, Springer-Verlag, 1986; Section 8, pp. 125-7.

Doodads, Bells, and Whistles

Drawing Method

The "passes option" (`X` options screen or "passes=" parameter) selects one of the single-pass, dual-pass, triple-pass, solid-guessing (default), solid-guessing after pass n, boundary tracing, tesseral, synchronous orbits, or orbits modes.

The "perturbation" option (`X` options screen or "perturbation=" parameter) selects whether supported fractal types use the perturbation orbit algorithm. The default value is "auto", which uses perturbation when arbitrary precision is active.

The "perturbation-tolerance" option (`X` options screen or "perturbation-tolerance=" parameter) controls perturbation glitch detection. The value must be greater than zero. The default is 1e-6.

This option applies to most fractal types.

Single-pass mode ("1") draws the screen pixel by pixel.

Dual-pass ("2") generates a half-resolution screen first as a preview using 2x2-pixel boxes, and then generates the rest of the dots with a second pass. Dual-pass uses no more time than single-pass.

Triple-pass ("3") generates the coarse first pass of the solidguessing mode (see "g" below), then switches to either "1" (with low resolution video modes) or "2" (with higher resolution video modes). The advantage of '3' vs '2' is that when using high resolution modes, the first pass has a much lower resolution (about 160x120) and is therefore much quicker than the first pass of the passes=2 mode. However, with the '2' mode, the first pass does not represent wasted time. The '3' mode wastes the effort of generating the coarse first screen.

The single, dual, and triple pass modes all result in identical images. These modes are for those who desire the highest possible accuracy. Most people will want to use the guessing mode, described next.

Solid-guessing ("g") is the default. It performs from two to four visible passes - more in higher resolution video modes. Its first visible pass is actually two passes - one pixel per 4x4, 8x8, or 16x16 pixel box is generated, and the guessing logic is applied to fill in the blocks at the next level (2x2, 4x4, or 8x8). Subsequent passes fill in the display at the next finer resolution, skipping blocks which are surrounded by the same color. Solid-guessing can guess wrong, but it sure guesses quickly!

Solid-guessing stop after pass n ("g1" through "g6") are a variation on the guessing mode in which the algorithm stops after the nth pass. This facility is for exploring in low resolution when you'd rather see a low resolution image with large blocky pixels filling the whole screen than a small low resolution image such as you get with the `V` (View Windows) command. Note that on the `X` screen

you can't directly type g1 or g2. Press g repeatedly until you get the option you want, or else use the left or right arrow keys.

Boundary Tracing ("b"), which only works accurately with fractal types (such as the Mandelbrot set, but not the Newton type) that do not contain "islands" of colors, finds a color boundary, traces it around the screen, and then "blits" in the color over the enclosed area.

Tesseral ("t") is a sort of "super-solid-guessing" option that successively divides the image into subsections and colors in rectangles that have a boundary of a solid color. It's actually slower than the solid-guessing algorithm, but it looks neat, so we left it in. This mode is also subject to errors when islands of color appear inside the rectangles.

Diffusion Scan ("d") is a drawing type based on dithering techniques. It scans the image spreading the points evenly and to each point it paints a square of the appropriate size so that the image will be incrementally enhanced. This method calculates all the points in the image being a good substitute for single/double/triple pass and presents a quick visualization even in the slowest fractals. With "fillcolor=0" (below) the squares are not painted and the points are spread over the image until all have being calculated (sort of a "fade in").

Perturbation is an orbit algorithm that is particularly fast when used with deep-zooms of the Mandelbrot set that require arbitrary precision arithmetic. Use "perturbation=yes" to force it on for supported fractal types, or "perturbation=no" to force it off. The "passes=p" value is accepted for compatibility and is equivalent to "perturbation=yes" using the default solid-guessing traversal.

Use "perturbation-tolerance=<n>" to adjust glitch detection. Smaller values make glitch detection stricter; the default is 1e-6.

The "fillcolor=" option in the `X` screen or on the command line sets a fixed color to be used by the boundary tracing and tesseral calculations for filling in defined regions. The effect of this is to show off the boundaries of the areas delimited by these two methods.

Orbits ("o") draws an image by plotting the orbits of the escape time fractals. This technique uses the same coordinates to draw an image as the other passes options, sets "passes=1" and no symmetry, and then plots the orbits for each pixel. Zooming into a "passes=o" image is in fact zooming into the "passes=1" image, and the resulting image may not be what is expected. To find interesting places to investigate, press `O` after an image has completed and watch the behavior of the orbits as the cursor is moved around the screen. See Orbits Window.

The "outside=summ" option causes orbits to increment a pixel's color number every time an orbit touchs it; the resulting display is a 2-d histogram. If "outside=" is some other value, then the "inside=" color determines the color of the plotted orbits. If "inside=0", then the color number is incremented at the start of each pixel of the passes=1 image.

The "orbitdelay=" option controls how many orbits are computed before the orbits are displayed on the screen. This allows the orbits to settle down. The "orbitinterval=" option causes Orbits to plot every nth orbit point. A non-zero value of the "periodicity=" option causes orbits to not plot orbits that have reached the bailout conditions or where an orbit goes off the visible area of the image. A zero value of periodicity will plot all orbits except as modified by orbitdelay and orbitinterval.

Synchronous orbits ("s") is an experimental mode using the "fractal witchcraft" algorithm based on

the Almondbread implementation by Michael Ganss. This algorithm optimizes deep zooms by calculating parallel orbits starting at different points, and subdividing when the orbits break formation.

Synchronous orbits (also known as SOI) has some limitations. SOI is loosely coupled with Id and most options don't work with it. Only types mandel and julia are implemented. SOI is only useful for very deep zooms, but only up to the limit of double precision. Within this narrow magnification range, SOI can result in tremndous speedups. If you invoke Id with "debug=3444" on the command line, a long double (rather than double) version will be used, which allows zooming about 1000 times deeper. SOI really needs to be ported to Id's arbitary precision.

Color Maps

You can save and restore color palettes for use with any image. To load a palette onto an existing image, use the `L` command in color-cycling or palette-editing mode. To save a palette, use the `S` command in those modes. To change the default palette for an entire run, use the command line "map=" parameter.

The default filename extension for color-map files is ".map".

These color-maps are ASCII text files set up as a series of RGB triplet values (one triplet per line, encoded as the red, green, and blue [RGB] components of the color). Map file color values are integers - values go from 0 (low) to 255 (high).

Id is distributed with some sample .map files: altern.map the famous "Peterson-Vigneau Pseudo-Grey Scale" blues.map for rainy days, by Daniel Egnor chroma.map general purpose, chromatic default.map the default start-up values firestrm.map general purpose, muted fire colors gamma1.map, Lee Crocker's response gamma2.map to altern.map glasses1.map used with 3D glasses modes glasses2.map used with 3D glasses modes green.map shaded green grey.map another gray variant grid.map for stereo surface grid images headache.map major stripes, by D. Egnor (try cycling and hitting <2>) landscap.map Guruka Singh Khalsa's favorite map for plasma "landscapes" neon.map a flashy map, by Daniel Egnor royal.map the royal purple, by Daniel Egnor topo.map Monte Davis's contribution to full color terrain volcano.map an explosion of lava, by Daniel Egnor

Autokey Mode

The autokey feature allows you to set up beautiful self-running demo "loops". You can set up hypnotic sequences to attract people to a booth, to generate sequences for special effects, to teach how fractal exploring is done, etc.

A sample autokey file (demo.key) and a script to run it (demo.bat) is included with Id. Type "demo" at a command prompt to run it.

Autokey record mode is enabled with the command line parameter "autokey=record". Keystrokes are saved in an intelligible text format in a file called auto.key. You can change the file name with the "autokeyname=" parameter.

Playback is enabled with the parameter "autokey=play". Playback can be terminated by pressing the `Esc` key.

After using record mode to capture an autokey file, you'll probably want to touch it up using your editor before playing it back.

Separate lines are not necessary but you'll probably find it easier to understand an autokey file if you put each command on a separate line. Autokey files can contain the following:

```
Quoted strings.  Id reads whatever is between the quotes just as if
you had typed it.  For example,
    "t" "ifs" issues the "t" (type) command and then enters the
letters i", "f", and "s" to select the ifs type.
```

```
Symbols for function keys used to select a video mode.  Examples:
    F3  -- Function key <F3>
    SF3 -- <Shift+F3> together
```

```
Special keys:
  ENTER   ESC     F1              PAGEUP PAGEDOWN
  HOME END                        LEFT RIGHT UP DOWN
  INSERT DELETE                   TAB
  CTRL_RIGHT CTRL_LEFT            CTRL_DOWN  CTRL_UP
  CTRL_HOME CTRL_END
```

```
Control characters can be written with caret notation.  For example,
    ^S -- <Ctrl+S> together
    ^B -- <Ctrl+B> together
```

```
WAIT <nnn.n> -- wait nnn.n seconds before continuing
```

```
CALCWAIT -- pause until the current fractal calculation or file save
or restore or RDS generation is finished.  This command makes demo
files more robust since calculation times depend on the speed of the
machine running the demo - a "WAIT 10" command may allow enough time
to complete a fractal on one machine, but not on another.  The record
mode does not generate this command - it should be added by hand to
the autokey file whenever there is a process that should be allowed to
run to completion.
```

```
GOTO target -- The autokey file continues to be read from the label
"target".  The label can be any word that does not duplicate a key
word.  It must be present somewhere in the autokey file with a colon
after it.  Example:
    MESSAGE 2 This is executed once
```

```
    start:
    MESSAGE 2 This is executed repeatedly
    GOTO start
GOTO is mainly useful for writing continuous loop demonstrations.  It
can also be useful when debugging an autokey file, to skip sections of
it.
```

```
; -- A semi-colon indicates that the rest of the line containing it is
a comment.
```

```
MESSAGE nn <Your message here> -- Places a message on the top of the
screen for nn seconds
```

Making Id demos can be tricky. Here are some suggestions which may help:

```
Start Id with "id autokeyname=mydemo.key autokey=record".  Use a
unique name each time you run so that you don't overwrite prior files.
```

```
When in record mode, avoid using the arrow keys to select filenames,
fractal types, formula names, etc.  Instead, try to type in names.
This will ensure that the exact item you want gets chosen during
playback even if the list is different on playback.
```

```
Beware of video mode assumptions.  It is safest to build a separate
demo for different resolution video modes.
```

```
When in the record mode, try to type names quickly, then pause.  If
you pause partway through a name Id will break up the string in the
.KEY file.  For example, if you paused in the middle of typing
fract001, you might get:
    "fract"
    WAIT 2.2
    "001"
No harm done, but messy to clean up.  Id ignores pauses less than
about 1/2 second.
```

```
Do pause when you want the viewer to see what is happening during
playback.
```

```
When done recording, clean up your mydemo.key file.  Insert a CALCWAIT
after each keystroke which triggers something that takes a variable
```

```
amount of time (calculating a fractal, restoring a file, saving a
file).
```

```
Add comments with ";" to the file so you know what is going on in
future.
```

```
It is a good idea to use INSERT before a GOTO which restarts the demo.
The <Insert> key resets Id as if you exited the program and restarted
it.
```

Warning: an autokey file built for this version of Id will probably require some retouching before it works with future releases of Id. We have no intention of making sure that the same sequence of keystrokes will have exactly the same effect from one version of Id to the next. That would require pretty much freezing Id development, and we just love to keep enhancing it!

Distance Estimator Method

This is Phil Wilson's implementation of an alternate method for the M and J sets, based on work by mathematician John Milnor and described in "The Science of Fractal Images", p. 198. While it can take full advantage of your color palette, one of the best uses is in preparing monochrome images for a printer. Using the 1600x1200x2 disk video mode and an HP LaserJet, we have produced pictures of quality equivalent to the black and white illustrations of the M-set in "The Beauty of Fractals."

The distance estimator method widens very thin "strands" which are part of the "inside" of the set. Instead of hiding invisibly between pixels, these strands are made one pixel wide.

Though this option is available with any escape time fractal type, the formula used is specific to the mandel and julia types - for most other types it doesn't do a great job.

To turn on the distance estimator method with any escape time fractal type, set the "Distance Estimator" value on the `Y` options screen (or use the "distest=" command line parameter).

Setting the distance estimator option to a negative value -nnn enables edge-tracing mode. The edge of the set is displayed as color number nnn. This option works best when the "inside" and "outside" color values are also set to some other value(s).

In a 2 color (monochrome) mode, setting to any positive value results in the inside of the set being expanded to include edge points, and the outside points being displayed in the other color.

In color modes, setting to value 1 causes the edge points to be displayed using the inside color and the outside points to be displayed in their usual colors. Setting to a value greater than one causes the outside points to be displayed as contours, colored according to their distance from the inside of the set. Use a higher value for narrower color bands, a lower value for wider ones. 1000 is a good value to start with.

The second distance estimator parameter ("width factor") sets the distance from the inside of the set

which is to be considered as part of the inside. This value is expressed as a percentage of a pixel width, the default is 71. Negative values are now allowed and give a fraction of a percent of the pixel width. For example: -71 gives 1/71 % of the pixel width.

You should use 1 or 2 pass mode with the distance estimator method, to avoid missing some of the thin strands made visible by it. For the highest quality, "maxiter" should also be set to a high value, say 1000 or so. You'll probably also want "inside" set to zero, to get a black interior.

Enabling the distance estimator method automatically toggles to floating point mode. When you reset distest back to zero, remember to also turn off floating point mode if you want it off.

Unfortunately, images using the distance estimator method can take a long time to calculate even on a fast machine, especially if a high "maxiter" value is used. One way of dealing with this is to leave it turned off while you find and frame an image. Then hit `B` to save the current image information in a parameter file (see Parameter Save/Restore Commands). Use an editor to change the parameter file entry, adding "distest=1", "video=something" to select a high-resolution monochrome disk-video mode, "maxiter=1000", and "inside=0". Run the parameter file entry with the `@` command when you won't be needing your machine for a while (over the weekend?).

To obtain the finest strands it is necessary to set the "width factor" to a large negative value, such as -32000.

Inversion

Many years ago there was a brief craze for "anamorphic art": images painted and viewed with the use of a cylindrical mirror, so that they looked weirdly distorted on the canvas but correct in the distorted reflection. (This byway of art history may be a useful defense when your friends and family give you odd looks for staring at fractal images color-cycling on a CRT.)

The inversion option performs a related transformation on most of the fractal types. You define the center point and radius of a circle; Id maps each point inside the circle to a corresponding point outside, and vice-versa. This is known to mathematicians as inverting (or if you want to get precise, "everting") the plane, and is something they can contemplate without getting a headache. John Milnor (also mentioned in connection with the Distance Estimator Method), made his name in the 1950s with a method for everting a seven-dimensional sphere, so we have a lot of catching up to do.

For example, if a point inside the circle is 1/3 of the way from the center to the radius, it is mapped to a point along the same radial line, but at a distance of (3 * radius) from the origin. An outside point at 4 times the radius is mapped inside at 1/4 the radius.

The inversion parameters on the `Y` options screen allow entry of the radius and center coordinates of the inversion circle. A default choice of -1 sets the radius at 1/6 the smaller dimension of the image currently on the screen. The default values for Xcenter and Ycenter use the coordinates currently mapped to the center of the screen.

Try this one out with a Newton plot, so its radial "spokes" will give you something to hang on to. Plot a Newton-method image, then set the inversion radius to 1, with default center coordinates. The center "explodes" to the periphery.

Inverting through a circle not centered on the origin produces bizarre effects that we're not even going to try to describe. Aren't computers wonderful?

Decomposition

You'll remember that most fractal types are calculated by iterating a simple function of a complex number, producing another complex number, until either the number exceeds some pre-defined "bailout" value, or the iteration limit is reached. The pixel corresponding to the starting point is then colored based on the result of that calculation.

The decomposition option ("decomp=", on the `X` screen) toggles to another coloring protocol. Here the points are colored according to which quadrant of the complex plane (negative real/positive imaginary, positive real/positive imaginary, etc.) the final value is in. If you use 4 as the parameter, points ending up in each quadrant are given their own color; if 2 (binary decomposition), points in alternating quadrants are given 2 alternating colors.

The result is a kind of warped checkerboard coloring, even in areas that would ordinarily be part of a single contour. Remember, for the M-set all points whose final values exceed 2 (by any amount) after, say, 80 iterations are normally the same color; under decomposition, Id runs 80 iterations and then colors according to where the actual final value falls on the complex plane.

When using decomposition, a higher bailout value will give a more accurate plot, at some expense in speed. You might want to set the bailout value (in the parameters prompt following selection of a new fractal type; present for most but not all types) to a higher value than the default. A value of about 50 is a good compromise for M/J sets.

Logarithmic Palettes and Color Ranges

By default, Id maps iterations to colors 1:1. For example, if the calculation for a fractal "escapes" (exceeds the bailout value) after N iterations, the pixel is colored as color number N. If N is greater than the number of colors available, it wraps around. So, if you are using a 16-color video mode, and you are using the default maximum iteration count of 150, your image will run through the 16-color palette 150/16 = 9.375 times.

When you use Logarithmic palettes, the entire range of iteration values is compressed to map to one span of the color range. This results in spectacularly different images if you are using a high iteration limit and are zooming in on an area near a "lakelet".

When using a compressed palette in a 256 color mode, we suggest changing your colors from the usual defaults. The last few colors in the default color map are black. This results in points nearest the "lake" smearing into a single dark band, with little contrast from the blue (by default) lake.

Id has a number of types of compressed palettes, selected by the "Log Palette" line on the `X` screen, or by the "logmap=" command line parameter:

```
logmap=1: for standard logarithmic palette.
```

```
logmap=N (>1): Same as logmap=1, but starting from iteration count N.
Pixels with iteration counts less than N are mapped to color 1.  This
is useful when zooming in an area near the lake where no points in the
image have low iteration counts - it makes use of the low colors which
would otherwise be unused.
```

```
logmap=-N (< -1): Similar to logmap=N, but uses a square root
distribution of the colors instead of a logarithmic one.
```

```
logmap=2 or -2: Auto calculates the logmap value for maximum effect.
```

```
logmap=-1: "old" logarithmic palette.  It differs from logmap=1 in
that some colors are not used - logmap=1 "spreads" low color numbers
which are unused by logmap=-1's pure logarithmic mapping so that all
colors are assigned.
```

Another way to change the 1:1 mapping of iteration counts to colors is to use the "ranges=" parameter. It has the format: ranges=aa/bb/cc/dd/...

Iteration counts up to and including the first value are mapped to color number 0, up to and including the second value to color number 1, and so on. The values must be in ascending order.

A negative value can be specified for "striping". The negative value specifies a stripe width, the value following it specifies the limit of the striped range. Two alternating colors are used within the striped range.

Example: ranges=0/10/30/-5/65/79/32000 This example maps iteration counts to colors as follows:

```
    color    iterations
    -------------------
      0      unused (formula always iterates at least once)
      1       1 to 10
      2      11 to 30
      3      31 to 35, 41 to 45, 51 to 55, and 61 to 65
      4      36 to 40, 46 to 50, and 56 to 60
      5      66 to 79
      6      80 and greater
Note that the maximum value in a ranges parameter is 32767 and the
maximum value for the number of iterations is also 32767 when using
ranges.
```

Biomorphs

Related to Decomposition are the "biomorphs" invented by Clifford Pickover, and discussed by A. K.

Dewdney in the July 1989 "Scientific American", page 110. These are so-named because this coloring scheme makes many fractals look like one-celled animals. The idea is simple. The escape-time algorithm terminates an iterating formula when the size of the orbit value exceeds a predetermined bailout value. Normally the pixel corresponding to that orbit is colored according to the iteration when bailout happened. To create biomorphs, this is modified so that if EITHER the real OR the imaginary component is LESS than the bailout, then the pixel is set to the "biomorph" color. The effect is a bit better with higher bailout values: the bailout is automatically set to 100 when this option is in effect. You can try other values with the "bailout=" option.

The biomorph option is turned on via the "biomorph=nnn" command-line option (where "nnn" is the color to use on the affected pixels). When toggling to Julia sets, the default corners are three times bigger than normal to allow seeing the biomorph appendages. Does not work with all types - in particular it fails with any of the mandelsine family. However, it works great in two-color modes. Try it with the marksmandel and marksjulia types.

Continuous Potential

Note: This option can only be used with 256 color video modes.

Id's images are usually calculated by the "level set" method, producing bands of color corresponding to regions where the calculation gives the same value. When "3D" transformed (see "3D" Images), most images other than plasma clouds are like terraced landscapes: most of the surface is either horizontal or vertical.

To get the best results with the "illuminated" 3D fill options 5 and 6, there is an alternative approach that yields continuous changes in colors.

Continuous potential is approximated by calculating

$$\text{potential} = \log \frac{\text{modulus}}{2^{\text{iterations}}}$$

where "modulus" is the orbit value (magnitude of the complex number) when the modulus bailout was exceeded, at the "iterations" iteration. Clear as mud, right?

Fortunately, you don't have to understand all the details. However, there are a few points to understand. First, Id's criterion for halting a fractal calculation, the "modulus bailout value", is generally set to 4. Continuous potential is inaccurate at such a low value.

The parameters for continuous potential are: potential=maxcolor[/slope[/modulus[/16bit]]] These parameters are present on the `Y` options screen.

"Maxcolor" is the color corresponding to zero potential, which plots as the top of the mountain. Generally this should be set to one less than the number of colors, i.e. usually 255. Remember that the last few colors of the default palette are black, so you won't see what you are really getting unless you change to a different palette.

"Slope" affects how rapidly the colors change — the slope of the "mountains" created in 3D. If this is too low, the palette will not cover all the potential values and large areas will be black. If it is too high, the range of colors in the picture will be much less than those available. There is no easy way

to predict in advance what this value should be.

"Modulus" is the bailout value used to determine when an orbit has "escaped". Larger values give more accurate and smoother potential. A value of 500 gives excellent results.

"16bit": If you transform a continuous potential image to 3D, the illumination modes 5 and 6 will work fine, but the colors will look a bit granular. This is because even with 256 colors, the continuous potential is being truncated to integers. The "16bit" option can be used to add an extra 8 bits of goodness to each stored pixel, for a much smoother result when transforming to 3D.

Id's visible behavior is unchanged when 16bit is enabled, except that solid guessing and boundary tracing are not used. But when you save an image generated with 16bit continuous potential: * The saved file is a fair bit larger. * Id names the file with a .pot extension instead of .gif, if you didn't specify an extension in "savename". * The image can be used as input to a subsequent `3` command to get the promised smoother effect. * If you happen to view the saved image with a GIF viewer other than Id, you'll find that it is twice as wide as it is supposed to be. (Guess where the extra goodness was stored!) Though these files are structurally legal GIF files the double-width business made us think they should perhaps not be called GIF - hence the .pot filename extension.

Because .pot files store the extra 8 bits by doubling the saved GIF width, 16bit potential images can be saved only when the fractal image is no larger than 32767 pixels wide by 65535 pixels high.

A 16bit (.pot) file can be converted to an ordinary 8 bit GIF by `R`estoring it, changing "16bit" to "no" on the `Y` options screen, and `S`aving.

You might find with 16bit continuous potential that there's a long delay at the start of an image, and disk activity during calculation. Id uses its disk-video cache area to store the extra 8 bits per pixel - if there isn't sufficient memory available, the cache will page to disk.

The following commands can be used to recreate the image that Mark Peterson first prototyped for us, and named "MtMand":

```
type=mandel
corners=-0.19920/-0.11/1.0/1.06707
inside=255
maxiter=255
potential=255/2000/1000/16bit
passes=1
float=yes
```

Starfields

Once you have generated your favorite fractal image, you can convert it into a fractal starfield with the `A` transformation (for 'astronomy'? - once again, all of the good letters were gone already). Stars are generated on a pixel-by-pixel basis - the odds that a particular pixel will coalesce into a star are based (partially) on the color index of that pixel.

(The following was supplied by Mark Peterson, the starfield author.)

If the screen were entirely black and the "Star Density per Pixel" were set to 30 then a starfield transformation would create an evenly distributed starfield with an average of one star for every 30 pixels.

If you're on a 320x200 screen then you have 64000 pixels and would end up with about 2100 stars. By introducing the variable of "Clumpiness" we can create more stars in areas that have higher color values. At 100% clumpiness a color value of 255 will change the average of finding a star at that location to 50:50. A lower clumpiness values will lower the amount of probability weighting. To create a spiral galaxy draw your favorite spiral fractal (IFS, Julia, or Mandelbrot) and perform a starfield transformation. For general starfields I'd recommend transforming a plasma fractal.

Real starfields have many more dim stars than bright ones because very few stars are close enough to appear bright. To achieve this effect the program will create a bell curve based on the value of ratio of dim stars to bright stars. After calculating the bell curve the curve is folded in half and the peak used to represent the number of dim stars.

Starfields can only be shown in 256 colors. Id will automatically try to load altern.map and abort if the map file cannot be found.

Bailout Test

The bailout test is used to determine if we should stop iterating before the maximum iteration count is reached. This test compares the value determined by the test to the "bailout" value set via the `Z` screen. The default bailout test compares the magnitude or modulus of a complex variable to some bailout value:

bailout test = $|z| = \sqrt{x^2 + y^2} \geq 2$

As a computational speedup, we square both sides of this equation and the bailout test used by Id is:

bailout test = $|z|^2 = x^2 + y^2 \geq 4$

Using a bailout other than 4 allows us to change when the bailout will occur.

The following bailout tests have been implemented on the `Z` screen:

mod	$x^2 + y^2 \geq$ bailout
real	$x^2 \geq$ bailout
imag	$y^2 \geq$ bailout
or	$x^2 \geq bailout\ y^2 \geq$ bailout
and	$x^2 \geq bailout\ y^2 \geq$ bailout
manh	$(\text{abs}\ x + \text{abs}\ y)^2 \geq$ bailout (based on Manhattan metric)
manr	$(x + y)^2 \geq$ bailout

The bailout test feature has not been implemented for all applicable fractal types. This is due to the speedups used for these types.

Parameter Explorer/Evolver

Since Id is such a wonderfully complex program it has more than a few parameters to tweak and options to select. To the inexperienced user the choice is bewildering. Even for the experts the chaotic nature of the mathematical processes involved make it difficult to know what to change in order to achieve the desired effect.

In order to help with this situation the Id parameter evolver has been developed. It varies those parameters for you and puts the results on screen as a grid of small images. You can then choose the one which you like best and regenerate it full screen, or if you don't like any of the variations, you can try again to see if anything better turns up!

Enough explanations for now, lets see how easy it is to use:

With the default Mandlebrot set on the screen simply hold down the `Alt` + `1` key on the top row (DON'T use the numeric keypad to the right, it won't work). You'll see a screen full of images generated starting from the middle and spiraling outwards. The perfect Mandlebrot set will be in the middle and the others will be warped and distorted by having had the initial value of Z perturbed at random... but you don't need to know that (which is the whole point really!).

`Alt` + `1` produces a low level of mutation of the fractal, only 'mild' parameters are changed, those which have a more subtle effect. For much wilder variations try pressing `Alt` + `7` now. This give the maximum available mutation with just about everything being twiddled and fiddled to rather dramatic effect as you should now be seeing.

To select an image for full screen display simply bring up a zoombox by pressing 'Page-up' once. The center image will now have a white box around it. Hold down the `Ctrl` + `Arrow` keys to move this box around until it's outlining an image you like. Pressing `B` will now exit from evolver mode and redraw the selected image full size. If, rather than exiting from evolver mode, you just press `Enter`, then a whole new set of images is generated, all based around the one you selected (which is now the middle image of the new set).

From a basic point of view that's it! Just press `Alt` + `1` ... `Alt` + `7` to scramble things when you're out of inspiration, it works for any of the fractal types in Id including formulae... easy! (chaotic, but easy :-))

As this is an Id feature, there is, of course, a lot more to it than the basics described above...

For a start, there are some handy hotkeys to use, `F2` and `F3` are used to alter the amount of mutation or the amount by which the selected parameters can be varied. `F2` halves the amount of mutation, `F3` doubles it. So if things on-screen are looking a bit samey just press `F3` a few times to crank up the mutation level.

Using `F2` to decrease mutation levels is a way of moving towards a goal image. Say that a set of images contained one that looked a little like, maybe, a cats face and you wished to try and get something more cat like. To achieve this simply select the desired image and press `F2`. The newly generated images should be more alike, though probably still quite widely varied. With luck, one of the new images will be even more cat like. Select this one and press `F2` again. Continue like this, selecting the center image again if there are no improvements in the current generation, until

eventually all the images are alike and you've arrived at your goal (or at least you're probably as close as it's possible to get with that fractal type).

As you look for more details in the images it is useful to reduce the number of images per generation, thus producing larger sub images. Pressing `F4` will reduce the number of images per side on the grid by two and pressing `F5` increments the gridsize similarly.

`F6` will switch between normal random mutation and 'spread' random mutation. In 'spread' mode the amount of mutation allowed in an image is varied according to each images position in the grid. Those images near the center are allowed a lesser degree of freedom of mutation than those around the outside. This produces a sea of images, stable at the center with wilder variations around the edges. This mode is best used with larger gridsizes and becomes completely silly at a gridsize of three!

`Ctrl` + `E` brings up the evolver control screen on which you have manual access to the evolution parameters varied by the hotkeys described above. These are:

Parameter	Description
Gridsize	The number of sub images per side of the screen. Use odd numbers only.
Max Mutation	The maximum amount by which a parameter may be varied
Mutation Reduction	The Max mutation value is multiplied by this between generations.+ This can be used to automatically goal seek without having to use the <F2> key.
Grouting	Turns on or off the gap between sub images, at large values of gridsize this+ can reclaim valuable screen area for display use.

Pressing `F6` brings up a screen from which you can control what parameters get varied and in what manner. You'll notice that as well as the mutation modes 'random' and 'spread' there are other ways of stirring things around, read on……

As well as randomly mutating parameter values (referred to as 'evolver mode' or just 'evolving') a chosen set of parameters can be varied steadily across the screen thus allowing the user to explore what happens as a parameter is slowly changed ('explorer mode' or 'exploring'). For example, to get acquainted with parameter exploring and produce a map of the Julia sets, try this:

- Start Id and set the type to Julia and the resolution higher than 320x200, once the default Julia set has been generated, press `Ctrl` + `E` to bring up the evolver/explorer control panel.
- Set evolve mode to yes and then press `F6` to bring up the screen that allows you to choose what gets varied.
- Now set the first entry (param1) to 'x' and the second (param2) to 'y'. This tells Id to vary param1 (the real part of c) across the screen and param2 (the imaginary part of c) down the screen. Make sure all the other parameters are set to 'no' so that nothing else gets changed to confuse things.

- Press <Return> to go back to the main evolver control screen and you'll see that a few more items have appeared. These control just how much the parameters are varied across the screen and what their starting values should be, leave them as they are but increase gridsz to 15. Also switch on the parameter zoom box option.
- When you exit this control screen with the <Return> key, you'll see a grid of Julia sets generated all mapped out onto the imaginary plane, squint and you'll be able to spot the underlying Mset!
- When you press `PageUp` this time you'll notice that there are two boxes on screen with a larger box centered around the normal selection box. `Ctrl` + `PageUp` or `Ctrl` + `PageDown` varies the size of this box which represents the 'parameter' zoom box. The parameter zoombox allows you to look at smaller areas of the parameter space in more detail. To explain this further look at how the Julia sets change across the screen, around the area of 'seahorse valley' on the underlying Mset, the Julia sets undergo a sharp change in character. This area of change can be examined in more detail using parameter zooming. Make the outer zoombox a few grids across and select an image in the area of this change with the outer box straddling it. Look at the images right in the corners of the parm zoombox, when you press `Enter` and a new generation of images is generated the same images will be in the corners of the screen with more sub images between them, allowing a finer look at how the change progresses. In this way, you can observe the chaotic areas in parameter space with the unique pseudo four dimensional view offered by the explorer.

In the example shown above, you were just exploring the variation in two 'real' parameters, i.e. they can take fractional values, and the idea of being able to create an image half way between two others is valid. However, many of the parameters in Id are discrete, i.e. can be only one of a set of specific values. Examples of discrete parameters are inside coloring method or decomposition values and the way in which these are explored is different in that parameter zooming has no meaning for discrete parameters.

When a discrete parameter is set to vary with x or y it is simply cycled through all possible values and round again. Words are getting clumsy so it's time for another example methinks!

First press 'Insert' to restart Id and get everything back to its default values for a fresh start. Set the fractal type to 'fn*fn' this type requires the user to choose two trig functions and this choice is made on the 'Z' screen. There are around thirty different functions to choose from and checking out all the different combinations is a not inconsiderable task manually. With the explorer, however, it's a piece of cake!

Set the screen resolution to the highest you can view and press `Ctrl` + `E` to bring up the control panel and enable evolving mode. Set the gridsize to 29 and leave the parameters at their defaults. Now, press `F6` to enter 'variable tweak central' and set trig function 1 to 'x' and trig function 2 to 'y', and all the others to 'no'. Exit the two screens and you'll see generated all of the different combinations possible even if they are rather small examples!

To find out what particular combination of trig functions an image is using, just select the image using the zoombox and bring up the 'z' screen. You don't have to press `Enter`, simply highlighting the appropriate image with the Ctrl+arrow keys will do.

And that just about sums up the evolver! Much more could be written but it's better experienced, try writing your formulae with more variable parameters and trig functions so that their behavior

can be investigated.

Try using it with any fractal type, if in doubt just see what happens!

It should be noted here that some of the fractal types such as the IFS do not terminate, they run on forever and as such aren't usable with the evolver as the first sub image would never finish to allow the next one to generate. These fractal types are detected and you won't be allowed to start the evolver with these.

There now only remains to mention that you can save image sets and restore them later to carry on exploring from a different seed image: `S` saves and `R` restores as in normal Id operation and the screenfull is saved as a single GIF file.

Have fun! See Evolver Commands.

Random Dot Stereograms (RDS)

Random dot stereograms (RDS) are a way of encoding stereo images on a flat screen. Id can convert any image to a RDS using either the color number in the current palette or the grayscale value as depth. Try these steps. Generate a plasma fractal using the 256 color video mode. When the image on the screen is complete, press `Ctrl` + `S` ("S" for "Stereo"), and press `Enter` at the "RDS Parameters" screen prompt to accept the defaults. (More on the parameters in a moment.) The screen will be converted into a seemingly random collection of colored dots. Relax your eyes, looking through the screen rather than at the screen surface. The image will (hopefully) resolve itself into the hills and valleys of the 3D Plasma fractal.

Random-dot RDS output honors rseed=, so a fixed seed makes the dot pattern repeatable.

Because pressing the two-keyed `Ctrl` + `S` gets tiresome after a while, we have made `K` key a synonym for `Ctrl` + `S` for convenience. Don't get too attached to `K` though; we reserve the right to reuse it for another purpose later.

The RDS feature has five and sometimes six parameters. Pressing `Ctrl` + `S` always takes you to the parameter screen.

The first parameter allows you to control the depth effect. A larger value (positive or negative) exaggerates the sense of depth. If you make the depth negative, the high and low areas of the image are reversed. If your RDS image is streaky try either a lower depth factor or a higher resolution.

The second parameter indicates the overall width in inches of the image on your monitor. The default value is 10 inches. If you're having trouble seeing the image, enter the image width in inches. The issue here is that if the widest separation of left and right pixels is greater than the physical separation of your eyes, you will not be able to fuse the images. Conversely, a too-small separation may cause your eyes to hyper-converge (fuse the wrong pixels together). A larger width value reduces the width between left and right pixels. You can use the calibration feature to help set the width parameter - see below. Once you have found a good width setting, you can place the value in your sstools.ini file with the command stereowidth=<nnn>.

The third parameter allows you to control the method use to extract depth information from the original image. If your answer "no" at the "Use Grayscale value for Depth" prompt, then the color

number of each pixel will be used. This value is independent of active color palette. If you answer "yes" and the prompt, then the depth values are keyed to the brightness of the color, which will change if you change palettes.

The fourth parameter allows you to set the position of vertical stereo calibration bars to the middle or the top of the image, or have the bars initially turned off. Use this feature to help you adjust your eye's convergence to see the image. You will see two vertical bars on the screen. You can turn off and on these bars with the `Enter` or `Space` keys after generating the RDS image. If you save an RDS image by pressing `S`, if the bars are turned on at the time, they become a permanent part of the image.

As you relax your eyes and look past the screen, these bars will appear as four bars. When you adjust your eyes so that the two middle bars merge into one bar, the 3D image should appear. The bars are set for the average depth in the area near the bars. They should always be closer together than the physical separation of your eyes, but not much less than about 1.5 inches. About 1.75 inches is ideal for many images. The depth and screen width controls affect the width of the bars.

At the RDS Parameters screen, you can select bars at the middle of the screen or the top. If you select "none", the bars will initially be off, but immediately after generation of the image you can still turn on the bars with `Enter` or `Space` before you press any other keys. If the initial setting of the calibration bars is "none", then if the bars are turned on later they will appear in the middle. Hint: if you cycle the colors and find you can't see the calibration bar, press `Enter` or `Space` twice, and the bars will turn to a more visible color.

The fifth parameter asks if you want to use a texture map GIF file instead of using random dots. A texture map can give your RDS image a more interesting background texture than the random dots. If you answer "yes" at the "Use texture map?" prompt, Id will present you with a file selection list of GIF images. Id will then go ahead and transform your original image to RDS using the selected texture map to provide the "random" dots.

After you have selected a texture map file, the next time you reach the RDS Parameters screen you will see an additional prompt asking if you want to use the same texture map file again. Answering "yes" avoids the file selection menu.

The best images to use as texture maps are detailed textures with no solid spots. The default type=circle fractal works well, as do the barnsley fractals if you zoom in a little way. If the texture map is smaller than your RDS image, the texture map will be repeated to fill the space. If the texture map is larger, just the upper left corner of the texture map will be used.

The original image you are using for your stereogram is saved, so if you want to modify the stereogram parameters and try again, just press `Ctrl` + `S` (or `K`) to get the parameter screen, changes the parameters, and press `Enter`. The original image is restored and an RDS transform with the revised parameters is performed. If you press `S` when viewing an RDS image, after the RDS image is saved, the original is restored.

Try the RDS feature with continuous potential Mandelbrots as well as plasma fractals.

For a summary of keystrokes in RDS mode, see RDS Commands

Freestyle mode tutorial

It can be confusing working out what's going on in freestyle mode so here's a quick walk through...

Freestyle palette editing is intended to be a way of coloring an image in an intuitive fashion with the minimum of keyboard usage. In fact everything is controllable with the mouse, as the following shows:

To start with, generate a plasma type fractal as it has all 256 colors on screen at once. Now bring up the palette editor and press `W` to set up a grayscale palette as a blank canvas on which to splash some color. Pressing `F` puts us in freestyle mode... crosshairs appear on the screen and a color band is applied, centered on the cursor. Although, at the moment, the color of this band is gray and you won't see much!

In order to change the color of the band, hold down the primary mouse button and drag up and down. This changes the amount of red in the band. You'll see the values change in the status box above the palette grid. Double clicking the secondary mouse button changes the color component that's varied in an r-g-b-r-cycle.... try it out and conjure up any shade you like!

To vary the width of the band, drag up and down with the secondary button held down. Slower machines may show some 'lag' during this operation, so watch out as the mouse movements get buffered.

Once you've got the band in a satisfactory position then double click the primary button to fix it in place.

Continue like this for as long as you like, adding different colors to the gray palette.

You'll notice how the band relates to the existing color, the RGB values give the middle color which are then smoothly shaded out to the colors at the ends of the band. This can lead to some sudden jumps in the shading as the band is moved about the screen and the edges come to overlap different areas of color.

For really violent jumps in shading try starting with an image that has areas that change chaotically, such as a Mandlbrot set. You'll see what I mean when you move the cross hairs into an area close to the 'lake' where the change in value from one pixel to the next is sudden, chaotic and large. Watch out! — the strobing effect can be somewhat disturbing. This is nothing to worry about but just a consequence of the manipulation of the palette and the way in which the color bands are calculated.

I hope that you'll find this a useful tool in coloring an image. Remember that the `H` key can be used to hide the palette box and expose the whole image.

"3D" Images

3D Overview

Id can restore images in "3D". Important: we use quotation marks because it does not create images

of 3D fractal objects (there are such, but we're not there yet.) Instead, it restores .GIF images as a 3D projection or stereo image pair. The iteration values you've come to know and love, the ones that determine pixel colors, are translated into "height" so that your saved screen becomes a landscape viewed in perspective. You can even wrap the landscape onto a sphere for realistic-looking planets and moons that never existed outside your PC!

We suggest starting with a saved plasma-cloud screen. Hit `3` in main command mode to begin the process. Next, select the file to be transformed, and the video mode. (Usually you want the same video mode the file was generated in; other choices may or may not work.)

After hitting `3`, you'll be bombarded with a long series of options. Not to worry: all of them have defaults chosen to yield an acceptable starting image, so the first time out just pump your way through with the `Enter` key. When you enter a different value for any option, that becomes the default value the next time you hit `3`, so you can change one option at a time until you get what you want. Generally `Esc` will take you back to the previous screen.

Once you're familiar with the effects of the 3D option values you have a variety of options on how to specify them. You can specify them all on the command line, with an sstools.ini file, or with a parameter file.

Here's an example for you power Iterated Dynamicists, the command

```
id myfile savename=my3d 3d=yes batch=yes
```

would make Id load myfile.gif, re-plot it as a 3D landscape (taking all of the defaults), save the result as my3d.gif, and exit. By the time you've come back with that cup of coffee, you'll have a new world to view, if not conquer.

Note that the image created by 3D transformation is treated as if it were a plasma cloud - We have no idea how to retain the ability to zoom and pan around a 3D image that has been twisted, stretched, perspective- ized, and water-leveled. (Actually, we do, but it remains to be implemented in a future version of Id...)

3D Mode Selection

After hitting `3` and getting past the filename prompt and video mode selection, you're presented with a "3D Mode Selection" screen. If you wish to change the default for any of the following parameters, use the arrow keys to move through the menu. When you're satisfied press `Enter`.

Preview mode

Preview mode provides a rapid look at your transformed image using by skipping a lot of rows and filling the image in. Good for quickly

discovering the best parameters.

Show box

If you have selected preview mode you have another option to worry about. This is the option to show the image box in scaled and rotated coordinates x, y, and z. The box only appears in

rectangular transformations and shows how the final image will be oriented. If you select light source in the next screen, it will also show you the light source vector so you can tell where the light is coming from in relation to your image. Sorry no head or tail on the vector yet.

Coarseness

This sets how many divisions the image will be divided into in the y direction, if you select preview mode, ray tracing output, or grid fill in the "Select 3D Fill Type" screen.

Spherical projection

The next question asks if you want a sphere projection. This will take your image and map it onto a plane if you answer "no" or a sphere if you answer "yes" as described above. Try it and you'll see what we mean. See Spherical Projection.

Stereo

Stereo sound in Id? Well, not yet. Id now allows you to create 3D images for use with red/blue glasses like 3D comics you may have seen, or images like Captain EO.

Option 0 is normal old 3D you can look at with just your eyes.

Options 1 and 2 require the special red/blue-green glasses. They are meant to be viewed right on the screen or on a color print off of the screen. The image can be made to hover entirely or partially in front of the screen. Great fun! These two options give a gray scale image when viewed.

Option 1 gives 64 shades of gray but with half the spatial resolution you have selected. It works by writing the red and blue images on adjacent pixels, which is why it eats half your resolution. In general, we recommend you use this only with higher resolution video modes. Use this mode for continuous potential landscapes where you **need** all those shades.

Option 2 gives you full spatial resolution but with only 16 shades of gray. If the red and blue images overlap, the colors are mixed. Good for wire-frame images (we call them surface grids), lorenz3d and 3D IFS.

Option 3 is for creating stereo pair images for view later with more specialized equipment. It allows full color images to be presented in glorious stereo. The left image presented on the screen first. You may photograph it or save it. Then the second image is presented, you may do the same as the first image. You can then take the two images and convert them to a stereo image pair as outlined by Bruce Goren (see below).

Option 4 places left and right images on the screen simultaneously

as a stereo pair. Also see Stereo 3D Viewing.

Ray trace output

Id can create files of its 3D transformations which are compatible with many ray tracing programs. Currently four are supported directly: DKB (now obsolete), VIVID, MTV, RAYSHADE and DXF. In addition a "raw" output is supported which can be relatively easily transformed to be usable by many other products. One other option is supported: ACROSPIN. This is not a ray tracer, but the same Id options apply.

Table 4. Option Values

0	disables the creation of ray tracing output
1	DKB format (obsolete-see below)
2	VIVID format
3	generic format (must be massaged externally)
4	MTV format
5	RAYSHADE format
6	ACROSPIN format
7	DXF format

Users of POV-Ray can use the DKB output and convert to POV-Ray with the DKB2POV utility that comes with POV-Ray. A better (faster) approach is to create a raw output file and convert to POV-Ray with RAW2POV. A still better approach is to use POV-Ray's height field feature to directly read the fractal .gif or .pot file and do the 3D transformation inside POV-Ray.

All ray tracing files consist of triangles which follow the surface created by Id during the 3D transform. Triangles which lie below the "water line" are not created in order to avoid causing unnecessary work for the poor ray tracers which are already overworked. A simple plane can be substituted by the user at the waterline if needed.

The size (and therefore the number) of triangles created is determined by the "coarse" parameter setting. While generating the ray tracing file, you will view the image from above and watch it partitioned into triangles.

The color of each triangle is the average of the color of its vertices in the original image, unless brief is selected.

If brief is selected, a default color is assigned at the beginning of the

file and is used for all triangles.

+ Also see Interfacing with Ray Tracing Programs.

Brief output

This is a ray tracing sub-option. When it is set to yes, Id creates a considerably smaller and somewhat faster file. In this mode, all triangles use the default color specified at the beginning of the file. This color should be edited to supply the color of your choice.

Targa output

If you want any of the 3D transforms you select to be saved as a Targa-24 file or overlayed onto one, select yes for this option. The overlay option in the final screen determines whether you will create a new file or overlay an existing one.

Map file name

Imediately after selecting the previous options, you will be given the chance to select an alternate color map file. The default is to use the current map. If you want another map used,

then enter your selection at this point.

Output file name

This is a ray tracing sub-option, used to specify the name of the file to be written. The default name is fract001.ray. The name is incremented by one each time a file is written. If you have not set "overwrite=yes" then the file name will also be automatically incremented to avoid over-writing previous files.

When you are satisfied with your selections press enter to go to the next parameter screen.

Select Fill Type Screen

This option exists because in the course of the 3D projection, portions of the original image may be stretched to fit the new surface. Points of an image that formerly were right next to each other, now may have a space between them. This option generally determines what to do with the space between the mapped dots. It is not used if you have selected a value for ray other than 0.

For an illustration, pick the second option "just draw the points", which just maps points to corresponding points. Generally this will leave empty space between many of the points. Therefore you can choose various algorithms that "fill in" the space between the points in various ways.

Next, try the first option "make a surface grid." This option will make a grid of the surface which is as many divisions in the original "y" direction as was set in "coarse" in the first screen. It is very fast, and can give you a good idea what the final relationship of parts of your picture will look like.

Next, try the second option "connect the dots (wire frame)", then "surface fills" - "colors interpolated" and "colors not interpolated", the general favorites of the authors. Solid fill, while it reveals the pseudo-geology under your pseudo-landscape, inevitably takes longer.

Next, try the light source fill types. These two algorithms allow you to position the "sun" over your "landscape." Each pixel is colored according to the angle the surface makes with an imaginary light source. You will be asked to enter the three coordinates of the vector pointing toward the light in a following parameter screen - see Light Source Parameters.

"Light source before transformation" uses the illumination direction without transforming it. The light source is fixed relative to your computer screen. If you generate a sequence of images with progressive rotation, the effect is as if you and the light source are fixed and the object is rotating. Therefore, as the object rotates features of the object move in and out of the light.

"Light source after transformation" applies the same transformation to both the light direction and the object. Since both the light direction and the object are transformed, if you generate a sequence of images with the rotation progressively changed, the effect is as if the image and the light source are fixed in relation to each other and you orbit around the image. The illumination of features on the object is constant, but you see the object from different angles.

For ease of discussion we will refer to the following fill types by these numbers: 1 - surface grid 2 - (default) - no fill at all - just draw the dots 3 - wire frame - joins points with lines 4 - surface fill - (colors interpolated) 5 - surface fill - (interpolation turned off) 6 - solid fill - draws lines from the "ground" up to the point 7 - surface fill with light model - calculated before 3D transforms 8 -

surface fill with light model - calculated after 3D transforms

Types 4, 7, and 8 interpolate colors when filling, making a very smooth fill if the palette is continuous. This may not be desirable if the palette is not continuous. Type 5 is the same as type 4 with interpolation turned off. You might want to use fill type 5, for example, to project a .gif photograph onto a sphere. With type 4, you might see the filled-in points, since chances are the palette is not continuous; type 5 fills those same points in with the colors of adjacent pixels. However, for most fractal images, fill type 4 works better.

This screen is not available if you have selected a ray tracing option.

Stereo 3D Viewing

The "Funny Glasses Parameters" (stereo 3D) screen is presented only if you select a non-zero stereo option in the prior 3D parameters. (See 3D Mode Selection.) We suggest you definitely use defaults at first on this screen.

When you look at an image with both eyes, each eye sees the image in slightly different perspective because they see it from different places.

The first selection you must make is ocular separation, the distance the between the viewers eyes. This is measured as a % of screen and is an important factor in setting the position of the final stereo image in front of or behind the CRT Screen.

The second selection is convergence, also as a % of screen. This tends to move the image forward and back to set where it floats. More positive values move the image towards the viewer. The value of this parameter needs to be set in conjunction with the setting of ocular separation and the perspective distance. It directly adjusts the overall separation of the two stereo images. Beginning anaglyphers love to create images floating mystically in front of the screen, but grizzled old 3D veterans look upon such antics with disdain, and believe the image should be safely inside the monitor where it belongs!

Left and Right Red and Blue image crop (% of screen also) help keep the visible part of the right image the same as the visible part of the left by cropping them. If there is too much in the field of either eye that the other doesn't see, the stereo effect can be ruined.

Red and Blue brightness factor: the generally available red/blue-green glasses, made for viewing on ink on paper and not the light from a CRT, let in more red light in the blue-green lens than we would like. This leaves a ghost of the red image on the blue-green image (definitely not desired in stereo images). We have countered this by adjusting the intensity of the red and blue values on the CRT. In general you should not have to adjust this.

The final entry is the map file name (present only if stereo=1 or stereo=2 was selected). If you have a special map file you want to use for stereo 3D this is the place to enter its name. Generally glasses1.map is for type 1 (alternating pixels), and glasses2.map is for type 2 (superimposed pixels). Grid.map is great for wire-frame images using 16 color modes.

This screen is not available if you have selected a ray tracing option.

Rectangular Coordinate Transformation

The first entries are rotation values around the X, Y, and Z axes. Think of your starting image as a flat map: the X value tilts the bottom of your monitor towards you by X degrees, the Y value pulls the left side of the monitor towards you, and the Z value spins it counter- clockwise. Note that these are NOT independent rotations: the image is rotated first along the X-axis, then along the Y-axis, and finally along the Z-axis. Those are your axes, not those of your (by now hopelessly skewed) monitor. All rotations actually occur through the center of the original image. Rotation parameters are not used when a ray tracing option has been selected.

Then there are three scaling factors in percent. Initially, leave the X and Y axes alone and play with Z, now the vertical axis, which translates into surface "roughness." High values of Z make spiky, on-beyond-Alpine mountains and improbably deep valleys; low values make gentle, rolling terrain. Negative roughness is legal: if you're doing an M-set image and want Mandelbrot Lake to be below the ground, instead of eerily floating above, try a roughness of about -30%.

Next we need a water level — really a minimum-color value that performs the function "if (color < waterlevel) color = waterlevel". So it plots all colors "below" the one you choose at the level of that color, with the effect of filling in "valleys" and converting them to "lakes."

Now we enter a perspective distance, which you can think of as the "distance" from your eye to the image. A zero value (the default) means no perspective calculations, which allows use of a faster algorithm. Perspective distance is not available if you have selected a ray tracing option.

For non-zero values, picture a box with the original X-Y plane of your flat fractal on the bottom, and your 3D fractal inside. A perspective value of 100% places your eye right at the edge of the box and yields fairly severe distortion, like a close view through a wide-angle lens. 200% puts your eye as far from the front of the box as the back is behind. 300% puts your eye twice as far from the front of the box as the back is, etc. Try about 150% for reasonable results. Much larger values put you far away for even less distortion, while values smaller than 100% put you "inside" the box. Try larger values first, and work your way in.

Next, you are prompted for two types of X and Y shifts (now back in the plane of your screen) that let you move the final image around if you'd like to re-center it. The first set, x and y shift with perspective, move the image and the effect changes the perspective you see. The second set, "x and y adjust without perspective", move the image but do not change perspective. They are used just for positioning the final image on the screen. Shifting of any type is not available if you have selected a ray tracing option.

3D Color Parameters

You are asked for a range of "transparent" colors, if any. This option is most useful when using the 3D Overlay Mode. Enter the color range (minimum and maximum value) for which you do not want to overwrite whatever may already be on the screen. The default is no transparency (overwrite everything).

Now, for the final option. This one will smooth the transition between colors by randomizing them and reduce the banding that occurs with some maps. Select the value of randomize to between 0

(for no effect) and 7 (to randomize your colors almost beyond use). 3 is a good starting point. This randomization honors rseed=.

That's all for this screen. Press `Enter` for these parameters and the next and final screen will appear (honestly!).

Light Source Parameters

This one deals with all the aspects of light source and Targa files.

You must choose the direction of the light from the light source. This will be scaled in the x, y, and z directions the same as the image. For example, 1,1,3 positions the light to come from the lower right front of the screen in relation to the untransformed image. It is important to remember that these coordinates are scaled the same as your image. Thus, "1,1,1" positions the light to come from a direction of equal distances to the right, below and in front of each pixel on the original image. However, if the x,y,z scale is set to 90,90,30 the result will be from equal distances to the right and below each pixel but from only 1/3 the distance in front of the screen i.e.. it will be low in the sky, say, afternoon or morning.

Then you are asked for a smoothing factor. Unless you used Continuous Potential when generating the starting image, the illumination when using light source fills may appear "sparkly", like a sandy beach in bright sun. A smoothing factor of 2 or 3 will allow you to see the large-scale shapes better.

Smoothing is primarily useful when doing light source fill types with plasma clouds. If your fractal is not a plasma cloud and has features with sharply defined boundaries (e.g. Mandelbrot Lake), smoothing may cause the colors to run. This is a feature, not a bug. (A copyrighted response of [your favorite commercial software company here], used by permission.)

The ambient option sets the minimum light value a surface has if it has no direct lighting at all. All light values are scaled from this value to white. This effectively adjusts the depth of the shadows and sets the overall contrast of the image.

If you selected the full color option, you have a few more choices. The next is the haze factor. Set this to make distant objects more hazy. Close up objects will have little effect, distant objects will have most. 0 disables the function. 100 is the maximum effect, the farthest objects will be lost in the mist. Currently, this does not really use distance from the viewer, we cheat and use the y value of the original image. So the effect really only works if the y-rotation (set earlier) is between +/- 30.

Next, you can choose the name under which to save your Targa file. If you have not set "overwrite=yes" then the file name will be incremented to avoid over-writing previous files. If you are going to overlay an existing Targa file, enter its name here.

Next, you may select the background color for the Targa file. The default background on the Targa file is sky blue. Enter the Red, Green, and Blue component for the background color you wish.

Finally, absolutely the last option (this time we mean it): you can now choose to overlay an existing Targa-24, type 2, non mapped, top-to- bottom file, such as created by Id or PVRay. The Targa file specified above will be overlayed with new info just as a GIF is overlayed on screen. Note: it is not necessary to use the `0` overlay command to overlay Targa files. The targa_overlay parameter must

be set to yes, however.

You'll probably want to adjust the final colors for monochrome fill types using light source via color cycling. Try one of the more continuous palettes (`F8` through `F10`), or load the gray palette with the `A`lternate-map command.

Now, lie down for a while in a quiet room with a damp washcloth on your forehead. Feeling better? Good — because it's time to go back almost to the top of the 3D options and just say yes to:

Spherical Projection

Picture a globe lying on its side, "north" pole to the right. (It's our planet, and we'll position it the way we like.) You will be mapping the X and Y axes of the starting image to latitude and longitude on the globe, so that what was a horizontal row of pixels follows a line of longitude. The defaults exactly cover the hemisphere facing you, from longitude 180 degrees (top) to 0 degrees (bottom) and latitude -90 (left) to latitude 90 (right). By changing them you can map the image to a piece of the hemisphere or wrap it clear around the globe.

The next entry is for a radius factor that controls the over-all size of the globe. All the rest of the entries are the same as in the landscape projection. You may want less surface roughness for a plausible look, unless you prefer small worlds with big topography, a la "The Little Prince."

WARNING

When the "construction" process begins at the edge of the globe (default) or behind it, it's plotting points that will be hidden by subsequent points as the process sweeps around the sphere toward you. Our nifty hidden-point algorithms "know" this, and the first few dozen lines may be invisible unless a high mountain happens to poke over the horizon. If you start a spherical projection and the screen stays black, wait for a while (a longer while for higher resolution or fill type 6) to see if points start to appear. Would we lie to you? If you're still waiting hours later, first check that the power's still on, then consider a faster system.

3D Overlay Mode

While the `3` command (see "3D" Images) creates its image on a blank screen, the `#` (or `Shift` + `3`) command draws a second image over an existing displayed image. This image can be any restored image from a `R` command or the result of a just executed `3` command. So you can do a landscape, then press `#` and choose spherical projection to re-plot that image or another as a moon in the sky above the landscape. `#` can be repeated as many times as you like.

It's worth noting that not all that many years ago, one of us watched Benoit Mandelbrot and fractal-graphics wizard Dick Voss creating just such a moon-over-landscape image at IBM's research center in Yorktown Heights, NY. The system was a large and impressive mainframe with floating-point facilities, running LBLGRAPH — what Mandelbrot calls "an independent-minded and often very ill-mannered heap of graphics programs that originated in work by Alex Hurwitz and Jack Wright of IBM Los Angeles."

We'd like to salute LBLGRAPH, its successors, and their creators, because it was their graphic

output (like "Planetrise over Labelgraph Hill", plate C9 in Mandelbrot's "Fractal Geometry of Nature") that helped turn fractal geometry from a mathematical curiosity into a phenomenon. We'd also like to point out that it wasn't as fast, flexible or pretty as Id. Now, a lot of the difference has to do with the incredible progress of micro-processor power since then, so a lot of the credit should go to Intel rather than to our highly tuned code. OK, twist our arms — it is awfully good code.

Making Terrains

If you enjoy using Id for making landscapes, we have several features for you to work with. When doing 3D transformations, banding tends to occur because all pixels of a given height end up the same color. Now, colors can be randomized to make the transitions between different colors at different altitudes smoother. Use the "randomize=n" parameter to accomplish this; use "rseed=n" as well when you need repeatable randomization. If your light source images all look like lunar landscapes since they are all monochrome and have very dark shadows, we now allow you to set the ambient light for adjusting the contrast of the final image. Use the "ambient=n" parameter. In addition to being able to create scenes with light sources in monochrome, you can now do it in full color as well. Setting the parameter "fullcolor=1" will generate a Targa-24 file with a full color image which will be a combination of the original colors of the source image (or map file if you select map=something) and the amount of light which reflects off a given point on the surface. Since there can be 256 different colors in the original image and 256 levels of light, you can now generate an image with lots of colors.

Using the full color option allows you to also set a haze factor with the "haze=n" parameter to make more distant objects more hazy.

As a default, full color files also have the background set to sky blue. The file is always created on disk. Try the following settings of the new parameters in sequence to get a feel for the effect of each one: ; use this with any filltype map=topo randomize=3 ; adjusting this smooths color transitions

```
                ; now add this using filltype 5 or 6
ambient=20      ; adjusting this changes the contrast
filltype=6
smoothing=2     ; makes the light not quite as granular
                ; as the terrain
```

```
                ; now add the following
fullcolor=1     ; output a Targa lightfile
```

```
                ; and finally this
haze=20         ; sets the amount of haze for distant objects
```

When full color is being used, the image you see on the screen will represent the amount of light being reflected, not the colors in the final image. Don't be disturbed if the colors look weird, they are an artifact of the process being used. The image being created in the lightfile won't look like the screen.

However, if you are worried, hit `Esc` several times and when Id gets to the end of the current line it will abort. Your partial image will be there as fract001.tga or with whatever file name you selected with the lightname option. Examine it and adjust any parameters you are not happy with. Its a little awkward, but we haven't figured out a better way — yet.

Making 3D Slides

Bruce Goren, a stereoscopic maven, contributed these tips on what to do with your 3D images. (Bruce inspired and prodded us so much we automated much of what follows, allowing both this and actual on screen stereo viewing, but we included it here for reference and a brief tutorial.)

"I move the viewport or imaginary camera left and right to create two separate views of the stationary object in x,y,z, space. The distance between the two views, known as the inter-ocular distance, toe-in or convergence angle, is critical. It makes the difference between good 3- D and headache-generating bad 3-D.

"For a 3D fractal landscape, I created and photographed the left and right eye views as if flying by in an imaginary airplane and mounted the film chips for stereo viewing. To make my image, first I generated a plasma cloud based on a color map I calculated to resemble a geological survey map. In the 3D reconstruction, I used a perspective value of 150 and shifted the camera -15 and +15 on the X-axis for the left and right views. All other values were left to the defaults.

"The images are captured on a Matrix 3000 film recorder — basically a box with a high-resolution black and white TV and a 35mm film camera (Konica FS-1) looking at the TV screen through a filter wheel. I glass mount the film chips myself.

"Each frame is exposed three times, once through each of the red, blue, and green filters to create a color image from computer video without the scan-lines which normally result from photographing television screens. The aspect ratio of the resulting images led me to mount the chips using the 7-sprocket Busch-European Emde masks."

Interfacing with Ray Tracing Programs

(Also see "Ray Tracing Output", "Brief", and "Output File Name" in 3D Mode Selection.)

Id allows you to save your 3D transforms in files which may be fed to a ray tracer (or to "Acrospin"). However, they are not ready to be traced by themselves. For one thing, no light source is included. They are actually meant to be included within other ray tracing files.

Since the intent is to produce an object which may be included in a larger ray tracing scene, it is expected that all rotations, shifts, and final scaling will be done by the ray tracer. Thus, in creating the images, no facilities for rotations or shifting is provided. Scaling is provided to achieve the correct aspect ratio.

WARNING! The files created using the ray tracing options can be large. Setting coarse to 40 will result in over 2000 triangles. Each triangle can utilize from 50 to 200 bytes each to describe, so your ray tracing files can rapidly approach or exceed 1 MB.

Each file starts with a comment identifying the version of Id by which it was created. The file ends

with a comment giving the number of triangles in the file.

The files consist of long strips of adjacent triangles. Triangles vertices are listed in clockwise or counter clockwise order depending on the target ray tracer. Currently, MTV and Rayshade are the only ones which use counter clockwise order. The size of the triangles is set by the coarse setting in the main 3D menu. Color information about each individual triangle is included for all files unless in the brief mode.

To keep the poor ray tracer from working too hard, if waterline is set to a non zero value, no triangle which lies entirely at or below the current setting of waterline is written to the ray tracing file. These may be replaced by a simple plane in the syntax of the ray tracer you are using.

Id's coordinate system has the origin of the x-y plane at the upper left hand corner of the screen, with positive x to the right and positive y down. The ray tracing files have the origin of the x-y plane moved to the center of the screen with positive x to the right and positive y up. Increasing values of the color index are out of the screen and in the +z direction. The color index 0 will be found in the xy plane at z=-1.

When x, y and z scale are set to 100, the surface created by the triangles will fall within a box of +/- 1.0 in all 3 directions. Changing scale will change the size and/or aspect ratio of the enclosed object.

We will only describe the structure of the raw format here. If you want to understand any of the ray tracing file formats besides raw, please see your favorite ray tracer docs.

The raw format simply consists of a series of clockwise triangles. If brief=yes, Each line is a vertex with coordinates x, y, and z. Each triangle is separated by a couple of CR's from the next. If brief=no, the first line in each triangle description if the r,g,b value of the triangle.

Setting brief=yes produces shorter files with the color of each triangle removed - all triangles will be the same color. These files are otherwise identical to normal files but will run faster than the non brief files. Also, with brief=yes, you may be able to get files with more triangles to run than with brief=no.

The DKB format is now obsolete. POV-Ray users should use the raw output and convert to POV-Ray using the POV Group's RAW2POV utility. POV-Ray users can also do all 3D transformations within POV-Ray using height fields.

Command Line Parameters, Parameter Files, Batch Mode

Introduction to Parameters

Id accepts command-line parameters that allow you to start it with a particular video mode, fractal type, starting coordinates, and just about every other parameter and option.

These parameters can also be specified in a sstools.ini file, to set them every time you run Id.

They can also be specified as named groups in a .par (parameter) file which you can then call up

while running Id by using the `@` command.

In all three cases (command line, sstools.ini, and parameter file) the parameters use the same syntax, usually a series of keyword=value commands like sound=off. Each parameter is described in detail in subsequent sections.

Using the Command Line

You can specify parameters when you start Id by using a command like:

```
id sound=off filename=myimage.gif
```

The individual parameters are separated by one or more spaces (an parameter itself may not include spaces). Upper or lower case may be used, and parameters can be in any order.

Id has a special command you can use when you have a lot of startup parameters (or have a set of parameters you use frequently):

```
id @myfile
```

When @filename is specified on the command line, Id reads parameters from the specified file as if they were keyed on the command line. You can create the file with a text editor, putting one "keyword=value" parameter on each line.

Libraries

Id has a variety of associated data files: formula definitions, IFS definitions, autokey scripts, L-system definitions, color maps, and parameter set definitions. Additionally, Id can output other types of files such as raw orbit dumps, raytrace output and so-on.

Id knows about two kinds of libraries: one or more read libraries and a single save library. A read library is used to locate input files. A save library is used for writing output files. Read library locations are specified with the librarydirs parameter and the save library location is specified with the savedir parameter.

The default save library is "Iterated Dynamics" in the user's Documents folder on Windows. On Linux, the Documents folder is taken from XDG_DOCUMENTS_DIR or from user-dirs.dirs in XDG_CONFIG_HOME or $HOME/.config. If no Linux Documents folder is configured, $HOME/Documents is used. On macOS, the user's $HOME/Documents folder is used.

To locate an input file, Id looks in the following locations. First, if the input file is specified as an absolute path, it is loaded directly without consulting the library locations.

Next, each read library is examined followed by the save library. Using the save library as a last ditch read library allows Id to easily use any created output files as subsequent input files. For each library, the appropriate subdirectory for the type of file is examined. For instance, if looking for a formula file the formula subdirectory is examined first.

If the file isn't found in the appropriate subdirectory of any of the library locations, then each library location is consulted again, only this time the library directory itself is examined to see if it contains the file.

Finally, the value of the FRACTDIR environment variable is treated as a library location if it is defined.

The following subdirectories are preferred to locate input files from read libraries:

```
config          Configuration files (id.cfg)
formula         Formula definition files (.frm)
ifs             IFS definitions files (.ifs)
image           Image files (.gif)
key             Autokey files (.key)
lsystem         L-system definition files (.l)
map             Colormap definition files (.map)
par             Parameter set definition files (.par)
```

The following subdirectories in the save library are created for output files:

```
config          Configuration files (id.cfg)
debug           Debug output files (.txt, .json)
formula         Formula definition files (.frm)
ifs             IFS definitions files (.ifs)
image           Image files (.gif)
key             Autokey files (.key)
lsystem         L-system definition files (.l)
map             Colormap definition files (.map)
orbit           Raw orbit files (.raw)
par             Parameter set definition files (.par)
raytrace        Raytrace output files (.ray)
sound           Raw sound output (.txt)
```

Some files, such as MIG scripts, are written directly to the save library location and not to a subdirectory.

Setting Defaults (sstools.ini File)

Every time Id runs, it searches the current directory, and then the directories in your PATH, for a file named sstools.ini. If it finds this file, it begins by reading parameters from it. This file is useful for setting parameters you always want.

The file sstools.ini is divided into sections belonging to particular programs. Each section begins with a label in brackets. Id looks for the label [id], and ignores any lines it finds in the file belonging to any other label. If an sstools.ini file looks like this:

```
[id]
```

```
sound=off      ; (for home use only)
inside=0       ; using "traditional" black
[startrek]
warp=9.5       ; Captain, I dinna think the engines can take it!
```

Id will use only the second and third lines of the file. (Why use a convention like that when Id is the only program you know of that uses an sstools.ini file? Because there are other programs (such as Lee Crocker's PICLAB) that now use the same file, and there may one day be other, sister programs to Id using that file.)

Parameter Files and the `@` Command

You can change parameters on-the-fly while running Id by using the `@` or `2` command and a parameter file. Parameter files contain named groups of parameters, looking something like this:

```
quickdraw {      ; a set of parameters named quickdraw
   maxiter=150
   float=no
}
```

```
slowdraw {       ; another set of parameters
   maxiter=2000
   float=yes
}
```

If you use the `@` or `2` command and select a parameter file containing the above example, Id will show two choices: quickdraw and slowdraw. You move the cursor to highlight one of the choices and press `Enter` to set the parameters specified in the file by that choice.

The default parameter file name is id.par. A different file can be selected with the "parmfile=" option, or by using `@` or `2` and then hitting `F6`.

You can create parameter files with a text editor, or for some uses, by using the `B` command. Parameter files can be used in a number of ways, some examples:

- To save the parameters for a favorite image. Id can do this for you with the `B` command.
- To save favorite sets of 3D transformation parameters. Id can do this for you with the `B` command.
- To set up different sets of parameters you use occasionally.
- To save image parameters for later use in batch mode - see Batch Mode.

Formulas, ifs, and lsystem entries referred to in a parameter entry can be included in a .par file by adding the prefix frm:, ifs:, or lsys: respectively, for example frm:myformula {rest of that formula}. Note that the prefix is a label, not part of the formula name, so the reference in the image entry would be formulaname=myformula. The formula, ifs, and lsystem entries added to a parfile are

accessed only when the image entry in the parfile is run. To make these formulas generally accessible, they must be added to a .frm, .ifs or .l file (without the identifying prefix, of course).

See Parameter Save/Restore Commands for details about the `@` and `B` commands.

General Parameter Syntax

Parameters must be separated by one or more spaces.

Upper and lower case can be used in keywords and values. In the parameter reference that follows, the name is shown in upper case. When mentioned in other contexts the name is generally shown in lower case.

Anything on a line following a ; (semi-colon) is ignored, i.e. is a comment.

In parameter files and sstools.ini: * Individual parameters can be entered on separate lines. * Long values can be split onto multiple lines by ending a line with a \ (backslash) - leading spaces on the following line are ignored, the information on the next line from the first non-blank character onward is appended to the prior line.

Some terminology: KEYWORD=nnn enter a number in place of "nnn" KEYWORD=[filename] you supply filename KEYWORD=yes|no|whatever choose one of "yes", "no", or "whatever" KEYWORD=1st[/2nd[/3rd]] the slash-separated parameters "2nd" and "3rd" are optional

Startup Parameters

@FILENAME Causes Id to read "filename" for parameters. When it finishes, it resumes reading its own command line — i.e., "id maxiter=250 @myfile passes=1" is legal. This option is only valid on the command line, as Id is not clever enough to deal with multiple indirection. If the file contains multiple parameter sets, the first parameter set is used.

@FILENAME/groupname Like @FILENAME, but reads a named group of parameters from a parameter file. See Parameter Files and the `@` Command.

TEMPDIR=[directory] This command allows to specify the directory where Id writes temporary files.

WORKDIR=[directory] This command sets the directory where miscellaneous non-debug Id files get written. Debug output is written under the save library debug subdirectory.

DEBUGFLAG=nnn, DEBUG=nnn This developer option enables internal debugging behavior. Debug output files are written under the save library debug subdirectory. New debug files honor overwrite=no by choosing an incremented name, while append- mode debug logs keep appending to the same file.

FILENAME=[name] Causes Id to read the named file, which must either have been saved from an earlier Id session or be a generic GIF file, and use that as the starting point, bypassing the initial information screens. The filetype is optional and defaults to .gif. Non-Id GIF files are restored as fractal type "plasma".

On the command line you may omit filename= and just give the file name.

CURDIR=yes Id uses directories set by various commands, possibly in the sstools.ini file. If you want to try out some files in the current directory, such as a modified copy of id.frm, you won't want Id to read the copy in your official FRM directory. Setting curdir=yes at the command line will cause Id to look in the current directory for requested files first before looking in the default directory set by the other commands. Warning: `Tab` screen may not reflect actual file opened in cases where the file was opened in the current directory.

MAKEPAR=parfile/entryname This command invokes a batch facility to copy fractal and color information stored in GIF files to par format. The syntax is: id filename.gif makepar=parfile.par/entryname The entryname is optional and defaults to the name of the GIF filename if absent. Other parameters can appear before the makepar= command, but anything after will ignored. If the parfile and entryname exist, the entry will replace the previous entry. If the entry doesn't exist, it will be added. If the parfile doesn't exist it will be created.

If you leave the GIF filename out of the command lime but add a map= command, then the makepar command will write a par named for the color map with only the colors in the par entry. This is a handy tool for converting maps into compressed par colors entry. For example, you could type: id map=lyapunov.map makepar=mycolors This adds a colors-only par entry called lyapunov.map to mycolors.par.

MAXLINELENGTH=nnn This command sets the maximum width of lines in a par entry.

COMMENT=[comment1]/[comment2]/[comment3]/[comment4] Inserts comments into par files. These comments can include variables that are expanded at the time the par file is created. Variables are indicated by $varname$. Underscore characters are expanded to blanks. If you want to include the special characters '$', '_', or '\' in a comment, precede the character with '\'. Supported variables are:

```
Variable  Expands to   Example   Variable   Expands to    Example
==============================   ================================
$year$    time:year    2024      $date$     mo. day, yr   Mar 17, 2024
$month$   time:month   Mar       $calctime$ h:m:s         4:34:45.3
$day$     time:day     17        $version$  version       100
$hour$    time:hour    21        $patch$    patch number  0
$min$     time:minute  34        $xdots$    horiz res.    640
$sec$     time:sec     14        $ydots$    vertical res. 480
$time$    time:h:m:s   21:34:14  $vidkey$   video key     SF4
```

The variable cpu expands to information about your CPU, such as "Intel Core i7-6600U CPU @ 2.60GHz".

You can leave any of the four comment fields unchanged by leaving that position blank. For example, the command comment=//Created_$date$ inserts the text "Created Mar 17, 2024" into the third comment.

BATCH=yes See Batch Mode.

AUTOKEY=play|record Specifying "play" runs Id in playback mode - keystrokes are read from the autokey file (see next parameter) and interpreted as if they're being entered from the keyboard.

Specifying "record" runs Id in recording mode - all keystrokes are recorded in the autokey file. Unless overwrite=yes is specified, the autokey filename will be incremented to avoid overwriting existing files.

See also Autokey Mode.

AUTOKEYNAME=[filename] Specifies the file name to be used in autokey mode. The default file name is auto.key.

MAKEDOC[=filename] Create Id documentation file (for printing or viewing with a text editor) and then exit. Filename defaults to id.txt. There's also a function in Id's online help which can be used to produce the documentation file - use "Printing Id Documentation" from the main help index.

MAXHISTORY=<nnn> Id maintains a list of parameters of the past 10 images that you generated in the current Id session. You can revisit these images using the `H` and `Ctrl` + `H` commands. The maxhistory parameter allows you to set the number of image parameter sets stored in memory. Keep in mind that every time you color cycle, another image parameter set is saved, so the default ten images are used up quickly.

Calculation Mode Parameters

PASSES=1|2|3|g|g1|g2|g3|g4|g5|g6|b|t|s|o|p Selects single-pass, dual-pass, triple-pass, solid-guessing mode, solid- guessing stop after pass n, boundary tracing, tesseral, synchronous orbits, orbits, or the perturbation algorithm. See Drawing Method and Passes Parameters.

FILLCOLOR=normal|<nnn> Sets a color to be used for block fill by boundary tracing and tesseral algorithms. See Drawing Method.

SYMMETRY=xxx Forces symmetry to none, X axis, Y axis, XY axis, origin, or pi symmetry. Useful as a speedup for symmetrical fractals. This is not a kaleidoscope feature for imposing symmetry where it doesn't exist. Use only when the fractal actual exhibits the symmetry, or else results may not be satisfactory.

BFDIGITS=<nnn> Forces nnn digits if arbitrary precision used. You can use this if Id's precision detection changes to arbitrary precision too late and regular double precision is used with poor results.

MATHTOLERANCE=<nnn>/<nnn> This commands controls the logic that automatically selects one of float/arbitrary precision based on precision requirements of the current zoom depth. The first number was used to control the integer/float transition and is now ignored. The second number controls the float/arbitrary precision transition. The default value of .05 means that the ratio between the exact and calculated width and height is between .95 and 1.05. A larger value than .05 (say .10) makes the test looser so that the lower precision math is used longer. A value ⇐ 0 means the test is always failed and the higher precision math type is used. A value >= 1 means that the test is always passed and the lower precision math type is used.

MINSTACK=<nnn> This sets the minimum number of stack bytes required for passes=s in order to

do another SOI recursion. If you get bad results, try setting this to a value above the default value of 1100. If the value is too large, the image will be OK but generation will be slower.

Fractal Type Parameters

TYPE=[name] Selects the fractal type to calculate. The default is type "mandel".

PARAMS=n/n/n/n... Set optional (required, for some fractal types) values used in the calculations. These numbers typically represent the real and imaginary portions of some startup value, and are described in detail as needed in Fractal Types. (Example: "id type=julia params=-0.48/0.626" would wait at the opening screen for you to select a video mode, but then proceed straight to the Julia set for the stated x (real) and y (imaginary) coordinates.)

FUNCTION=[fn1[/fn2[/fn3[/fn4]]]] Allows setting variable functions found in some fractal type formulae. Possible values are sin, cos, tan, cotan, sinh, cosh, tanh, cotanh, exp, log, sqr, recip (i.e. 1/z), ident (i.e. identity), cosxx (cos with a bug), flip, zero, one, asin, asinh, acos, acosh, atan, atanh, sqrt, abs (abs(x)+i*abs(y)), cabs (sqrt(x*x+y*y)), and the various rounding-to- integer functions: floor (round down), ceil (round up), trunc (round toward zero), and round (round to nearest). At reset, fn1 defaults to sin, fn2 defaults to sqr, fn3 defaults to sinh, and fn4 defaults to cosh. A function= command with fewer than four values leaves the omitted functions unchanged; function= with no values leaves all current functions unchanged.

FORMULANAME=[formulaname] Specifies the default formula name for type=formula fractals. (That is, the name of a formula defined in the formulafile.) Required if you want to generate one of these fractal types in batch mode, as this is the only way to specify a formula name in that case.

LNAME=[lsystemname] Specifies the default L-system name. (That is, the name of an entry in the lfile.) Required if you want to generate one of these fractal types in batch mode, as this is the only way to specify an L-system name in that case.

IFS=[ifsname] Specifies the default IFS name. (That is, the name of an entry in the ifsfile.) Required if you want to generate one of these fractal types in batch mode, as this is the only way to specify an IFS name in that case.

Image Calculation Parameters

MAXITER=nnn Reset the iteration maximum — the number of iterations at which the program gives up and says "OK, this point seems to be part of the set in question and should be colored with the inside color." — from the default 150. Values range from 2 to 2,147,483,647 (super-high iteration limits like 200000000 are useful when using logarithmic palettes). See The Mandelbrot Set for a description of the iteration method of calculating fractals. This parameter can also be used to adjust the number of orbits plotted for 3D "attractor" fractal types such as lorenz3d and kamtorus.

CORNERS=[xmin/xmax/ymin/ymax[/x3rd/y3rd]] Example: corners=-0.739/-0.736/0.288/0.291 Begin with these coordinates as the range of x and y coordinates, rather than the default values of (for type=mandel) -2.0/2.0/-1.5/1.5. When you specify four values (the usual case), this defines a rectangle: x coordinates are mapped to the screen, left to right, from xmin to xmax, y coordinates are mapped to the screen, bottom to top, from ymin to ymax. Six parameters can be used to

describe any rotated or stretched parallelogram: (xmin,ymax) are used for the top-left corner of the screen, (xmax,ymin) are used for the bottom-right corner, and (x3rd,y3rd) are used for the bottom-left corner. Entering just "corners=" tells Id to use this form rather than "center- mag" (see below) when saving parameters with the `B` command.

CENTER-MAG=[Xctr/Yctr/Mag[/Xmagfactor/rotation/skew]] This is an alternative way to enter corners as a center point and a magnification that is popular with some fractal programs and publications. Entering just "center-mag=" tells Id to use this form (the default mode) rather than "corners=" (see above) when saving parameters with the `B` command. The `Tab` status display shows the "corners" in both forms. When you specify three values (the usual case), this defines a rectangle: (Xctr, Yctr) specifies the coordinates of the center of the image while Mag indicates the amount of magnification to use. Six parameters can be used to describe any rotated or stretched parallelogram: Xmagfactor tells how many times bigger the x-magnification is than the y-magnification, rotation indicates how many degrees the image has been turned, and skew tells how many degrees the image is leaning over. Positive angles will rotate and skew the image counter-clockwise.

BAILOUT=nnn Overrides the default bailout criterion for escape-time fractals. Can also be set from the parameters screen after selecting a fractal type. See description of bailout in The Mandelbrot Set.

BAILOUTEST=mod|real|imag|or|and|manh|manr Specifies the Bailout Test used to determine when the fractal calculation has exceeded the bailout value. The default is mod and not all fractal types can utilize the additional tests.

RESET Causes Id to reset all calculation related parameters to their default values. Non-calculation parameters such as "sound=", and "savename=" are not affected. Reset should be specified at the start of each parameter file entry (used with the `@` command) which defines an image, so that the entry need not describe every possible parameter - when invoked, all parameters not specifically set by the entry will have predictable values (the defaults).

INITORBIT=pixel INITORBIT=nnn/nnn Allows control over the value used to begin each Mandelbrot-type orbit. "initorbit=pixel" is the default for most types; this command initializes the orbit to the complex number corresponding to the screen pixel. The command "initorbit=nnn/nnn" uses the entered value as the initializer. See the discussion of the Mandellambda Sets for more on this topic.

RSEED=nnnn The initial random-number "seed" for plasma clouds is taken from your computer's internal clock-timer. This argument forces a value (which you can see in the `Tab` display), and allows you to reproduce plasma clouds. A detailed discussion of why a truly random number may be impossible to define, let alone generate, will have to wait for "Iterated Dynamics: The 3-MB Doc File."

SHOWDOT=[auto|bright|medium|dark|<nnn>[/<size>]] Colors the current dot being calculated color <nnn> or an automatically calculated color taken from the current palette. The second parameter is the size of the traveling pointer in units of pixels of 1/1024th of a screen.

The traveling pointer may strobe or flicker with fast fractals because of interaction with the monitor's vertical refresh. The orbitdelay parameter can be used to introduce a per-pixel delay

when showdot is turned on. Try orbitdelay=1000 with "showdot=b/20" to get a feel for how the showdot triangle works.

ASPECTDRIFT=<nn> When zooming in or out, the aspect ratio (the width to height ratio) can change slightly due to rounding and the noncontinuous nature of pixels. If the aspect changes by a factor less than <nn>, then the aspect is set to it's normal value, making the center-mag Xmagfactor parameter equal to 1. (see CENTER-MAG above.) The default is 0.01. A larger value adjusts more often. A value of 0 does no adjustment at all.

Color Parameters

INSIDE=nnn|maxiter|bof60|bof61|zmag|epscross|startrail|period|atan|fmod Set the color of the interior: for example, "inside=0" makes the M-set "lake" a stylish basic black with the default color map. The option "maxiter" makes the inside color the same as the value of maxiter.

Eight more options reveal hidden structure inside the lake. The options "bof60" and "bof61", are named after the figures on pages 60 and 61 of "The Beauty of Fractals". The option "zmag" is a method of coloring based on the magnitude of Z after the maximum iterations have been reached. The affect along the edges of the Mandelbrot is like thin- metal welded sculpture. The option "fmod" is a method of coloring based on the magnitude of the last orbit within a set distance from the origin. The option "period" colors pixels according to the period of their eventual orbit. The option "atan" colors by determining the angle in degrees the last iterated value has with respect to the real axis, and using the absolute value. See Inside=bof60|bof61|zmag|fmod|period|atan for a brilliant explanation of what these do!

The option "epscross" colors pixels green or yellow according to whether their orbits swing close to the Y-axis or X-axis, respectively. The option "startrail" has a coloring scheme based on clusters of points in the orbits. Best with outside=<nnn>. For more information, see Inside=epscross|startrail.

Note that the "Look for finite attractor" option on the `Y` options screen will override the selected inside option if an attractor is found - see Finite Attractors.

OUTSIDE=nnn|iter|real|imag|summ|mult|atan|fmod|tdis The classic method of coloring outside the fractal is to color according to how many iterations were required before Z reached the bailout value, usually 4. This is the method used with the option "iter".

However, when Z reaches bailout the real and imaginary components can be at very diferent values. The options "real" and "imag" color using the iteration value plus the real or imaginary values. The option "summ" uses the sum of all these values. These options can give a startling 3D quality to otherwise flat images and can change some boring images to wonderful ones. The option "mult" colors by multiplying the iteration by real divided by imaginary. There was no mathematical reason for this, it just seemed like a good idea. The option "atan" colors by determining the angle in degrees the last iterated value has with respect to the real axis, and using the absolute value.

The option "fmod" colors pixels according to the magnitude of the last orbit point which is within a set distance from the origin. Then: color = magnitude * colors / closeprox The magnitude used for the comparison is now based on the same calculation as is used for the bailout test. The value of closeprox can be varied interactively. This feature was contributed by Iain Stirling. There is a problem with the mandel fractal type when "outside=fmod" is used with "inside=bof6x" and

bailoutest set to "real", "imag", or "manr". This is likely due to changes made in the code so that bof images could be reproduced. Select a different fractal type that produces the default mandel image to explore using these parameters.

The option "tdis" colors the pixels according to the total distance traveled by the orbit. This feature was suggested by Steve Robinson.

The option "nnn" sets the color of the exterior to some number of your choosing: for example, "outside=1" makes all points not inside the fractal set to color 1 (blue in the default color map). Note that defining an outside color forces any image to be a two-color one: either a point is inside the set, or it's outside it.

MAP=filename Reads in a replacement color map from filename. This map replaces the default color map. The difference between this argument and an alternate map read in via `L` in color-command mode is that this one applies to the entire run. See Color Maps.

COLORS=@filename|colorspecification Sets colors for the current image, like the `L` function in color cycling and palette editing modes. Unlike the "map=" parameter, colors set with "colors=" do not replace the default - when you next select a new fractal type, colors will revert to their defaults.

The option "@filename" tells Id to use a color map file named "filename". See Color Maps.

The option "colorspecification" specifies the colors directly. The value of "colorspecification" is rather long (768 characters for 256 color modes), and its syntax is not documented here. This form of the "colors=" parameter is not intended for manual use - it exists for use by the `B` command when saving the description of a nice image.

RECORDCOLORS=auto|comment|yes Controls the method of writing colors in par files. The option "auto" causes the colors to be written in the "colors=@mapfile" form if the colors were loaded from a map. Use this mode if you manage your colors using map files. If you share par files with others, and have trouble remembering to send them the map file, use the option "comment" or "yes". These modes force the writing of compressed color maps in the par file in all cases. The only difference is that the comment option also writes the map filename in a comment so you can remember where the colors came from.

CYCLERANGE=nnn/nnn Sets the range of color numbers to be animated during color cycling. The default is 1/255, i.e. just color number 0 (usually black) is not cycled.

CYCLELIMIT=nnn Sets the speed of color cycling. Technically, the number of color values updated during a single update. Legal values are 1 - 256; the default is 55.

TEXTCOLORS=mono Set text screen colors to simple black and white.

TEXTCOLORS=aa/bb/cc/... Set text screen colors. Omit any value to use the default (e.g. textcolors=////50 to set just the 5th value). Each value is a 2 digit hexadecimal value; 1st digit is background color (from 0 to 7), 2nd digit is foreground color (from 0 to F). Color values are: 0 black 8 gray 1 blue 9 light blue 2 green A light green 3 cyan B light cyan 4 red C light red 5 magenta D light magenta 6 brown E yellow 7 white F bright white

31 colors can be specified, their meanings are as follows: Heading: 1 Iterated Dynamics version info

2 heading line development info (not used in released version) Help: 3 sub-heading 4 main text 5 instructions at bottom of screen 6 hotlink field 7 highlighted (current) hotlink Menu, Selection Boxes, Parameter Input Boxes: 8 background around box and instructions at bottom 9 emphasized text outside box 10 low intensity information in box

```
   11  medium intensity information in box
   12  high intensity information in box (e.g. heading)
   13  current keyin field
   14  current keyin field when it is limited to one of n values
   15  current choice in multiple choice list
   16  speed key prompt in multiple choice list
   17  speed key keyin in multiple choice list
  General (Tab Key Display, IFS parameters, "Thinking" Display):
   18  high intensity information
   19  medium intensity information
   20  low intensity information
   21  current keyin field
  Disk Video:
   22  background around box
   23  high intensity information
   24  low intensity information
  Diagnostic Messages:
   25  error
   26  information
  Credits Screen:
   27  bottom lines
   28  high intensity divider line
   29  low intensity divider line
   30  primary authors
   31  contributing authors
The default is
   textcolors=1F/1A/2E/70/28/71/31/78/70/17/1F/1E/2F/3F/5F/07/
              0D/71/70/78/0F/70/0E/0F/4F/20/17/20/28/0F/07
(In a real command file, all values must be on one line.)
```

OLDDEMMCOLORS=yes|no Sets the old coloring scheme used with the distance estimator method.

TRUECOLOR=yes You can save either the default color scheme or the iteration escape value to a file called fract001.tga. The number is incremented each time a file is saved unless the parameter overwrite is set to yes. This will allow experimentation with truecolor algorithms. Someday we'll have real truecolor support...

TRUEMODE=def|iter Determines whether the fract001.tga file produced when "truecolor=yes" contains the iteration value or the default coloring scheme.

NOBOF=yes|no Setting this parameter to yes causes the bof60 and bof61 inside options to function the same as the other inside options by making the per pixel initialization the same. The per pixel initialization is normally different for the bof60 and bof61 options to reproduce the images in the book, "The Beauty of Fractals". The default is no.

Doodad Parameters

LOGMAP=yes|old|n Selects a compressed relationship between escape-time iterations and palette colors. See Logarithmic Palettes and Color Ranges for details.

LOGMODE=fly/table|auto Forces the use of the on-the-fly routine or the logarithm table for the calculation of log palettes. Not normally needed. The auto option cannot be used at the same time as the other two. Auto causes the logmap value to be automatically recalculated when zooming. Changing almost anything will turn this feature off. Set logmode=auto from the `G` screen prompt.

RANGES=nn/nn/nn/... Specifies ranges of escape-time iteration counts to be mapped to each color number. See Logarithmic Palettes and Color Ranges for details.

DISTEST=nnn/nnn A nonzero value in the first parameter enables the distance estimator method. The second parameter specifies the "width factor" and defaults to 71. See Distance Estimator Method for details.

DECOMP=2|4|8|16|32|64|128|256 Invokes the corresponding decomposition coloring scheme. See Decomposition for details.

BIOMORPH=nnn Turn on biomorph option; set affected pixels to color nnn. See Biomorphs for details.

POTENTIAL=maxcolor[/slope[/modulus[/16bit]]] Enables the continuous potential coloring mode for all fractal types except plasma clouds, attractor types such as lorenz, and IFS. The four arguments define the maximum color value, the slope of the potential curve, the modulus bailout value, and whether 16 bit values are to be calculated. Example: "potential=240/2000/40/16bit". The Mandelbrot and Julia types ignore the modulus bailout value and use their own hardwired value of 4.0 instead. See Continuous Potential for details.

INVERT=nn/nn/nn Turns on inversion. The parameters are radius of inversion, x- coordinate of center, and y-coordinate of center. -1 as the first parameter sets the radius to 1/6 the smaller screen dimension; no x/y parameters defaults to center of screen. The values are displayed with the `Tab` command. See Inversion for details.

FINATTRACT=no|yes Another option to show coloring inside some Julia "lakes" to show escape time to finite attractors. Works with lambda, magnet types, and possibly others. See Finite Attractors for more information.

EXITNOASK=yes This option forces Id to bypass the final "are you sure?" exit screen when the `Esc` key is pressed from the main image-generation screen. Added at the request of Ward Christensen. It's his funeral <grin>.

File Parameters

In Id you can use various filename variables to specify files and set default directories. You can specify default directories for read libraries and the save library using these parameters in your sstools.ini file.

LIBRARYDIRS=<path>[,...,<path>] Specifies a list of locations to use as read libraries. Separate multiple locations with a comma. See Libraries.

SAVEDIR=<path> Sets the save library location used for writing output files. All output files are created under this location. The default save library is "Iterated Dynamics" in the user's Documents folder on Windows. On Linux, the Documents folder is read from XDG_DOCUMENTS_DIR or user-dirs.dirs, falling back to $HOME/Documents. On macOS, the user's $HOME/Documents folder is used. See Libraries.

SAVENAME=[<path>\][name] Set the path and/or filename to use when you `S`ave a screen. The default filename is fract001. The .gif extension is optional (Example: "savename=myfile"). If an absolute path is given, that value is used, otherwise the filename is used with the save library. See Libraries.

OVERWRITE=no|yes Sets the savename overwrite flag (default is no). If yes, saved files will overwrite existing files from previous sessions; otherwise the automatic incrementing of fract001.gif will find the first unused filename.

SAVETIME=nnn Tells Id to automatically do a save every nnn minutes while a calculation is in progress. This is mainly useful with long batches - see Batch Mode.

DITHER=yes Dither a color file into two colors for monochrome display. This gives a poor-quality display of gray levels.

PARMFILE=[<path>\][parmfilename] Specifies the default parameter path and/or file to be used by the `@` (or `2`) and `B` commands. If not specified, the default is id.par. If an absolute path is given, that file will be used if it exists. Otherwise, the read libraries are used to locate the filename. See Libraries.

FORMULAFILE=[<path>\][formulafilename] Specifies the formula path and/or file for type=formula fractals (default is id.frm). Handy if you want to generate one of these fractal types in batch mode. If an absolute path is given, that file will be used if it exists. Otherwise, the read libraries are used to locate the filename. See Libraries.

LFILE=[<path>\][lsystemfile] Specifies the default L-system path and/or file for type=lsystem fractals (default is id.l). If an absolute path is given, that file will be used if it exists. Otherwise, the read libraries are used to locate the filename. See Libraries.

IFSFILE=[<path>\][ifsfilename] Specifies the default path and/or file for type=ifs fractals (default is id.ifs). If an absolute path is given, that file will be used if it exists. Otherwise, the read libraries are used to locate the filename. See Libraries.

FILENAME=[.suffix] Sets the default file extension used for the `R` command. When this parameter is omitted, the default file mask shows .gif and .pot files. You might want to specify this parameter and the SAVENAME= parameter in your sstools.ini file if you keep your fractal images separate from other .gif files by using a different suffix for them.

ORBITSAVE=yes Causes the orbits file to be opened and the points generated by orbit fractals or IFS fractals to be saved in a raw format. The file will be overwritten if "overwrite=yes". Otherwise, each time a new fractal is generated a new file is written with this option. (see Barnsley IFS Fractals

Orbit Fractals).

ORBITSAVENAME=[filename] Sets the name of the orbit file to be written when "orbitsave=yes" is used. Defaults to orbits.raw. The file is written to the save library location; see Libraries.

Video Parameters

VIDEO=xxx Set the initial video mode (and bypass the informational screens). Handy for batch runs. (Example: "video=f4") You can obtain the current video values (key assignments) from the "Select Video Mode" screen inside Id. If you want to do a batch run with a video mode which isn't currently assigned to a key, you'll have to modify the key assignments - see Video Mode Function Keys.

ASKVIDEO=yes|no If "no", this eliminates the prompt asking you if a file to be restored is OK for your current video hardware.

VIEWWINDOWS=xx[/xx[/yes|no[/nn[/nn]]]] Set the reduction factor, final media aspect ratio, crop starting coordinates (y/n), explicit x size, and explicit y size, see View Window .

FASTRESTORE=yes|no If "yes", resets viewwindow to "no" prior to restoring a GIF file. Otherwise, images saved in full view will be drawn in reduced view if viewwindows has been set to "yes" previously. Also, when "yes", bypasses the warning when restoring a GIF in a video mode other than the one in which the GIF was saved. Default is "no". This feature is useful when cycling through a group of GIFs in autokey mode. When combined with "askvideo=no", allows loading images with the last successfully used video mode. This is handy when viewing 1600x1200 images when you only have 1024x768 available.

VIRTUAL=yes|no With a suitable video mode it is possible to set virtual screen modes using the View Window options. With certain video modes it may be necessary to disable the check for virtual screen modes if this check prevents Id from loading correctly. The default is "yes". Setting this to "no" disables the check for and the ability to use virtual screen sizes.

Sound Parameters

SOUND=off|beep|x|y|z/pc|fm/quant We're all much too busy to waste time with Id at work, and no doubt you are too, so "sound=off" is included only for use at home, to avoid waking the kids or your significant other, late at night. (By the way, didn't you tell yourself "just one more zoom on LambdaSine" an hour ago?) Suggestions for a "boss" hot-key will be cheerfully ignored, as this sucker is getting big enough without including a spreadsheet screen too. The "sound=x|y|x" options are for the attractor fractals, like the Lorenz fractals - they play with the sound as they are generating an image, based on the X or Y or Z co-ordinate they are displaying at the moment.

You can now hear the sound of fractal orbits—just turn on sound from the command line, the `X` menu, or the `Ctrl` + `F` menu, fire up a fractal, and try the `O`rbits command (or set "showorbit=yes"). Use the "orbitdelay=<nnn>" parameter (also on the `Ctrl` + `F` menu) to dramatically alter the effect, which ranges from an unearthly scream to a series of discrete tones. Not recommended when people you have to live with are nearby! Remember, we don't promise that it will sound beautiful! There is also an option to quantize the notes generated by Id. See Sound Controls.

You can also "hear" any image that Id can decode; turn on sound before using `R` to read in a GIF file. We have no idea if this feature is useful. It was inspired by the comments of an on-line friend who is blind. We solicit feedback and suggestions from anyone who finds these sound features interesting or useful. The orbitdelay parameter also affects the sound of decoding images.

HERTZ=<nnn> Adjusts the sound produced by the "sound=x|y|z" option. The actual sounds produced are limited to the range of 20-5000 Hz. Setting a negative Hertz value allows shifting the range of sounds produced down into the bass range. This also eliminates some of the notes since anything under 20 Hz or over 5000 Hz will not be played.

ORBITDELAY=<nn> Slows up the display of orbits using the `0` command for folks with hot new computers. Units are in 1/10000 seconds per orbit point. ORBITDELAY=10 therefore allows you to see each pixel's orbit point for about one millisecond. For best display of orbits, try "passes=1" and a moderate resolution such as 900x600. Note that the first time you press the `0` key with the orbitdelay function active, your computer will pause for a half-second or so to calibrate a high-resolution timer.

SHOWORBIT=yes|no Causes the during-generation orbits feature, toggled by the `0` command, to start off in the "on" position each time a new fractal calculation starts.

ORBITSAVE=sound This option causes the hertz value played with sound=x|y|z option to be written to a file "sound.txt" in the current directory. Bill Jemison has made some intriguing music with this option.

Sound Controls

On this screen, accessed by hitting `Ctrl` + `F`, you'll find a few parameters that you recognize from the `X` menu handily gathered together, plus a few new ones. You can find the descriptions for the basic parms over at the Sound Parameters help.

Id produces sounds using FM synthesis as made popular by the OPL-3 chipset on early PC sound cards. The OPL-3 chipset has within it several voices, each capable of generating sound independently of the others, and with a variety of controllable parameters. See Advanced Sound Controls for details.

The sound card volume control given here will be overridden by any volume control applications that come with your OS, such as the Windows mixer control panel.

The other parameter that needs explaining here is note pitch quantisation. When this is enabled, tones that are to be played are first rounded to the nearest note on the western even tempered scale (i.e. the notes that you find on a piano) as opposed to the full spectrum of frequencies. The Advanced Sound Controls screen also has a facility to restrict the notes played further so that you can produce tunes in a particular key, if you're so inclined.

NOTE	Iterated Dynamics 1.0 goes through all the motions to produce sound, but doesn't actually generate any audio. A suitable audio library for FM synthesis will be used in the future to restore sound generation.

Advanced Sound Controls

The sound output from Id is a tad versatile. This comes into its own when playing sounds based on fractal orbits, producing results from haunting chordal pulses through to unearthly arpeggios.

The voices used to generate notes on the sound card have variable Envelopes and can be played polyphonically so we have three parameters to vary the sounds produced:

POLYPHONY=nn This controls how many different note are allowed to sound at once, voices are assigned cyclically as notes are played: i.e. after the first orbit is calculated a frequency is assigned to the first voice and that note is triggered on. Id then waits for one orbitdelay period, calculates the next orbit, assigns a frequency to the second voice, and triggers it on. This continues until the number of voices sounding is equal to the polyphony value whereupon the first selected voice is triggered off and the note allowed to decay. That probably all sounds horrendously complex but just think of it as playing the organ, holding down each new note for as long as possible but with the polyphony value controlling how many fingers you have.

WAVETYPE=nn WaveForm 0: Sine WaveForm 1: Half-Sine | /^\ | /^\ /^\ |/ \ / |/ \ / \ -/-----\-----/-- -|-----=-----=-----=- | \/ | | _/ |

```
WaveForm 2: Abs-Sine   WaveForm 3: Pulse-Sine
 | /^\   /^\             | /^|         /^|
 |/   \ /   \ /          |/  |        /  |
-|-----=------=--        -|---=-------=---=---
 |                       |
```

```
WaveForm 4: Sine - even periods only
 | /^\                     /^\
 |/   \                   /   \
-|-----\-----=-----------=-----\-----=-
 |      \   /                   \   /
 |       \_/                     \_/
```

```
WaveForm 5: Abs-Sine - even periods only
 | /^\   /^\               /^\   /^\
 |/   \ /   \             /   \ /   \
-|-----=-----=-----------=-----=------=-
 |
```

```
WaveForm 6: Square
 |-----+     +-----+     +-----+     +-
 |     |     |     |     |     |     |
-|-----|-----|-----|-----|-----|-----|--
 |     |     |     |     |     |     |
 |     +-----+     +-----+     +-----+
```

```
WaveForm 7: Derived Square
 |          |\                |\
 |         |  \              |  \
-|--__----|----\------__----|----\------
 |    \  |             \  |
 |     \|               \|
```

ATTACK=nn This controls the time taken for a note to hit full volume after being triggered on. Low values give a punchy percussive sound while long values give a softer bowed sound. If attack time is set too long and orbitdelay is low then notes are played too fast to ever achieve any volume and the net result is low or non existent output. To put it another way, if orbitdelay is low then this value should be low also.

DECAY=nn This controls the time taken for a note to die down to the sustain level. Setting this time too low may produce a rhythmic click between notes. The attack and decay values don't allow much fine control.

SUSTAIN=nn This controls the volume level at which a note is held.

SRELEASE=nn This controls the delay in time for releasing a note. A release value of 15 seems to equate to 'never let the note release' on the machine used to develop the sound card drivers. But, as ever, with other manufacturer's emulations your mileage may vary.

VOLUME=nn It is possible to adjust the volume of an FM synth card. This has no effect on the PC speaker.

ATTENUATE=none|low|mid|high On an FM synth card it is possible to attenuate the high pitched notes. The default is none. The low value attenuates at 1.5 dB/octave, the mid at 3.0 dB/octave, and the high at 6.0 dB/octave.

SCALEMAP=nn/nn/nn/... The scale map list is a way of controlling which notes are played by Id and only has any effect if note quantisation is switched on in the basic sound parameters screen. It can be very useful when the polyphony is up and the chords you're getting are more like dischords. :-)

The way it works is this: When a sound is played with frequency quantisation on its frequency is rounded to the nearest 'proper' note on the musical scale, this note number is then looked up in the scale map table and substituted by the note indicated in that entry of the table. The end result of this is that, for instance, using this sequence of numbers as a scale map: 2,2,4,4,4,7,7,9,9,11,11,11 Will ensure that Id uses only the black notes on the keyboard, just the sharps and flats for a vaguely oriental feel (to my western ears anyway :-)).

The above probably appears to be sheer gobbledegook at first reading, sorry, but if you keep at it you'll unlock this powerful feature for keeping your fractals melodic. (Unless you're totally turned on by microtonal frequencies in which case forget all this quantisation nonsense and tune in to the true sound of chaos!)

The passes=1 setting (Drawing Method) almost always works best with sound.

Envelopes

Here's some more information about the concept of a note envelope for those unfamiliar with sound synthesis terminology, it's all about how the dynamics of a note are defined (i.e the way in which the loudness changes during the life of a note)

```
Graph of a note's volume during it's life cycle:
```

```
    /\______                    /\_______
__/         \_____ ... _______/         \_________
0011111111110000000 ... 00000001111111111110000000000
  A  D  S   R                 AAADSSSSSSSR
```

The string of 0's and 1's represents the keyon/off state of the voice, 1's indicate the key being held down. The attack, decay, sustain, and release portions of the envelope are represented by ADSR, this is what happens:

When the note is first triggered on the volume rises to peak volume at a rate determined by the attack value.

Once at the full level the decay period starts and the volume dies down (at a rate set by the decay value) to a level that is set by the sustain value.

The note continues to sound at this volume until triggered off (the 'key' is released) whereupon it dies down to zero volume at the release rate.

And so on. Now, with the current voice assignment method, while orbits are being generated continuously, timing looks like this (with four note polyphony in this example)

```
   Voice:
     1 *111.....*111....
     2 .*111.....*111...
     3 ..*111.....*111..
     4 ...*111.....*111.
     5 ....*111.....*111
     6 .....*111.....*11
     7 ......*111.....*1
     8 .......*111.....*
     9 ........*111.....
orbit: 12345678901234567
```

Where: . = key off (silent or releasing note) * = note assigned a value and triggered on, attack, decay, sustain phase begins 1 = note held on, you'll notice that above there are only ever four notes held on at any one time, though more than four may be sounding if there is a long release value.

3D Parameters

To stay out of trouble, specify all the 3D parameters, even if you want to use what you think are the default values. It takes a little practice to learn what the default values really are. The best way to create a set of parameters is to use the `B` command on an image you like and then use an editor to modify the resulting parameter file.

3D=Yes 3D=Overlay Resets all 3D parameters to default values. If "filename=" is given, forces a restore to be performed in 3D mode (handy when used with "batch=yes" for batch-mode 3D images). If specified, "3d=yes" should come before any other 3D parameters on the command line or in a parameter file entry. The form "3d=overlay" is identical except that the previous graphics screen is not cleared, as with the `#` (`Shift` + `3`) overlay command. Useful for building parameter files that use the 3D overlay feature.

The parameters below override the 3D defaults: PREVIEW=yes Turns on 3D preview default mode SHOWBOX=yes Turns on 3D showbox default mode COARSE=nn Sets Preview coarseness default value SPHERE=yes Turns on spherical projection mode STEREO=n Selects the type of stereo image creation

RAY=nnn|name Selects raytrace output file format BRIEF=yes Selects brief or verbose file for DKB output USEGRAYSCALE=yes Use grayscale as depth instead of color number

INTEROCULAR=nn Sets the interocular distance for stereo CONVERGE=nn Determines the overall image separation CROP=nn/nn/nn/nn Trims the edges off stereo pairs BRIGHT=nn/nn Compensates funny glasses filter parameters LONGITUDE=nn/nn Longitude minimum and maximum LATITUDE=nn/nn Latitude minimum and maximum RADIUS=nn Radius scale factor ROTATION=nn[/nn[/nn]] Rotation about x, y and z axes SCALEXYZ=nn/nn/nn X, y and z scale factors ROUGHNESS=nn Same as z scale factor WATERLINE=nn Colors nn and below will be "inside" color FILLTYPE=nn 3D filltype PERSPECTIVE=nn Perspective distance XYSHIFT=nn/nn Shift image in x and y directions with perspective LIGHTSOURCE=nn/nn/nn Coordinates for light-source vector SMOOTHING=nn Smooths images in light-source fill modes TRANSPARENT=min/max Defines a range of colors to be treated as transparent when <#> overlaying 3D images. XYADJUST=nn/nn This shifts the image in the x/y dir without perspective

The ray parameter can either be specified as a numeric value (nnn) or as text. See the table in 3D Mode Selection.

Below are commands that support Marc Reinig's terrain features.

RANDOMIZE=nnn (0 - 100) This feature randomly varies the color of a pixel to near by colors. Useful to minimize map banding in 3D transformations. Usable with all filltypes. "0" disables, the maximum value is 7. Try 3 - 5.

AMBIENT=nnn (0 - 100) Set the depth of the shadows when using full color and light source filltypes. "0" disables the function, higher values lower the contrast.

FULLCOLOR=yes Valid with any light source filltype. Allows you to create a Targa-24 file which uses the color of the image being transformed or the map you select and shades it as you would see it in real life. Well, its better than B&W. A good map file to use is topo.map.

HAZE=nnn (0 - 100) Gives more realistic terrains by setting the amount of haze for distant objects when using full color in light source FILLTYPES. Works only in the "y" direction currently, so don't use it with much y rotation. Try "rotation=85/0/0". 0 disables.

BACKGROUND=nn/nn/nn Sets the background color of the Targa-24 file by setting the red, green, and blue values (rr/gg/bb).

LIGHTNAME=<filename> The name of the Targa-24 file to be created when using full color with light source. Default is light001.tga. If "overwrite=no" (the default), the file name will be incremented until an unused filename is found. Background in this file will be the color set by background=r/g/b, the default is sky blue.

STEREOWIDTH=<nnn> This parameter allows you to specify the width in inches of the image for the purpose of getting the correct stereo effect when viewing RDS images. See Random Dot Stereograms (RDS).

Batch Mode

It is possible, believe it or not, to become so jaded with the screen drawing process, so familiar with the types and options, that you just want to hit a key and do something else until the final images are safe on disk. To do this, start Id with the "batch=yes" parameter. To set up a batch run with the parameters required for a particular image you might: * Find an interesting area. Note the parameters from the `Tab` display. Then use an editor to write a batch file. * Find an interesting area. Set all the options you'll want in the batch run. Use the `B` command to store the parameters in a file. Then use an editor to add the additional required batch mode parameters (such as "video=") to the generated parameter file entry. Then run the batch using "id @myname.par/myentry" (if you told the `B` command to use file "myname" and to name the entry "myentry").

Another approach to batch mode calculations, using "filename=" and resume, is described later.

When modifying a parameter file entry generated by the `B` command, the only parameters you must add for a batch mode run are "batch=yes", and "video=xxx" to select a video mode. You might want to also add "savename=[name]" to name the result as something other than the default fract001.gif. Or, you might find it easier to leave the generated parameter file unchanged and add these parameters by using a command like:

```
id @myname.par/myentry batch=y video=AF3 savename=mygif
```

The parameter "batch=yes" tells Id to run in batch mode — that is, Id draws the image using whatever other parameters you specified, then acts as if you had hit `S` to save the image, then exits.

The parameter "filename=" can be used with "batch=yes" to resume calculation of an incomplete image. For instance, you might interactively find an image you like; then select some slow options (a high resolution disk video mode, distance estimator method, high maxiter, or whatever); start the calculation; then interrupt immediately with a `S`ave. Rename the save file (fract001.gif if it is the first in the session and you didn't name it with the `X` options or "savename=") to xxx.gif. Later you can run Id in batch mode to finish the job:

```
id batch=yes filename=xxx savename=xxx
```

The parameter "savetime=nnn" is useful with long batch calculations, to store a checkpoint every nnn minutes. If you start a many hour calculation with say "savetime=60", and a power failure occurs during the calculation, you'll have lost at most an hour of work on the image. You can resume calculation from the save file as above. Automatic saves triggered by savetime do not increment the save file name. The same file is overwritten by each auto save until the image completes. But note that Id does not directly over-write save files. Instead, each save operation writes a temporary file id.tmp, then deletes the prior save file, then renames id.tmp to be the new save file. This protects against power failures which occur during a save operation - if such a power failure occurs, the prior save file is intact and there's a harmless incomplete id.tmp on your disk.

If you want to spread a many-hour image over multiple bits of free machine time you could use a command like:

```
id batch=yes filename=xxx savename=xxx savetime=60 video=F3
```

While this batch is running, hit `S` (almost any key actually) to tell Id to save what it has done so far and give your machine back. A status code of 2 is returned by Id to the batch file. Kick off the batch again when you have another time slice for it.

When the savetime parameter is negative, Id will save the image after the requested time and exit. This is useful in batch files where you want to generate several images with a time limit on each image.

While running a batch file, pressing any key will cause Id to exit with a status code of 2. Any error that interrupts an image save to disk will cause an exit with status code 2. Any error that prevents an image from being generated will cause an exit with status code 1.

The savetime= parameter, and batch resumes of partial calculations, only work with fractal types which can be resumed. See Interrupting and Resuming for information about non-resumable types.

Browser Parameters

This Screen enables you to control Id's built in file browsing utility. If you don't know what that is see Browse Commands. This screen is selected with `Ctrl` + `B` from just about anywhere.

"Autobrowsing" Select yes if you want the loaded image to be scanned for sub images immediately without pressing 'L' every time.

"Ask about GIF video mode" Allows turning on and off the display of the video mode table when loading GIFs. This has the same effect as the askvideo= command.

"Type/Parm check" Select whether the browser tests for fractal type or parms when deciding whether a file is a sub image of the current screen or not. DISABLE WITH CAUTION! or things could get confusing. These tests can be switched off to allow such situations as wishing to display old images that were generated using a formula type which is now implemented as a built in fractal

type. "Confirm deletes" Set this to No if you get fed up with the double prompting that the browser gives when deleting a file. It won't get rid of the first prompt however.

"Smallest window" This parameter determines how small the image would have to be onscreen before the browser decides not to include it in the selection of files. The size is entered in decimal pixels so, for instance, this could be set to 0.2 to allow images that are up to around three maximum zooms away (depending on the current video resolution) to be loaded instantly. Set this to 0 to enable all sub images to be detected. This can lead to a very cluttered screen! The primary use is in conjunction with the search file mask (see below) to allow location of high magnification images within an overall view (like the whole Mandelbrot set).

"Smallest box" This determines when the image location is shown as crosshairs rather than a rather small box. Set this according to how good your eyesight is (probably worse than before you started staring at fractals all the time :-)) or the resolution of your screen. WARNING the crosshairs routine centers the cursor on one corner of the image box at the moment so this looks misleading if set too large. "Search Mask" Sets the file name pattern which the browser searches, this can be used to search out the location of a file by setting this to the filename and setting smallest image to 0 (see above).

Passes Parameters

PERIODICITY=no|show|nnn Controls periodicity checking (see Periodicity Logic). "no" turns it off, "show" lets you see which pixels were painted as "inside" due to being caught by periodicity. Specifying a number causes a more conservative periodicity test (each increase of 1 divides test tolerance by 2). Entering a negative number lets you turn on "show" with that number. Note: "type=lambdafn" with "function=exp" needs periodicity turned off to be accurate — there may be other cases.

A non-zero value of the "periodicity=" option causes "passes=o" to not plot orbits that have reached the bailout conditions or where an orbit goes off the visible area of the image. A zero value of periodicity will plot all orbits except as modified by orbitdelay and orbitinterval.

ORBITDELAY=<nn> This option controls how many orbits are computed before the orbits are displayed on the screen when using the "passes=o" option, or the fractal types mandelcloud and dynamic. This allows the orbits to settle down before plotting starts.

This option also slows down the display of orbits using the `0` command for folks with hot new computers. Units are in 1/10000 seconds per orbit point. "orbitdelay=10" therefore allows you to see each pixel's orbit point for about one millisecond. For best display of orbits, try "passes=1" and a moderate resolution such as 800x600. Note that the first time you press the `0` key with the orbitdelay function active, your computer will pause for a half-second or so to calibrate a high-resolution timer.

ORBITINTERVAL=<nn> This parameter causes "passes=o" to plot every nth orbit point ranging from 1, which plots every orbit, to 255, which plots every 255th orbit. This value must be lower than the value of maxit to make any sense.

SCREENCOORDS=yes|no This parameter maintains the screen coordinates and lets you zoom into an image changing the coordinates of the line or rectangle used to generate the image, but keeps

the display coordinates the same. The screen coordinates can be zoomed, rotated, and skewed using the F2 Screen Coordinates Screen. If set to no, the screen and image coordinates are maintained the same when an image is zoomed.

ORBITDRAWMODE=rect|line The rect(angle) method plots the orbits in a rectangle that can be zoomed, rotated, and skewed using the F6 Image Coordinates Screen, and the straight line method plots the orbits between two points specified on the F6 image coordinates screen.

Screen Coordinates Screen

You can directly enter corner coordinates on this screen for use with the passes='o' option. You can also use F4 to reset the coordinates to the defaults for the current fractal type.

There are two formats for the display: corners or center-mag. You can toggle between the two by using F7.

In corners mode, corner coordinate values are entered directly. Usually only the top-left and bottom-right corners need be specified - the bottom left corner can be entered as zeros to default to an ordinary unrotated rectangular area. For rotated or skewed images, the bottom left corner must also be specified.

In center-mag mode the screen area is described by entering the coordinates for the center of the rectangle, and its magnification factor. Usually only these three values are needed, but the user can also specify the amount that the image is stretched, rotated and skewed.

Image Coordinates Screen

You can directly enter corner coordinates on this screen instead of using the zoom box to move around. You can also use F4 to reset the coordinates to the defaults for the current fractal type.

There are two formats for the display: corners or center-mag. You can toggle between the two by using F7.

In corners mode, corner coordinate values are entered directly. Usually only the top-left and bottom-right corners need be specified - the bottom left corner can be entered as zeros to default to an ordinary unrotated rectangular area. For rotated or skewed images, the bottom left corner must also be specified.

In center-mag mode the image area is described by entering the coordinates for the center of the rectangle, and its magnification factor. Usually only these three values are needed, but the user can also specify the amount that the image is stretched, rotated and skewed.

Hardware Support

Notes on Video Modes, "Standard" and Otherwise

"Disk-Video" Modes

These "video modes" do not involve a graphic display at all. They use memory or your disk drive (as file id.$$$) to store the fractal image. These modes are useful for creating images beyond the size of your screen, right up to the GIF image limit of 65535 x 65535 x 256, memory permitting. They're also useful for background processing - create an image in a disk-video mode, save it, then restore it in a real video mode.

While you are using a disk-video mode, your screen will display text information indicating whether memory or your disk drive is being used, and what portion of the "screen" is being read from or written to. A "Cache size" figure is also displayed. 64K is the maximum cache size. With a very low cache size such as 4 or 6k, performance gets considerably worse in cases using solid guessing, boundary tracing, plasma, or anything else which paints the screen non-linearly.

The zoom box is disabled during disk-video modes (you couldn't see where it is anyway). So is the orbit display feature. Color Cycling can be used during disk-video modes, but only to load or save a color palette.

When using real disk for your disk-video, for some "attractor" types (e.g. Lorenz) or IFS images, Id gives you a warning message but lets you proceed. You can end the calculation with `Esc` if you think your hard disk is getting too strenuous a workout.

Customized Video Modes, id.cfg

While Id no longer drives the graphics card directly, the concept of a "video mode" still survives. It determines the dimesions of the display window and the dimensoins of the saved image. You can customize the table of modes by modifying the id.cfg file with your text editor. Saving a backup copy of id.cfg first is of course highly recommended!

The table is loaded from the file id.cfg each time Id is run. It can contain information for up to 300 video modes. The table entries, and the function keys they are tied to, are displayed in the "select video mode" screen.

Since the saved image size is determined by the video mode, customizing this table is the only way to introduce new image sizes at this time. The default video modes listed in this file as distributed with Iterated Dynamics are arbitrary, but chosen to reflect common file dimensions.

Lines in id.cfg which begin with a semi-colon are treated as comments. The rest of the lines must have seven fields separated by commas. The fields are defined as:

1. Key assignment. F2 to F10, SF1 to SF10, CF1 to CF10, or AF1 to AF10. Blank if no key is assigned to the mode.
2. The name of the video mode (25 chars max, no leading blanks). This field is only used for display in the vide mode selection screen.
3. The number of pixels across the screen (X, from 2 to 65535)
4. The number of pixels down the screen (Y, from 2 to 65535)
5. The number of available colors (2, 4, 16, or 256)

6. The name of the driver for the video mode. The Win32 GDI, Linux X11 and macOS X11 drivers use "gdi". Id accepts 256-color "gdi" modes that fit on the current screen. Disk modes use "disk".
7. A comment describing the mode (25 chars max, leading blanks are OK)

Disk-video modes may use X and Y values up to 65535, the GIF image limit. Windowed display modes are limited by the current display size.

Common Problems

Here are some common problems and their possible solutions:

- If Id hangs on startup, it may have difficulty loading the video mode table in id.cfg. Without a video mode, Id doesn't have any sort of driver to interact with the screen and therefore can't display any sort of error message. Try starting Id from the directory containing this file.
- When interacting with filenames and directories, Id may truncate names to 8.3 form. Until all known filename and path length restrictions are lifted, use 8.3 filenames and directory names to be safe.

Fractals and the PC

A Little History

Before Mandelbrot

Like new forms of life, new branches of mathematics and science don't appear from nowhere. The ideas of fractal geometry can be traced to the late nineteenth century, when mathematicians created shapes — sets of points — that seemed to have no counterpart in nature. By a wonderful irony, the "abstract" mathematics descended from that work has now turned out to be more appropriate than any other for describing many natural shapes and processes.

Perhaps we shouldn't be surprised. The Greek geometers worked out the mathematics of the conic sections for its formal beauty; it was two thousand years before Copernicus and Brahe, Kepler and Newton overcame the preconception that all heavenly motions must be circular, and found the ellipse, parabola, and hyperbola in the paths of planets, comets, and projectiles.

In the 17th century Isaac Newton and Gottfried Wilhelm Leibniz created calculus, with its techniques for "differentiating" or finding the derivative of functions — in geometric terms, finding the tangent of a curve at any given point. True, some functions were discontinuous, with no tangent at a gap or an isolated point. Some had singularities: abrupt changes in direction at which the idea of a tangent becomes meaningless. But these were seen as exceptional, and attention was focused on the "well-behaved" functions that worked well in modeling nature.

Beginning in the early 1870s, though, a 50-year crisis transformed mathematical thinking. Karl Weierstrass described a function that was continuous but nondifferentiable — no tangent could be described at any point. Georg Cantor showed how a simple, repeated procedure could turn a line into a dust of scattered points, and Peano generated a convoluted curve that eventually touches

every point on a plane. These shapes seemed to fall between the usual categories of one-dimensional lines, two-dimensional planes and three-dimensional volumes. Most still saw them as pathological cases, but here and there they began to find applications.

In other areas of mathematics, too, strange shapes began to crop up. Henri Poincare attempted to analyze the stability of the solar system in the 1880s and found that the many-body dynamical problem resisted traditional methods. Instead, he developed a qualitative approach, a state space in which each point represented a different planetary orbit, and studied what we would now call the topology — the connectedness — of whole families of orbits. This approach revealed that while many initial motions quickly settled into the familiar curves, there were also strange, chaotic orbits that never became periodic and predictable.

Other investigators trying to understand fluctuating, noisy phenomena — the flooding of the Nile, price series in economics, the jiggling of molecules in Brownian motion in fluids — found that traditional models could not match the data. They had to introduce apparently arbitrary scaling features, with spikes in the data becoming rarer as they grew larger, but never disappearing entirely.

For many years these developments seemed unrelated, but there were tantalizing hints of a common thread. Like the pure mathematicians' curves and the chaotic orbital motions, the graphs of irregular time series often had the property of self-similarity: a magnified small section looked very similar to a large one over a wide range of scales.

Who Is This Guy, Anyway?

While many pure and applied mathematicians advanced these trends, it is Benoit Mandelbrot above all who saw what they had in common and pulled the threads together into the new discipline.

He was born in Warsaw in 1924, and moved to France in 1935. In a time when French mathematical training was strongly analytic, he visualized problems whenever possible, so that he could attack them in geometric terms. He attended the Ecole Polytechnique, then Caltech, where he encountered the tangled motions of fluid turbulence.

In 1958 he joined IBM, where he began a mathematical analysis of electronic noise — and began to perceive a structure in it, a hierarchy of fluctuations of all sizes, that could not be explained by existing statistical methods. Through the years that followed, one seemingly unrelated problem after another was drawn into the growing body of ideas he would come to call fractal geometry.

As computers gained more graphic capabilities, the skills of his mind’s eye were reinforced by visualization on display screens and plotters. Again and again, fractal models produced results — series of flood heights, or cotton prices — that experts said looked like "the real thing."

Visualization was extended to the physical world as well. In a provocative essay titled "How Long Is the Coast of Britain?", Mandelbrot noted that the answer depends on the scale at which one measures: it grows longer and longer as one takes into account every bay and inlet, every stone, every grain of sand. And he codified the self-similarity characteristic of many fractal shapes — the reappearance of geometrically similar features at all scales.

First in isolated papers and lectures, then in two editions of his seminal book, "The Fractal

Geometry of Nature", he argued that many of science's traditional mathematical models are ill-suited to natural forms and processes: in fact, that many of the pathological shapes mathematicians had discovered generations before are useful approximations of tree bark and lung tissue, clouds and galaxies.

Mandelbrot was named an IBM Fellow in 1974, and continued to work at the IBM Watson Research Center until 1987 when IBM ended pure research in his division. He joined the Department of Mathematics at Yale University and obtained his first tenured position in 1999 at the age of 75. At the time of his retirement in 2005, he was Sterling Professor of Mathematical Sciences.

Mandelbrot died from pancreatic cancer at the age of 85 in Cambridge, Massachusetts on October 14th, 2010. Reacting to news of his death, mathematician Heinz-Otto Peitgen said:

```
"if we talk about impact inside mathematics,
and applications in the sciences,
he is one of the most important figures of the last fifty years."
```

A Little Code

Periodicity Logic

The "Mandelbrot lake" in the center of the M-set images is the traditional bane of plotting programs. It sucks up the most computer time because it always reaches the iteration limit — and yet the most interesting areas are invariably right at the edge the lake. (See The Mandelbrot Set for a description of the iteration process.)

Thanks to Mark Peterson for pointing out (well, he more like beat us over the head until we paid attention) that the iteration values in the middle of Mandelbrot Lake tend to decay to periodic loops (i.e., $Z_{n+m} = Z_n$, a fact that is pointed out on pages 58-61 of "The Beauty of Fractals"). An intelligent program (like the one he wrote) would check for this periodicity once in a while, recognize that iterations caught in a loop are going to max out, and bail out early.

For speed purposes, the current version of the program turns this checking algorithm on only if the last pixel generated was in the lake. (The checking itself takes a small amount of time, and the pixels on the very edge of the lake tend to decay to periodic loops very slowly, so this compromise turned out to be the fastest generic answer).

Try a full M-set plot with a 1000-iteration maximum with any other program, and then try it on this one for a pretty dramatic proof of the value of periodicity checking.

You can get a visual display of the periodicity effects if you press `0` rbits while plotting. This toggles display of the intermediate iterations during the generation process. It also gives you an idea of how much work your poor little PC is going through for you! If you use this toggle, it's best to disable solid-guessing first using `1` or `2` because in its second pass, solid-guessing bypasses many of the pixel calculations precisely where the orbits are most interesting.

Arbitrary Precision and Deep Zooming

The zoom limit of Id is approximately 10^15 (10 to the fifteenth power). This limit is due to the precision possible with the computer representation of numbers as 64 bit double precision data. To give you an idea of just how big a magnification 10^15 is, consider this. At the scale of your computer screen while displaying a tiny part of the Mandelbrot set at the deepest possible zoom, the entire Mandelbrot set would be many millions of miles wide, as big as the orbit of Jupiter.

Big as this zoom magnification is, your PC can do better using something called arbitrary precision math. Instead of using 64 bit double precision to represent numbers, your computer software allocates as much memory as needed to create a data type supporting as many decimals of precision as you want.

Incorporation of this feature in Id was inspired by Jay Hill and his DeepZoom program which uses the shareware MFLOAT programming library. Several of the Stone Soup programmers noticed Jay's posts in the Internet sci.fractals newsgroup and began to investigate adding arbitrary precision to Id. High school math and physics teacher Wes Loewer wrote an arbitrary precision library, and the Stone Soup team incorporated Wes's library into Id's predecessor. Initially, support was added for fractal types mandel, julia, manzpower, and julzpower. Later, support for the fractal type dividebrot5 was added.

Normally, when you reach Id's zoom limit, Id simply refuses to let you zoom any more. When using the fractal types that support arbitrary precision, you will not reach this limit, but can keep on zooming. When you pass the threshold between double precision and arbitrary precision, Id will dramatically slow down. The `Tab` status screen can be used to verify that Id is indeed using arbitrary precision.

Fractals with arbitrary precision are slow, as much as ten times slower than if the math were done with your CPU, and even slower simply because the zoom depth is greater.

Maybe the real point of arbitrary precision math is to prolong the "olden" days when men were men, women were women, and real fractal programmers spent weeks generating fractals. Computers are getting faster every day, and soon a new generation of fractal lovers might forget that fractal generation is supposed to be slow, just as it was in Grandpa's day when they only had Pentium chips. The solution to this educational dilemma is Id's arbitrary precision feature. Even the newest machines are going to have to chug for days or weeks at the extreme zoom depths now possible ...

So how far can you zoom? How does 10^1600 sound—roughly 1600 decimal digits of precision. To put this magnification in perspective, the "tiny" ratio of 10^61 is the ratio of the entire visible universe to the smallest quantum effects. With 1600 digits to work with, you can expand an electron-sized image up to the size of the visible universe, not once but more than twenty times. So you can examine screen-sized portions of a Mandelbrot set so large all but a tiny part of it would be vastly farther away than the billion or so light year limit of our best telescopes.

Lest anyone suppose that we Stone Soupers suffer from an inflated pride over having thus spanned the universe, current inflationary cosmological theories estimate the size of the universe to be unimaginably larger than the tiny part we can see.

Note: many of Id's options do not work with arbitrary precision. To experiment with arbitrary

precision at the speedier ordinary magnifications, start Id with the debug=3200 command-line option. With the exception of mandel, manzpower and dividebrot5 perturbations, values that would normally be entered in the Parameters and Coordinates screens need to be entered using the command-line interface or .par files. Other known things that do not yet work with arbitrary precision are: biomorph, decomp, distance estimator, inversion, Julia-Mandel switch, history, orbit-in-window, and the browse feature.

Perturbation Algorithm

The perturbation algorithm was originally implemented in a program called "SuperFractalThing" by K. I. Martin. The core observation is that orbits for nearby points in the complex plane can be related to each other through a difference equation. Therefore, if a single orbit is computed at high precision from a reference point, the values in the orbits of nearby points can be computed from the reference orbit using lower precision math.

This is the essence of how perturbation methods speed up deep zooms of the Mandelbrot set. Only a few reference points need to be calculated at full precision using slower operations and nearby points can be calculated using faster lesser precision operations.

For the Mandelbrot set, the formula for computing the orbit in the complex plane is:

$$\begin{aligned} z_{n+1} &= z_n^2 + z_0 \\ z_0 &= c \end{aligned}$$

If we consider the orbit of some other point y near c, say c_2, we can write the orbit of this point as:

$$\begin{aligned} y_{n+1} &= y_n^2 + y_0 \\ y_0 &= c_2 \end{aligned}$$

The difference between these two orbits at each iteration is:

$$\begin{aligned} d_n &= y_n - z_n \\ d_{n+1} &= y_{n+1} - z_{n+1} \\ &= (y_n^2 + y_0) - (z_n^2 + z_0) \\ &= \left((z_n + d_n)^2 + z_0 + d_0\right) - (z_n^2 + z_0) \\ &= (z_n^2 + 2z_n d_n + d_n^2 + z_0 + d_0) - (z_n^2 + z_0) \\ &= 2z_n d_n + d_n^2 + d_0 \end{aligned}$$

As long as the terms in d_n are "small", we can use hardware floating-point math to compute the values of the orbit for y_n, having computed the reference orbit for z_n using multiple-precision math.

The Id "Fractal Engine" Architecture

Several of the authors would never admit this, but Id has evolved a powerful and flexible architecture that makes adding new fractals very easy. (They would never admit this because they pride themselves on being the sort that mindlessly but happily hacks away at code and "sees if it works and doesn't hang the machine".)

Many fractal calculations work by taking a rectangle in the complex plane, and, point by point, calculating a color corresponding to that point. Furthermore, the color calculation is often done by iterating a function over and over until some bailout condition is met. (See The Mandelbrot Set for a description of the iteration process.)

In implementing such a scheme, there are three fractal-specific calculations that take place within a framework that is pretty much the same for them all. Rather than copy the same code over and over, we created a standard fractal engine that calls three functions that may be bolted in temporarily to the engine. The bolting in process uses the C++ language mechanism of variable function pointers.

These three functions are:

```
1) a setup function that is run once per image, to do any required
initialization of variables,
```

```
2) a once-per-pixel function that does whatever initialization has to
be done to calculate a color for one pixel, and
```

```
3) a once-per-orbit-iteration function, which is the fundamental
fractal algorithm that is repeatedly iterated in the fractal
calculation.
```

The common framework that calls these functions can contain all sorts of speedups, tricks, and options that the fractal implementor need not worry about. All that is necessary is to write the three functions in the correct way, and bingo! — all options automatically apply. What makes it even easier is that usually one can re-use functions 1) and 2) written for other fractals, and therefore only need to write function 3).

Then it occurred to us that there might be more than one sort of fractal engine, so we even allowed that to be bolted in. And we created a data structure for each fractal that includes pointers to these four functions, various prompts, a default region of the complex plane, and various miscellaneous bits of information that allow toggling between Julia and Mandelbrot or toggling between the various kinds of math used in implementation.

That sounds pretty flexible, but there is one drawback - you have to be a C programmer and have a C compiler to make use of it! So we took it a step further, and designed a built-in high level compiler, so that you can enter the formulas for the various functions in a formula file in a straightforward algebra-like language, and Id will compile them and bolt them in for you!

There is a terrible down side to this flexibility. Id users everywhere are going berserk. Fractal-inventing creativity is running rampant. Proposals for new fractal types are clogging the mail and the telephones.

All we can say is that non-productivity software has never been so potent, and we're sorry, it's our fault!

Id was compiled using Microsoft Visual Studio on Windows and the GNU C compiler, g, on Unix-like systems. Id source code and release packages are available directly through its project page on github. See Distribution of Iterated Dynamics.

Mathematics of the Fractal Types

Summary of Fractal Types

ant Generalized Ant Automaton as described in the July, 1994 Scientific American. Some ants wander around the screen. A rule string (the first parameter) determines the ant's direction. When the type 1 ant leaves a cell of color k, it turns right if the kth symbol in the first parameter is a 1, or left otherwise. Then the color in the old cell is incremented. The 2nd parameter is a maximum iteration to guarantee that the fractal will terminate. The 3rd parameter is the number of ants. The 4th is the ant type 1 or 2. The 5th parameter determines if the ants wrap the screen or stop at the edge. The 6th parameter is a random seed. You can slow down the ants to see them better using the <P> screen Orbit Delay.

barnsleyj1 z(0) = pixel; if real(z) >= 0 z(n+1) = (z-1)*c else z(n+1) = (z+1)*c Two parameters: real and imaginary parts of c.

barnsleyj2 z(0) = pixel; if real(z(n)) * imag(c) + real(c) * imag(zn >= 0 z(n+1) = (z(n)-1)*c else z(n+1) = (z(n)+1)*c Two parameters: real and imaginary parts of c.

barnsleyj3 z(0) = pixel; if real(z(n) > 0 then z(n+1) = (real(z(n))^2 - imag(z(n))^2 - 1) + i * (2*real(zn * imag(zn) else z(n+1) = (real(z(n))^2 - imag(z(n))^2 - 1 + real(c) * real(z(n)) + i * (2*real(zn * imag(zn + imag(c) * real(z(n)) Two parameters: real and imaginary parts of c.

barnsleym1 z(0) = c = pixel; if real(z) >= 0 then z(n+1) = (z-1)*c else z(n+1) = (z+1)*c. Parameters are perturbations of z(0).

barnsleym2 z(0) = c = pixel; if real(z)*imag(c) + real(c)*imag(z) >= 0 z(n+1) = (z-1)*c else z(n+1) = (z+1)*c Parameters are perturbations of z(0).

barnsleym3 z(0) = c = pixel; if real(z(n) > 0 then z(n+1) = (real(z(n))^2 - imag(z(n))^2 - 1)

```
     + i * (2*real(z((n)) * imag(z((n))) else
  z(n+1) = (real(z(n))^2 - imag(z(n))^2 - 1 + real(c) * real(z(n))
     + i * (2*real(z((n)) * imag(z((n)) + imag(c) * real(z(n))
Parameters are perturbations of z(0).
```

bifurcation Pictorial representation of a population growth model. Let P = new population, p = oldpopulation, r = growth rate The model is: P = p + r*fn(p)*(1-fn(p)). Three parameters: Filter Cycles, Seed Population, and Function.

bif+sinpi Bifurcation variation: model is: P = p + r*fn(PI*p). Three parameters: Filter Cycles, Seed Population, and Function.

bif=sinpi Bifurcation variation: model is: P = r*fn(PI*p). Three parameters: Filter Cycles, Seed

Population, and Function.

biflambda Bifurcation variation: model is: P = r*fn(p)*(1-fn(p)). Three parameters: Filter Cycles, Seed Population, and Function.

bifstewart Bifurcation variation: model is: P = (r*fn(p)*fn(p)) - 1. Three parameters: Filter Cycles, Seed Population, and Function.

bifmay Bifurcation variation: model is: P = r*p / ((1+p)^beta). Three parameters: Filter Cycles, Seed Population, and Beta.

burning-ship Classic Mandelbrot Derivative fractal. z(0) = c = pixel; real(tmp) = sqr(real(z(n))) - sqr(imag(z(n))) imag(tmp) = abs(real(z(n)) * imag(z(n))) * -2.0 z(n+1) = tmp + c. Three parameters: perturbations of z(0) and degree.

cellular One-dimensional cellular automata or line automata. The type of CA is given by kr, where k is the number of different states of the automata and r is the radius of the neighborhood. The next generation is determined by the sum of the neighborhood and the specified rule. Four parameters: Initial String, Rule, Type, and Starting Row Number. For Type = 21, 31, 41, 51, 61, 22, 32, 42, 23, 33, 24, 25, 26, 27 Rule = 4, 7, 10, 13, 16, 6, 11, 16, 8, 15, 10, 12, 14, 16 digits

chip Chip attractor from Michael Peters - orbit in two dimensions. x(0) = y(0) = 0; x(n+1) = y(n) - sign(x(n)) * cos(sqr(ln(abs(b*x(n)-c)))) * arctan(sqr(ln(abs(c*x(n)-b)))) y(n+1) = a - x(n) Parameters are a, b, and c.

circle Circle pattern by John Connett x + iy = pixel z = a*(x^2 + y^2) c = integer part of z color = c modulo(number of colors)

cmplxmarksjul A generalization of the marksjulia fractal. z(0) = pixel; z(n+1) = $c^{(exp-1)*z(n)}2$ + c. Four parameters: real and imaginary parts of c, and real and imaginary parts of exponent.

cmplxmarksmand A generalization of the marksmandel fractal. z(0) = c = pixel; z(n+1) = $c^{(exp-1)*z(n)}2$ + c. Four parameters: real and imaginary parts of perturbation of z(0), and real and imaginary parts of exponent.

complexnewton, complexbasin Newton fractal types extended to complex degrees. Complexnewton colors pixels according to the number of iterations required to escape to a root. Complexbasin colors pixels according to which root captures the orbit. The equation is based on the newton formula for solving the equation z^p = r z(0) = pixel; z(n+1) = p - 1) * z(n)^p + r)/(p * z(n)^(p - 1. Four parameters: real & imaginary parts of degree p and root r.

diffusion Diffusion Limited Aggregation. Randomly moving points accumulate. Three parameters: border width (default 10), type, and color change rate.

dividebrot5 DivideBrot5 formula by Jim Muth. z(0) = 0; z(n+1) = sqr(z(n)) / (z(n)^(-(a-2)) + (b+10^(-20))) + pixel Two parameters: a and b

dynamic Time-discrete dynamic system. x(0) = y(0) = start position. y(n+1) = y(n) + f(x(n)) x(n+1) = x(n) - f(y(n)) f(k) = sin(k + a*fn1(b*k)) For implicit Euler approximation: x(n+1) = x(n) - f(y(n+1)) Five parameters: start position step, dt, a, b, and the function fn1.

escher_julia Escher-like tiling of Julia sets from The Science of Fractal Images z(0) = pixel z(n+1) = z(n)^2 + (0, 0i) The target set is a second, scaled, Julia set: T = [z: | (z * 15.0)^2 + c | < BAILOUT] Two parameters: real and imaginary parts of c Iteration count and bailout size apply to both Julia sets.

fn(z)+fn(pix) c = z(0) = pixel; z(n+1) = fn1(z) + p*fn2(c) Six parameters: real and imaginary parts of the perturbation of z(0) and factor p, and the functions fn1, and fn2.

fn(z*z) z(0) = pixel; z(n+1) = fn(z(n)*z(n)) One parameter: the function fn.

fn*fn z(0) = pixel; z(n+1) = fn1(n)*fn2(n) Two parameters: the functions fn1 and fn2.

fn*z+z z(0) = pixel; z(n+1) = p1*fn(z(n))*z(n) + p2*z(n) Five parameters: the real and imaginary components of p1 and p2, and the function fn.

fn+fn z(0) = pixel; z(n+1) = p1*fn1(z(n))+p2*fn2(z(n)) Six parameters: The real and imaginary components of p1 and p2, and the functions fn1 and fn2.

formula Formula interpreter - write your own formulas as text files!

frothybasin Pixel color is determined by which attractor captures the orbit. The shade of color is determined by the number of iterations required to capture the orbit. Z(0) = pixel; Z(n+1) = Z(n)^2 - C*conj(Z(n)) where C = 1 + A*i, critical value of A = 1.028713768218725...

gingerbread Orbit in two dimensions defined by: x(n+1) = 1 - y(n) + |x(n)| y(n+1) = x(n) Two parameters: initial values of x(0) and y(0).

halley Halley map for the function: F = z(z^a - 1) = 0 z(0) = pixel; z(n+1) = z(n) - R * F / [F' - (F'' * F / 2 * F')] bailout when: abs(mod(z(n+1)) - mod(z(n)) < epsilon Four parameters: order a, real part of R, epsilon, and imaginary part of R.

henon Orbit in two dimensions defined by: x(n+1) = 1 + y(n) - a*x(n)*x(n) y(n+1) = b*x(n) Two parameters: a and b.

hopalong Hopalong attractor by Barry Martin - orbit in two dimensions. z(0) = y(0) = 0; x(n+1) = y(n) - sign(x(n))*sqrt(abs(b*x(n)-c)) y(n+1) = a - x(n) Parameters are a, b, and c.

hypercomplex HyperComplex Mandelbrot set. h(0) = (0,0,0,0) h(n+1) = fn(h(n)) + C. where "fn" is sin, cos, log, sqr etc. Two parameters: cj, ck C = (xpixel,ypixel,cj,ck)

hypercomplexj HyperComplex Julia set. h(0) = (xpixel,ypixel,zj,zk) h(n+1) = fn(h(n)) + C. where "fn" is sin, cos, log, sqr etc. Six parameters: c1, ci, cj, ck C = (c1,ci,cj,ck)

icon, icon3d Orbit in three dimensions defined by: p = lambda + alpha * magnitude + beta * (x(n)*zreal - y(n)*zimag) x(n+1) = p * x(n) + gamma * zreal - omega * y(n) y(n+1) = p * y(n) - gamma * zimag + omega * x(n) (3D version uses magnitude for z) Parameters: Lambda, Alpha, Beta, Gamma, Omega, and Degree.

IFS Barnsley IFS (Iterated Function System) fractals. Apply contractive affine mappings.

julfn+exp A generalized Clifford Pickover fractal. z(0) = pixel; z(n+1) = fn(z(n)) + e^z(n) + c. Three parameters: real & imaginary parts of c, and fn.

julfn+zsqrd z(0) = pixel; z(n+1) = fn(z(n)) + z(n)^2 + c Three parameters: real & imaginary parts of c, and fn.

julia Classic Julia set fractal. z(0) = pixel; z(n+1) = z(n)^2 + c. Two parameters: real and imaginary parts of c.

julia_inverse Inverse Julia function - "orbit" traces Julia set in two dimensions. z(0) = a point on the Julia Set boundary; z(n+1) = +- sqrt(z(n) - c) Parameters: Real and Imaginary parts of c Maximum Hits per Pixel (similar to max iters) Breadth First, Depth First or Random Walk Tree Traversal Left or Right First Branching (in Depth First mode only) Try each traversal method, keeping everything else the same. Notice the differences in the way the image evolves. Start with a fairly low Maximum Hit limit, then increase it. The hit limit cannot be higher than the maximum colors in your video mode.

julia(fn||fn) z(0) = pixel; if modulus(z(n)) < shift value, then z(n+1) = fn1(z(n)) + c, else z(n+1) = fn2(z(n)) + c. Five parameters: real, imag portions of c, shift value, fn1, fn2.

julia4 Fourth-power Julia set fractals, a special case of julzpower kept for speed. z(0) = pixel; z(n+1) = z(n)^4 + c. Two parameters: real and imaginary parts of c.

julibrot 'Julibrot' 4-dimensional fractals.

julzpower z(0) = pixel; z(n+1) = z(n)^m + c. Three parameters: real & imaginary parts of c, exponent m.

julzzpwr z(0) = pixel; z(n+1) = z(n)^z(n) + z(n)^m + c. Three parameters: real & imaginary parts of c, exponent m.

kamtorus, kamtorus3d Series of orbits superimposed. 3D version has 'orbit' the z dimension. x(0) = y(0) = orbit/3; x(n+1) = x(n)*cos(a) + (x(n)*x(n)-y(n))*sin(a) y(n+1) = x(n)*sin(a) - (x(n)*x(n)-y(n))*cos(a) After each orbit, 'orbit' is incremented by a step size. Parameters: a, step size, stop value for 'orbit', and points per orbit.

lambda Classic Lambda fractal. 'Julia' variant of Mandellambda. z(0) = pixel; z(n+1) = lambda*z(n)*(1 - z(n)). Two parameters: real and imaginary parts of lambda.

lambdafn z(0) = pixel; z(n+1) = lambda * fn(z(n)). Three parameters: real, imag portions of lambda, and fn.

lambda(fn||fn) z(0) = pixel; if modulus(z(n)) < shift value, then z(n+1) = lambda * fn1(z(n)), else z(n+1) = lambda * fn2(z(n)). Five parameters: real, imag portions of lambda, shift value, fn1, fn2.

latoocarfian Orbit in two dimensions defined by: x(n+1) = fn1 (y(n) * b) + c * fn2(x(n) * b) y(n+1) = fn3 (x(n) * a) + d * fn4(y(n) * a) Parameters: a, b, c, d fn1..4 (all sin=original).

lorenz, lorenz3d Lorenz two lobe attractor - orbit in three dimensions. In 2d the x and y components are projected to form the image.

```
z(0) = y(0) = z(0) = 1;
x(n+1) = x(n) + (-a*x(n)*dt) + (   a*y(n)*dt)
```

```
y(n+1) = y(n) + ( b*x(n)*dt) - (     y(n)*dt) - (z(n)*x(n)*dt)
z(n+1) = z(n) + (-c*z(n)*dt) + (x(n)*y(n)*dt)
```

Parameters are dt, a, b, and c.

lorenz3d1 Lorenz one lobe attractor, 3D orbit (Rick Miranda and Emily Stone)

```
z(0) = y(0) = z(0) = 1; norm = sqrt(x(n)^2 + y(n)^2)
x(n+1) = x(n) + (-a*dt-dt)*x(n) + (a*dt-b*dt)*y(n)
   + (dt-a*dt)*norm + y(n)*dt*z(n)
y(n+1) = y(n) + (b*dt-a*dt)*x(n) - (a*dt+dt)*y(n)
   + (b*dt+a*dt)*norm - x(n)*dt*z(n) - norm*z(n)*dt
z(n+1) = z(n) +(y(n)*dt/2) - c*dt*z(n)
```

Parameters are dt, a, b, and c.

lorenz3d3 Lorenz three lobe attractor, 3D orbit (Rick Miranda and Emily Stone)

```
z(0) = y(0) = z(0) = 1; norm = sqrt(x(n)^2 + y(n)^2)
x(n+1) = x(n) +(-(a*dt+dt)*x(n) + (a*dt-b*dt+z(n)*dt)*y(n))/3
    + ((dt-a*dt)*(x(n)^2-y(n)^2)
    + 2*(b*dt+a*dt-z(n)*dt)*x(n)*y(n))/(3*norm)
y(n+1) = y(n) +((b*dt-a*dt-z(n)*dt)*x(n) - (a*dt+dt)*y(n))/3
    + (2*(a*dt-dt)*x(n)*y(n)
    + (b*dt+a*dt-z(n)*dt)*(x(n)^2-y(n)^2))/(3*norm)
z(n+1) = z(n) +(3*x(n)*dt*x(n)*y(n)-y(n)*dt*y(n)^2)/2 - c*dt*z(n)
```

Parameters are dt, a, b, and c.

lorenz3d4 Lorenz four lobe attractor, 3D orbit (Rick Miranda and Emily Stone)

```
z(0) = y(0) = z(0) = 1;
x(n+1) = x(n) +(-a*dt*x(n)^3
   + (2*a*dt+b*dt-z(n)*dt)*x(n)^2*y(n) + (a*dt-2*dt)*x(n)*y(n)^2
   + (z(n)*dt-b*dt)*y(n)^3) / (2 * (x(n)^2+y(n)^2))
y(n+1) = y(n) +((b*dt-z(n)*dt)*x(n)^3 + (a*dt-2*dt)*x(n)^2*y(n)
   + (-2*a*dt-b*dt+z(n)*dt)*x(n)*y(n)^2
   - a*dt*y(n)^3) / (2 * (x(n)^2+y(n)^2))
z(n+1) = z(n) +(2*x(n)*dt*x(n)^2*y(n) - 2*x(n)*dt*y(n)^3 - c*dt*z(n))
```

Parameters are dt, a, b, and c.

lsystem Using a turtle-graphics control language and starting with an initial axiom string, carries out string substitutions the specified number of times (the order), and plots the result.

lyapunov Derived from the Bifurcation fractal, the Lyapunov plots the Lyapunov Exponent for a population model where the Growth parameter varies between two values in a periodic manner.

magnet1j z(0) = pixel; [z(n)^2 + (c-1)] 2 z(n+1) = | ---------- | [2*z(n) + (c-2)] Parameters: the real and imaginary parts of c.

magnet1m z(0) = 0; c = pixel; [z(n)^2 + (c-1)] 2 z(n+1) = | ---------- | [2*z(n) + (c-2)] Parameters: the real & imaginary parts of perturbation of z(0).

magnet2j z(0) = pixel; [z(n)^3 + 3*(C-1)**z(n)** + **(C-1)**(C-2)] 2 z(n+1) = | ---------- | [3*(z(n)^2) + 3*(C-2)**z(n)** + **(C-1)**(C-2) + 1] Parameters: the real and imaginary parts of c.

magnet2m z(0) = 0; c = pixel; [z(n)^3 + 3*(C-1)**z(n)** + **(C-1)**(C-2)] 2 z(n+1) = | ---------- | [3*(z(n)^2) + 3*(C-2)**z(n)** + **(C-1)**(C-2) + 1] Parameters: the real and imaginary parts of perturbation of z(0).

mandel Classic Mandelbrot set fractal. z(0) = c = pixel; z(n+1) = z(n)^2 + c. Two parameters: real & imaginary perturbations of z(0).

mandel(fn||fn) c = pixel; z(0) = p1 if modulus(z(n)) < shift value, then z(n+1) = fn1(z(n)) + c, else z(n+1) = fn2(z(n)) + c. Five parameters: real, imaginary portions of p1, shift value, fn1 and fn2.

mandelbrotmix4 From a Jim Muth favorite FOTD formula a=real(p1), b=imag(p1), d=real(p2), f=imag(p2), g=1/f, h=1/d, j=1/(f-b), z=(-a*b*g*h)^j, k=real(p3)1, l=imag(p3)+100, c=fn1(pixel): z=k*((a*(z^b))(d*(z^f)))+c Parameters: see above, sixth parameter used for bailout.

mandelcloud Displays orbits of Mandelbrot set: z(0) = c = pixel; z(n+1) = z(n)^2 + c. One parameter: number of intervals.

mandel4 Special case of mandelzpower kept for speed. z(0) = c = pixel; z(n+1) = z(n)^4 + c. Parameters: real & imaginary perturbations of z(0).

mandelfn z(0) = c = pixel; z(n+1) = c*fn(z(n)). Parameters: real & imaginary perturbations of z(0), and fn.

manlam(fn||fn) c = pixel; z(0) = p1 if modulus(z(n)) < shift value, then z(n+1) = fn1(z(n)) * c, else z(n+1) = fn2(z(n)) * c. Five parameters: real, imaginary parts of p1, shift value, fn1, fn2.

Martin Attractor fractal by Barry Martin - orbit in two dimensions. z(0) = y(0) = 0; x(n+1) = y(n) - sin(x(n)) y(n+1) = a - x(n) Parameter is a (try a value near pi).

mandellambda z(0) = .5; lambda = pixel; z(n+1) = lambda*z(n)*(1 - z(n)). Parameters: real & imaginary perturbations of z(0).

mandphoenix z(0) = c = pixel, y(0) = 0; For degree = 0: z(n+1) = z(n)^2 + c.x + c.y*y(n), y(n+1) = z(n) For degree >= 2: z(n+1) = z(n)^degree + c.x*z(n)^(degree-1) + c.y*y(n) y(n+1) = z(n) For degree ⇐ -3: z(n+1) = z(n)^|degree| + c.x*z(n)^(|degree|-2) + c.y*y(n) y(n+1) = z(n) Three parameters: real & imaginary perturbations of z(0), and degree.

mandphoenixclx z(0) = c = pixel, y(0) = 0; For degree = 0: z(n+1) = z(n)^2 + c + p2*y(n), y(n+1) = z(n) For degree >= 2: z(n+1) = z(n)^degree + c*z(n)^(degree-1) + p2*y(n), y(n+1) = z(n) For degree ⇐ -3: z(n+1) = z(n)^|degree| + c*z(n)^(|degree|-2) + p2*y(n), y(n+1) = z(n) Five parameters: real & imaginary perturbations of z(0), real & imaginary parts of p2, and degree.

manfn+exp 'Mandelbrot-Equivalent' for the julfn+exp fractal. z(0) = c = pixel; z(n+1) = fn(z(n)) + e^z(n) + C. Parameters: real & imaginary perturbations of z(0), and fn.

manfn+zsqrd 'Mandelbrot-Equivalent' for the Julfn+zsqrd fractal. z(0) = c = pixel; z(n+1) = fn(z(n)) + z(n)^2 + c. Parameters: real & imaginary perturbations of z(0), and fn.

manowar c = z1(0) = z(0) = pixel; z(n+1) = z(n)^2 + z1(n) + c; z1(n+1) = z(n); Parameters: real & imaginary perturbations of z(0).

manowarj z1(0) = z(0) = pixel; z(n+1) = z(n)^2 + z1(n) + c; z1(n+1) = z(n); Parameters: real & imaginary parts of c.

manzpower 'Mandelbrot-Equivalent' for julzpower. z(0) = c = pixel; z(n+1) = z(n)^exp + c; try exp = e = 2.71828... Parameters: real & imaginary perturbations of z(0), real & imaginary parts of exponent exp.

manzzpwr 'Mandelbrot-Equivalent' for the julzzpwr fractal. z(0) = c = pixel z(n+1) = z(n)^z(n) + z(n)^exp + C. Parameters: real & imaginary perturbations of z(0), and exponent.

marksjulia A variant of the julia-lambda fractal. z(0) = pixel; z(n+1) = $c^{(exp-1)*z(n)}2$ + c. Parameters: real & imaginary parts of c, and exponent.

marksmandel A variant of the mandel-lambda fractal. z(0) = c = pixel; z(n+1) = $c^{(exp-1)*z(n)}2$ + c. Parameters: real & imaginary parts of perturbations of z(0), and exponent.

marksmandelpwr The marksmandelpwr formula type generalized (it previously had fn=sqr hard coded). z(0) = pixel, c = z(0) ^ (z(0) - 1): z(n+1) = c * fn(z(n)) + pixel, Parameters: real and imaginary perturbations of z(0), and fn.

newtbasin Based on the Newton formula for finding the roots of z^p - 1. Pixels are colored according to which root captures the orbit. z(0) = pixel; z(n+1) = p-1)*z(n)^p + 1)/(p*z(n)^(p - 1. Two parameters: the polynomial degree p, and a flag to turn on color stripes to show alternate iterations.

newton Based on the Newton formula for finding the roots of z^p - 1. Pixels are colored according to the iteration when the orbit is captured by a root. z(0) = pixel; z(n+1) = p-1)*z(n)^p + 1)/(p*z(n)^(p - 1. One parameter: the polynomial degree p.

phoenix z(0) = pixel, y(0) = 0; For degree = 0: z(n+1) = z(n)^2 + p1.x + p2.x*y(n), y(n+1) = z(n) For degree >= 2: z(n+1) = z(n)^degree + p1.x*z(n)^(degree-1) + p2.x*y(n), y(n+1) = z(n) For degree ⇐ -3: z(n+1) = z(n)^|degree| + p1.x*z(n)^(|degree|-2) + p2.x*y(n), y(n+1) = z(n) Three parameters: real parts of p1 & p2, and degree.

phoenixcplx z(0) = pixel, y(0) = 0; For degree = 0: z(n+1) = z(n)^2 + p1 + p2*y(n), y(n+1) = z(n) For degree >= 2: z(n+1) = z(n)^degree + p1*z(n)^(degree-1) + p2*y(n), y(n+1) = z(n) For degree ⇐ -3: z(n+1) = z(n)^|degree| + p1*z(n)^(|degree|-2) + p2*y(n), y(n+1) = z(n) Five parameters: real & imaginary parts of p1 & p2, and degree.

pickover Orbit in three dimensions defined by: x(n+1) = sin(a*y(n)) - z(n)*cos(b*x(n)) y(n+1) = z(n)*sin(c*x(n)) - cos(d*y(n)) z(n+1) = sin(x(n)) Parameters: a, b, c, and d.

plasma Random, cloud-like formations. Requires 4 or more colors. A recursive algorithm repeatedly subdivides the screen and colors pixels according to an average of surrounding pixels and a random color, less random as the grid size decreases. Four parameters: 'graininess' (0, 0.125 to 100, default = 2), old/new algorithm, seed value used, 16-bit out output selection.

popcorn The orbits in 2D are plotted superimposed: x(0) = xpixel, y(0) = ypixel; x(n+1) = x(n) - real(h * fn1(y(n) + fn2(C * y(n))) - imag(h * fn3(x(n) + fn4(C * x(n))) y(n+1) = y(n) - real(h * fn3(x(n) + fn4(C * x(n))) - imag(h * fn1(y(n) + fn2(C * y(n))) Parameters: step size h, C, functions fn1..4 (original: sin,tan,sin,tan).

popcornjul Julia using the generalized Pickover Popcorn formula: x(0) = xpixel, y(0) = ypixel; x(n+1) = x(n) - real(h * fn1(y(n) + fn2(C * y(n))) - imag(h * fn3(x(n) + fn4(C * x(n))) y(n+1) = y(n) - real(h * fn3(x(n) + fn4(C * x(n))) - imag(h * fn1(y(n) + fn2(C * y(n))) Parameters: step size h, C, functions fn1..4 (original: sin,tan,sin,tan).

quadruptwo Quadruptwo attractor from Michael Peters - orbit in two dimensions. z(0) = y(0) = 0; x(n+1) = y(n) - sign(x(n)) * sin(ln(abs(b*x(n)-c))) * arctan(sqr(ln(abs(c*x(n)-b)))) y(n+1) = a - x(n) Parameters are a, b, and c.

quatjul Quaternion Julia set. q(0) = (xpixel,ypixel,zj,zk) q(n+1) = q(n)*q(n) + c. Four parameters: c, ci, cj, ck c = (c1,ci,cj,ck)

quat Quaternion Mandelbrot set. q(0) = (0,0,0,0) q(n+1) = q(n)*q(n) + c. Two parameters: cj,ck c = (xpixel,ypixel,cj,ck)

rossler3D Orbit in three dimensions defined by: x(0) = y(0) = z(0) = 1; x(n+1) = x(n) - y(n)*dt - z(n)*dt y(n+1) = y(n) + x(n)*dt + a*y(n)*dt z(n+1) = z(n) + b*dt + x(n)*z(n)*dt - c*z(n)*dt Parameters are dt, a, b, and c.

sierpinski Sierpinski gasket - Julia set producing a 'Swiss cheese triangle' z(n+1) = (2*x,2*y-1) if y > .5; else (2*x-1,2*y) if x > .5; else (2*x,2*y) No parameters.

spider c(0) = z(0) = pixel; z(n+1) = z(n)^2 + c(n); c(n+1) = c(n)/2 + z(n+1) Parameters: real & imaginary perturbation of z(0).

sqr(1/fn) z(0) = pixel; z(n+1) = (1/fn(z(n))^2 One parameter: the function fn.

sqr(fn) z(0) = pixel; z(n+1) = fn(z(n))^2 One parameter: the function fn.

test 'test' point letting us (and you!) easily add fractal types via the source file testpt.cpp. Default set up is a mandelbrot fractal.

```
Four parameters: user hooks (not used by default testpt.cpp).
```

tetrate z(0) = c = pixel; z(n+1) = c^z(n) Parameters: real & imaginary perturbation of z(0).

threeply Threeply attractor by Michael Peters - orbit in two dimensions. z(0) = y(0) = 0; x(n+1) = y(n) - sign(x(n)) * (abs(sin(x(n))*cos(b) +c-x(n)*sin(a+b+c))) y(n+1) = a - x(n) Parameters are a, b, and c.

tim's_error A serendipitous coding error in marksmandelpwr brings to life an ancient pterodactyl!

(Try setting fn to sqr.) z(0) = pixel, c = z(0) ^ (z(0) - 1): tmp = fn(z(n)) real(tmp) = real(tmp) * real(c) - imag(tmp) * imag(c); imag(tmp) = real(tmp) * imag(c) - imag(tmp) * real(c); z(n+1) = tmp + pixel; Parameters: real & imaginary perturbations of z(0) and function fn.

unity z(0) = pixel; x = real(z(n)), y = imag(z(n)) One = x^2 + y^2; y = (2 - One) * x; x = (2 - One) * y; z(n+1) = x + i*y No parameters.

volterra-lotka Volterra-Lotka fractal from The Beauty of Fractals x(0) = xpixel, y(0) = ypixel; dx/dt = x - xy = f(x,y) dy/dt = -y + xy = g(x,y) x(new) = x + h/2 * [f(x,y) + f[x + pf(x,y), y + pg(x,y)]] y(new) = y + h/2 * [g(x,y) + g[x + pf(x,y), y + pg(x,y)]] Two parameters: h and p. Recommended: zmag or bof60 inside coloring options.

Inside=bof60|bof61|zmag|fmod|period|atan

Here is an attempted explanation of what the "inside=bof60" and "inside=bof61" options do. This explanation is hereby dedicated to Adrian Mariano, who badgered it out of us! For the real explanation, see "Beauty of Fractals", page 62.

Let $p(z)$ be the function that is repeatedly iterated to generate a fractal using the escape-time algorithm. For example, $p(z) = z^2 + c$ in the case of a Julia set. Then let $p^k(z)$ be the result of iterating the function p for k iterations. We could also use the notation $p_c^k(z)$ when the function p has a parameter c, as it does in our example. Now hold your breath and get your thinking cap on. Define $a(c) = \inf_{k = 1, 2, 3, \ldots} |p_c^k(0)|$. In English - $a(c)$ is the greatest lower bound of the images of zero of as many iterations as you like. Put another way, $a(c)$ is the closest to the origin any point in the orbit starting with 0 gets. Then the $\mathrm{index}(c)$ is the value of k (the iteration) when that closest point was achieved. Since there may be more than one, $\mathrm{index}(c)$ is the least such. Got it? Good, because the "Beauty of Fractals" explanation of this, is, ahhhh, terse! Now for the punch line. "Inside=bof60" colors the lake alternating shades according to the level sets of a(c). Each band represents solid areas of the fractal where the closest value of the orbit to the origin is the same. "Inside=bof61" show domains where index(c) is constant. That is, areas where the iteration when the orbit swooped closest to the origin has the same value. Well, folks, that's the best we can do! Improved explanations will be accepted for the next edition!

In response to this request for lucidity, Herb Savage offers this explanation the bof60 and bof61 options:

```
The picture on page 60 of The Beauty of Fractals shows the distance to
origin of the closest point to the origin in the sequence of points
generated from a given X,Y coordinate.  The picture on page 61 shows
the index (or number) in the sequence of the closest point.
```

"Inside=zmag" is similar. This option colors inside pixels according to the magnitude of the orbit point when maxiter was reached, using the formula $\mathrm{color} = (x^2 + y^2) \cdot \frac{\mathrm{maxiter}}{2} + 1$.

"Inside=fmod" colors inside pixels according to the magnitude of the last orbit point which is within a set distance from the origin. Then $\mathrm{color} = \mathrm{magnitude} \cdot \frac{\mathrm{colors}}{\mathrm{closeprox}}$. The magnitude used for the comparison is now based on the same calculation as is used for the bailout test. The value of closeprox can be varied interactively. This feature was contributed by Iain Stirling.

"Inside=period" colors pixels according to the length of their eventual cycle. For example, points that approach a fixed point have color = 1. Points that approach a 2-cycle have color = 2. Points that do not approach a cycle during the iterations performed have color = maxit. This option works best with a fairly large number of iterations.

"Inside=atan" colors by determining the angle in degrees the last iterated value has with respect to the real axis, and using the absolute value. This feature should be used with periodicity=0.

Inside=epscross|startrail

Kenneth Hooper has written a paper entitled "A Note On Some Internal Structures of the Mandelbrot Set" published in "Computers and Graphics", Vol 15, No.2, pp. 295-297. In that article he describes Clifford Pickover's epsilon cross method which creates some mysterious plant-like tendrils in the Mandelbrot set. The algorithm is this. In the escape- time calculation of a fractal, if the orbit comes within .01 of the Y- axis, the orbit is terminated and the pixel is colored green. Similarly, the pixel is colored yellow if it approaches the X-axis. Strictly speaking, this is not an "inside" option because a point destined to escape could be caught by this bailout criterion.

The test distance, 0.01, can now be changed interactively on the `X` screen and via the "proximity=<nnn>" command line parameter. A negative value of the test distance triggers an alternative variation of epsilon cross that colors the epsilon bands with the iteration; otherwise they are colored normally to maintain compatibility.

Hooper has another coloring scheme called star trails that involves detecting clusters of points being traversed by the orbit. A table of tangents of each orbit point is built, and the pixel colored according to how many orbit points are near the first one before the orbit flies out of the cluster. This option looks fine with maxiter=16, which greatly speeds the calculation.

Both of these options should be tried with the outside color fixed ("outside=<nnn>") so that the lake structure revealed by the algorithms can be more clearly seen. Epsilon cross is fun to watch with boundary tracing turned on - even though the result is incorrect it is interesting! Shucks - what does "incorrect" mean in chaos theory anyway?!

Finite Attractors

Many of Id's fractals involve the iteration of functions of complex numbers until some bailout value is exceeded, then coloring the associated pixel according to the number of iterations performed. This process identifies which values tend to infinity when iterated, and gives us a rough measure of how quickly they get there.

In dynamical terms, we say that infinity is an attractor, as many initial values get attracted to it when iterated. The set of all points that are attracted to infinity is termed the basin of attraction of infinity. The coloring algorithm used divides this basin of attraction into many distinct sets, each a single band of one color, representing all the points that are attracted to infinity at the same rate. These sets (bands of color) are termed level sets - all points in such a set are at the same level away from the attractor, in terms of numbers of iterations required to exceed the bailout value.

Thus, Id produces colored images of the level sets of the basin of attraction of infinity, for all

fractals that iterate functions of complex numbers, at least. Now we have a sound mathematical definition of what Id's bailout processing generates, and we have formally introduced the terms attractor, basin of attraction, and level set, so you should have little trouble following the rest of this section!

For certain Julia-type fractals, Id can also display the level sets of basins of attraction of finite attractors. This capability is a by- product of the implementation of the magnetic fractal types, which always have at least one finite attractor.

This option can be invoked by setting the "Look for finite attractor" option on the `Y` options screen, or by giving the "finattract=yes" command-line option.

Most Julia-types that have a lake (normally colored blue by default) have a finite attractor within this lake, and the lake turns out to be, quite appropriately, the basin of attraction of this attractor.

The "finattract=yes" option (command-line or `Y` options screen) instructs Id to seek out and identify a possible finite attractor and, if found, to display the level sets of its basin of attraction, in addition to those of the basin of attraction of infinity. In many cases this results in a lake with colored waves in it; in other cases there may be little change in the lake's appearance.

For a quick demonstration, select a fractal type of lambda, with a parameter of 0.5 + 0.5i. You will obtain an image with a large blue lake. Now set "Look for finite attractor" to 1 with the `Y` menu. The image will be re-drawn with a much more colorful lake. A finite attractor lives in the center of one of the resulting ripple patterns in the lake - turn the `O`rbits display on to see where it is - the orbits of all initial points that are in the lake converge there.

Id tests for the presence of a finite attractor by iterating a critical value of the fractal's function. If the iteration doesn't bail out before exceeding twice the iteration limit, it is almost certain that we have a finite attractor - we assume that we have.

Next we define a small circle around it and, after each iteration, as well as testing for the usual bailout value being exceeded, we test to see if we've hit the circle. If so, we bail out and color our pixels according to the number of iterations performed. Result - a nicely colored-in lake that displays the level sets of the basin of attraction of the finite attractor. Sometimes!

First, this does not work for the lakes of Mandel-types. Every point in a Mandel-type is, in effect, a single point plucked from one of its related Julia-types. A Mandel-type's lake has an infinite number of points, and thus an infinite number of related Julia-type sets, and consequently an infinite number of finite attractors too. It may be possible to color in such a lake, by determining the attractor for every pixel, but this would probably treble (at least) the number of iterations needed to draw the image. Due to this overhead, finite attractor logic has not been implemented for Mandel-types.

Second, certain Julia-types with lakes may not respond to this treatment, depending on the parameter value used. E.g., the lambda set for 0.5 + 0.5i responds well; the lambda set for 0.0 + 1.0i does not - its lake stays blue. Attractors that consist of single points, or a cycle of a finite number of points are ok. Others are not. If you're into fractal technospeak, the implemented approach fails if the Julia- type is a parabolic case, or has Siegel disks, or has Herman rings.

However, all the difficult cases have one thing in common - they all have a parameter value that

falls exactly on the edge of the related Mandel-type's lake. You can avoid them by intelligent use of the Mandel-Julia Space-Bar toggle: Pick a view of the related Mandel-type where the center of the screen is inside the lake, but not too close to its edge, then use the space-bar toggle. You should obtain a usable Julia-type with a lake, if you follow this guideline.

Third, the initial implementation only works for Julia-types that use the standard fractal engine in Id. Fractals with their own special algorithms are not affected by finite attractor logic, as yet.

Finally, the finite attractor code will not work if it fails to detect a finite attractor. If the number of iterations is set too low, the finite attractor may be missed.

Despite these restrictions, the finite attractor logic can produce interesting results. Just bear in mind that it is principally a bonus off-shoot from the development of the magnetic fractal types, and is not specifically tuned for optimal performance for other Julia types.

(Thanks to Kevin Allen for the above).

There is a second type of finite attractor coloring, which is selected by setting "Look for Finite Attractor" to a negative value. This colors points by the phase of the convergence to the finite attractor, instead of by the speed of convergence.

For example, consider the Julia set for -0.1 + 0.7i, which is the three- lobed rabbit set. The finite attractor is an orbit of length three; call these values a, b, and c. Then, the Julia set iteration can converge to one of three sequences: a,b,c,a,b,c,..., or b,c,a,b,c,..., or c,a,b,c,a,b,... The finite attractor phase option colors the interior of the Julia set with three colors, depending on which of the three sequences the orbit converges to. Internally, the code determines one point of the orbit, say a, and the length of the orbit cycle, say 3. It then iterates until the sequence converges to a, and then uses the iteration number modulo 3 to determine the color.

Trig Identities

The following trig identities are invaluable for coding fractals that use complex-valued transcendental functions of a complex variable in terms of real-valued functions of a real variable, which are usually found in compiler math libraries. In what follows, we sometimes use "*" for multiplication, but leave it out when clarity is not lost. We use "^" for exponentiation; x^y is x to the y power.

$$(u+iv)+(x+iy)=(u+x)+i(v+y)(u+iv)-(x+iy)=(u-x)+i(v-y)(u+iv)(x+iy)=(ux-vy)+i(vx+uy)\frac{u+iv}{x+iy}=\frac{(ux+vy)+i(vx-uy)}{x^2+y^2}e^{x+iy}=e^x(\cos(y)+i\sin(y))\log(x+iy)=\left(\frac{1}{2}\right)\log(x^2+y^2)+i\left(\tan^{-1}\left(\frac{y}{x}\right)+2k\pi\right)\text{for}k=0,\ -1,\ 1,\ -2,\ 2,\ ...$$

(The $\log$ function refers to log base e, or $\ln$. The expression $\tan^{-1}\left(\frac{y}{x}\right)$ is an angle between $-\pi$ and π in the quadrant containing $(x,\ y)$.)

$$z^w=e^{w\log(z)}\sin(x+iy)=\sin(x)\cosh(y)+i\cos(x)\sinh(y)\cos(x+iy)=\cos(x)\cosh(y)-i\sin(x)\sinh(y)\tan(x+iy)=\sin\frac{x+iy}{\cos}(x+iy)\sinh(x+iy)=\sinh(x)\cos(y)+i\cosh(x)\sin(y)\cosh(x+iy)=\cosh(x)\cos(y)+i\sinh(x)\sin(y)\tanh(x+iy)=\sinh\frac{x+iy}{\cosh}(x+iy)\cosxx(x+iy)=\cos(x)\cosh(y)+i\sin(x)\sinh(y)$$

($\cosxx$ is present to provide compatibility with a bug in its $\cos$ calculation)

$$\tan(x+iy)=\frac{2x}{\cos(2x)+\cosh(2y)}+i\sinh\frac{2y}{\cos(2x)+\cosh(2y)}\sin^{-1}(x+iy)=\frac{i\log(1-iz)-i\log(1+iz)}{2}\cos^{-1}(x+iy)=\frac{\pi}{2}-\sin^{-1}(x+iy)\sinh^{-1}(x+iy)=\log\left(x+\sqrt{x^2+1}\right)\tan^{-1}(x+iy)=\left(\frac{i}{2}\right)\log\left(\frac{i+z}{i-z}\right)\cosh^{-1}(x+iy)=\log\left(x+\sqrt{x^2-1}\right)\tanh^{-1}(x+iy)=\left(\frac{1}{2}\right)\log\left(\frac{1+z}{1-z}\right)+i\pi/2\ \text{if}\ z<-1\ \text{or}\ z>1\ \text{else}\ 0\sqrt{x+iy}=\sqrt{\frac{x+\sqrt{x^2+y^2}}{2}}+i\sqrt{\frac{-x+\sqrt{x^2+y^2}}{2}}\cos\left(\sin^{-1}\frac{y}{x}\right)+i\sin\left(\sin^{-1}\frac{y}{x}\right)\text{if}\ \text{abs}(x+iy)=1\ \text{or}\ \exp(x+iy)=1\ \text{or}\ \text{cis}(x+iy)=1\ \text{or}\ \text{cosxx}(x+iy)=1\ \text{or}\ \text{sinxx}(x+iy)=1\ \text{or}\ \text{tanxx}(x+iy)=1\ \text{or}\ \text{cabs}(x+iy)=1\ \text{or}\ \sqrt{x^2+y^2}\ \text{or}\ \text{floor}(x+iy)=\text{floor}(x)+i\ \text{floor}(y)\ \text{or}\ \text{ceil}(x+iy)=\text{ceil}(x)+i\ \text{ceil}(y)\ \text{or}\ \text{trunc}(x+iy)=\text{trunc}(x)+i\ \text{trunc}(y)\ \text{or}\ \text{round}(x+iy)=\text{round}(x)+i\ \text{round}(y)$$

Id's definitions of $\text{abs}(x+iy)$ and $|x+iy|$ below are non-standard. Math texts define both absolute value and modulus of a complex number to be the same thing. They are both equal to $\text{cabs}(x+iy)$ as defined above.

$$|x+iy| = x^2+y^2 \quad \text{abs}(x+iy) = \sqrt{x^2}+i\sqrt{y^2} = (\text{cabs}(x+iy))^2$$

Quaternion and Hypercomplex Algebra

Quaternions are four dimensional generalizations of complex numbers. They almost obey the familiar field properties of real numbers, but fail the commutative law of multiplication, since xy is not generally equal to yx.

Quaternion algebra is most compactly described by specifying the rules for multiplying the basis vectors 1, i, j, and k. Quaternions form a superset of the complex numbers, and the basis vectors 1 and i are the familiar basis vectors for the complex algebra. Any quaternion q can be represented as a linear combination $q = x+yi+zj+wk$ of the basis vectors just as any complex number can be written in the form $z = a+bi$.

Multiplication rules for quaternion basis vectors:

$$\begin{array}{lll} ij = k & jk = i & ki = j \\ ji = -k & kj = -i & ik = -j \\ = -1 & jj = -1 & kk = -1 \\ ijk = -1 & & \end{array}$$

Note that $ij = k$ but $ji = -k$, showing the failure of the commutative law. The rules for multiplying any two quaternions follow from the behavior of the basis vectors just described. However, for your convenience, the following formula works out the details.

Let $q_1 = x_1 + y_1 i + z_1 j + w_1 k$ $q_2 = x_2 + y_2 i + z_2 j + w_2 k$. Then

$$\begin{array}{rcl} q_1 q_2 & = & (x_1x_2 - y_1y_2 - z_1z_2 - w_1w_2) + \\ & & i(y_1x_2 + x_1y_2 - w_1z_2 + z_1w_2) + \\ & & j(z_1x_2 + w_1y_2 + x_1z_2 - y_1w_2) + \\ & & k(w_1x_2 + z_1y_2 - y_1z_2 + x_1w_2) \end{array}$$

Quaternions are not the only possible four dimensional supersets of the complex numbers. William Rowan Hamilton, who discovered quaternions in 1843, considered the alternative called the hypercomplex number system. Unlike quaternions, the hypercomplex numbers satisfy the commutative law of multiplication. The law which fails is the field property that states that all non-zero elements of a field have a multiplicative inverse. For a non-zero hypercomplex number h, the multiplicative inverse 1/h does not always exist.

As with quaternions, we will define multiplication in terms of the basis vectors 1, i, j, and k, but with subtly different rules.

Multiplication rules for hypercomplex basis vectors:

$$\begin{array}{lll} ij = k & jk = -i & ki = -j \\ ji = k & kj = -i & ik = -j \\ = -1 & jj = -1 & kk = 1 \\ ijk = 1 & & \end{array}$$

Note that now $ij = k$ and $ji = k$, and similarly for other products of pairs of basis vectors, so the commutative law holds.

Hypercomplex multiplication formula: Let $h_1 = x_1 + y_1 i + z_1 j + w_1 k$ and $h_2 = x_2 + y_2 i + z_2 j + w_2 k$. Then

$$\begin{aligned} h_1 h_2 \quad = \quad & (x_1 x_2 - y_1 y_2 - z_1 z_2 + w_1 w_2) + \\ & i(y_1 x_2 + x_1 y_2 - w_1 z_2 - z_1 w_2) + \\ & j(z_1 x_2 - w_1 y_2 + x_1 z_2 - y_1 w_2) + \\ & k(w_1 x_2 + z_1 y_2 + y_1 z_2 + x_1 w_2) \end{aligned}$$

As an added bonus, we'll give you the formula for the reciprocal.

Let $\det = ((x-w)^2 + (y+z)^2)((x+w)^2 + (y-z)^2)$ Then

$$\begin{aligned} \frac{1}{h} \quad = \quad & \frac{x(x^2+y^2+z^2+w^2) - 2w(xw - yz)}{\det} + \\ & i\frac{-y(x^2+y^2+z^2+w^2) - 2z(xw - yz)}{\det} + \\ & j\frac{-z(x^2+y^2+z^2+w^2) - 2y(xw - yz)}{\det} + \\ & k\frac{w(x^2+y^2+z^2+w^2) - 2x(xw - yz)}{\det} \end{aligned}$$

A look at this formula shows the difficulty with hypercomplex numbers. In order to calculate 1/h, you have to divide by the quantity $\det = ((x-w)^2 + (y+z)^2)((x+w)^2 + (y-z)^2)$. So when this quantity is zero, the multiplicative inverse will not exist.

Hypercomplex numbers have an elegant generalization of any unary complex valued function defined on the complex numbers. First, note that hypercomplex numbers can be represented as a pair of complex numbers in the following way. Let

$$\begin{aligned} h &= x + yi + zj + wk \\ a &= (x - w) + i(y + z) \\ b &= (x + w) + i(y - z) \end{aligned}$$

The numbers a and b are complex numbers. We can represent h as the pair of complex numbers (a,b). Conversely, if we have a hypercomplex number given to us in the form (a,b), we can solve for x, y, z, and w. The solution to

$$\begin{aligned} c &= (x - w) + i(y + z) \\ d &= (x + w) + i(y - z) \end{aligned}$$

is

$$x = \frac{\text{real}(c) + \text{real}(d)}{2}$$
$$y = \frac{\text{imag}(c) + \text{imag}(d)}{2}$$
$$z = \frac{\text{imag}(c) - \text{imag}(d)}{2}$$
$$w = \frac{\text{real}(d) - \text{real}(c)}{2}$$

We can now, for example, compute $\sin(h)$. First compute the two complex numbers a and b as above, then set $c = \sin(a)$ and $d = \sin(b)$ where $\sin()$ is the complex version of the sin function. Now use the equations above to solve for x, y, z, and w in terms of c and d. The hypercomplex number (x, y, z, w) thus obtained is $\sin(h)$.

The beauty of this is that it really doesn't make any difference what function we use. Instead of sin, we could have used cos, sinh, ln, or z^2. Using this technique, Id can create 3D fractals using the formula $h' = \text{fn}(h) + c$, where "fn" is any of the built-in functions. Where fn is sqr(), this is the famous mandelbrot formula, generalized to four dimensions.

For more information, see *Fractal Creations, Second Edition* by Tim Wegner and Bert Tyler, Waite Group Press, 1993.

Stone Soup With Pixels: The Authors

The Stone Soup Story

Once upon a time, somewhere in Eastern Europe, there was a great famine. People jealously hoarded whatever food they could find, hiding it even from their friends and neighbors. One day a peddler drove his wagon into a village, sold a few of his wares, and began asking questions as if he planned to stay for the night.

"There's not a bite to eat in the whole province", he was told. "Better keep moving on."

"Oh, I have everything I need", he said. "In fact, I was thinking of making some stone soup to share with all of you." He pulled an iron cauldron from his wagon, filled it with water, and built a fire under it. Then, with great ceremony, he drew an ordinary-looking stone from a velvet bag and dropped it into the water.

By now, hearing the rumor of food, most of the villagers had come to the square or watched from their windows. As the peddler sniffed the "broth" and licked his lips in anticipation, hunger began to overcome their skepticism.

"Ahh", the peddler said to himself rather loudly, "I do like a tasty stone soup. Of course, stone soup with cabbage — that's hard to beat."

Soon a villager approached hesitantly, holding a cabbage he'd retrieved from its hiding place, and added it to the pot. "Capital!" cried the peddler. "You know, I once had stone soup with cabbage and a bit of salt beef as well, and it was fit for a king."

The village butcher managed to find some salt beef…and so it went, through potatoes, onions, carrots, mushrooms, and so on, until there was indeed a delicious meal for all. The villagers offered the peddler a great deal of money for the magic stone, but he refused to sell and traveled on the next day. And from that time on, long after the famine had ended, they reminisced about the finest soup they'd ever had.

```
***
```

That's the way Id has grown, with quite a bit of magic, although without the element of deception. (You don't have to deceive programmers to make them think that hours of painstaking, often frustrating work is fun… they do it to themselves.)

It wouldn't have happened, of course, without Benoit Mandelbrot and the explosion of interest in fractal graphics that has grown from his work at IBM. Or without the example of other Mandelplotters for the PC. Or without those wizards who first realized you could perform Mandelbrot calculations using integer math (it wasn't us - we just recognize good algorithms when we steal—uhh—see them). Or those graphics experts who hang around the CompuServe PICS forum and keep adding video modes to the program. Or…

A Word About the Authors

Id's ancestor, Fractint 20.04, is the result of a synergy between the main authors, many contributors, and published sources. All of the main authors have had a hand in many aspects of the code. However, each author has certain areas of greater contribution and creativity. Since there is not room in the credits screen for the contributions of the main authors, we list them here to further elaborate on their contributions.

Main Author of Iterated Dynamics

Richard Thomson is the main author of Id. The Fractint code had languished for many years, struggling to leave its DOS nest. This has resulted other fractal renderers passing it by as they adopted SIMD parallelism, GPU computation and fancy perturbation methods to eclipse its computational speed. Richard felt the investment in so many fractal types, and the extensive online help, couldn't just be left to rot and the code needed to move forward to a modern environment. Particularly the IFS, L-system and cellular automata types deserve to have a modern home as they are often omitted by other fractal rendering programs.

Richard forked the code from Subversion to the github repository. There are barely any lines of code that weren't touched by him after the fork from Fractint.

Richard decoupled the DOS and assembly language code from the rest of the code and created a driver abstraction that could be used to connect the code to modern environments. He implemented the disk, Win32 GDI, and X11 drivers used to provide their corresponding video modes. The CGA text window is emulated in order to have the user interface and help system. He converted the entirety of the code to C++ to allow object oriented programming paradigms to be used.

Many magic numbers throughout the code were replaced with enums or other symbolic names to make it easier to understand how these values and flags are used throughout. Many times an integer quantity was being used as a boolean on/off switch and those integer variables were replaced with boolean variables to make the intent more clear.

Over the years, the code has accumulated a large number of global variables through which the various functions communicate. Global variables have been renamed to start with a g_ prefix and variables with a file scope, e.g. static variables, were renamed to start with an s_ prefix. This makes it easier to understand the scope of variables when reading a particular piece of code.

Functions and variables were placed together into source files in order to suit the cramped memory limitations of DOS using overlays. Sometimes you have to break things down into isolated pieces before you can begin to see which ones are intimately connected to each other. For the largest source files containing multiple functions, the individual functions were split into their own source and header files.

He implemented the CMake based build system from scratch. He used the vcpkg package manager for C++ to allow Id to easily consume third party libraries for new features. One of the first libraries used was Google Test, bringing unit testing to the code for the first time. Making changes to the code requires careful attention to detail in how all these variables interact with each other. Unit testing changes before applying them helps ensure that forward progress doesn't come at the expense of bugs.

Github's actions provide continuous integration (CI) builds on Windows, Linux and macOS with every commit to the repository. The actions build all the code and run the unit and acceptance tests every time the repository is updated. Richard uses Windows as his primary development environment and the CI builds ensure that the code continues to build and run on Linux and macOS. (This trips you up more often than you might think!)

Other libraries Richard has brought in are the boost.algorithm library for some string algorithms, the boost.endian library for marshalling the GIF extension blocks between little and big endian machines and the giflib library for reading and writing GIF files.

The 2017 version of the ISO C++ language standard is required to build Id. This is readily available through open source compilers on Unix- like systems, such as gcc and clang, and on Windows through Visual Studio Community Edition. Visual Studio Community Edition is a full- featured compiler and development environment with no license restrictions. It can be downloaded from https://VisualStudio.com.

Every line of the help files were examined and updated to have a consistent style throughout. Discussion of old DOS related information was removed and, where appropriate, replaced with relevant discussion of operating in a Windowed environment. Stale links to web sites have been removed. The original help came from a more innocent time on the internet when people weren't afraid to post their phone numbers and home addresses. That personal information has been removed from the help files and only Richard's personal email address has been left.

Richard has a Bachelor of Science degree in Electrical Engineering from the University of Delaware and a Master of Philosophy degree in Computer Science from the University of Utah. From 2000 to 2010, he was a Microsoft MVP for Direct3D and wrote a book on the Direct3D APIs. The book, "The

Direct3D Graphics Pipeline", is available free download at https://legalizeadulthood.wordpress.com/the-direct3d-graphics-pipeline/

He has worked as a software engineer for his entire career on embedded systems, 3D graphics workstations, 3D paint software, digital video editing software, system management software, home automation software, 3D content creation software, solid-state storage management software and currently works on hardware accelerated ray tracing.

Main Authors of Fractint

Bert Tyler is the original author of Fractint. He wrote the "blindingly fast" 386-specific 32 bit integer math code and the original video mode logic. Bert made Stone Soup possible, and provides a sense of direction when we need it. His forte is writing fast 80x86 assembler, his knowledge of a variety of video hardware, and his skill at hacking up the code we send him!

Bert has a BA in mathematics from Cornell University. He has been in programming since he got a job at the computer center in his sophomore year at college - in other words, he hasn't done an honest day's work in his life. He has been known to pass himself off as a PC expert, a Unix expert, a statistician, and even a financial modeling expert. He is currently masquerading as an independent PC consultant, supporting the PC-to-Mainframe communications environment at NIH. If you sent mail from the Internet to an NIH staffer on his 3+Mail system, it was probably Bert's code that mangled it during the Internet-to-3+Mail conversion. He also claims to support the MS-Kermit environment at NIH. Fractint is Bert's first effort at building a graphics program.

Tim Wegner contributed the original implementation of palette animation, and is responsible for most of the 3D mechanisms. He provided the main outlines of the "Standard Fractal" engine and data structures, and is accused by his cohorts of being "obsessed with options". One of Tim's main interests is the use of four dimensional algebras to produce fractals. Tim served as team coordinator for version 19, and integrated Wes Loewer's arbitrary precision library into Fractint.

Tim has BA and MA degrees in mathematics from Carleton College and the University of California Berkeley. He worked for 7 years overseas as a volunteer, doing things like working with Egyptian villagers building water systems. Since returning to the US in 1982, he has written shuttle navigation software, a software support environment prototype, and supported strategic information planning, all at NASA's Johnson Space Center. After a two-year stint at full-time writing, he's back at NASA developing shuttle navigation software.

Jonathan Osuch started throwing pebbles into the soup around version 15.0 with a method for simulating an if-then-else structure using the formula parser. He has contributed the fn||fn fractal types, the built- in bailout tests, the increase in both the maximum iteration count and bailout value, and bug fixes too numerous to count. Jonathan worked closely with Robin Bussell to implement Robin's browser mechanism in Fractint.

Jonathan has a B.S. in Physics from the University of Dubuque and a B.S. in Computer Science from Mount Mercy College, both in Iowa. He is currently working as a consultant in the nuclear power industry.

Wes Loewer first got his foot in the Stone Soup door by writing fast floating point assembler routines for Mandelbrot, Julia, and Lyapunov fractals. He also rewrote the boundary trace

algorithms and added the frothybasin fractal. His most significant contribution is the addition of the arbitrary precision library which allows Fractint to perform incredibly deep zooms.

Wes has a B.S. in Physics from Wheaton College in Illinois. He also holds an M.S. in Physics and an M.Ed. in Education from Texas A&M University. Wes teaches physics and math at McCullough High School in The Woodlands, Texas where his pupils inspire him to keep that sense of amazement that students get when they understand a physical or mathematical principle for the first time. Since he uses Fractint to help teach certain mathematical principles, he's one of the few folks who actually gets to use Fractint on the job. Besides his involvement with Fractint, Wes is the author of WL-Plot, an equation graphing program, and MatCalc, a matrix calculator program.

George Martin first became known to Fractint users when he brought a modicum of order to the chaotic world of formula postings with his release of the Orgform program and formula compilation. George added if..else to the formula parser language for version 19.6. Among his other contributions are the ability to include formula, ifs, and lsystem entries in .par files, the scrolling of text in the `Z` and `F2` screens, and new autokey commands.

George received an A.B. in Economics from Dartmouth College and a J.D from the University of Michigan. When not playing with Fractint, he practices law in a small village about 40 miles northwest of Detroit.

Robin Bussell began contributing to Fractint in rudimentary fashion with the autologmap routine and has been producing more and more complex interface enhancements as he gets better at what he refers to as 'this C programming lark'. He is always grateful for the help the rest of the team have given in smoothing the rough edges of the ingredients he adds to the soup and regards the evolver feature as his greatest achievement to date.

Robin had far too much fun at college in London to actually get any qualifications there and has since worked his way up from a workshop job fixing computers back in the final days of CP/M, via some interesting work with Transputers (an innovative British cpu that was designed to run in massively parallel configurations, and made a very good Mandlebrot set calculating machine when a few dozen or more were set to the task), through to his position of senior engineer for a third party support company where he spends his time traveling the south west of Britain sorting out peoples IT problems.

When not playing with computers Robin likes to relax by experimenting with kite powered traction and can often be found hurtling around the local beaches on the end of a few square metres of fabric and carbon fibre in various configurations.

Distribution of Iterated Dynamics

New versions of Iterated Dynamics are released through github, https://github.com/LegalizeAdulthood/iterated-dynamics, and make their way to other systems from that point. Id is available for Windows, Linux and macOS. For Windows, you have a choice between a Setup program, a Windows MSI installer, or a ZIP file containing binaries. Linux and macOS users are provided with a tar.gz package. Since Id is open source, you can always obtain the source code from github and follow the instructions in the ReadMe for building. While source archives are provided automatically by github on the releases page, it is recommended that you clone the source code from github directly.

The latest version of the code can always be found on the github repository.

Contacting the Author

Communication with the author for development of the next version of Iterated Dynamics takes place through the github repository, https://github.com/LegalizeAdulthood/iterated-dynamics. From there you can file bugs, feature requests and so-on.

The current main author is Richard Thomson, who is the maintainer of the github repository. You can email him directly at legalize@xmission.com, but it is best to use github.

GIF Save File Format

The `S`ave command stores screen images as GIF (Graphics Interchange Format) files. The `R`estore command loads a GIF image and reads extra information from the file to restore all the fractal parameters used to generate the image. This GIF89a version of the format allows the placement of application data within the file in a standard manner.

GIF was originally created by CompuServe, but it used a compression algorithm patented by Unisys in 1985. This led to patent licensing issues with the file format, but all patents relating to the proprietary compression used for GIF have expired as of 2004.

Other Fractal Programs

Capow by Rudy Rucker is a Windows program for exploring continuous valued cellular automata. Rudy's original source code is on github here: https://github.com/rudyrucker/capow and a version that builds with current editions of Visual Studio (as of April, 2024) is here: https://github.com/legalizeadulthood/capow

Kalles-Fraktaler is a fractal renderer with SIMD and multithreading support. Download from https://fraktaler.mathr.co.uk/

ManpWIN is a Windows fractal generator derived from DOS FRACTINT with lots of additional features by Paul de Leeuw. You can download the latest release from his web site: https://www.deleeuw.com.au/; Paul has collaborated with Richard Thomson to put his source code on github at https://github.com/PaulTheLionHeart/manpwin.

Movie Maker 3D by Yann Le Bihan can create animations using frames rendered by Kalles-Fraktaler. http://www.chillheimer.de/kallesfraktaler/mmy/

XaoS is a program that specializes in fast, fluid, continuous zooming. https://xaos-project.github.io/

Bibliography

Barnsley, Michael: "Fractals Everywhere", Academic Press, 1988.

Davenport, Clyde: "A Hypercomplex Calculus with Applications to Relativity", ISBN 0-9623837-0-8.

This self-published expansion of Mr. Davenport's Master's thesis makes the case for using hypercomplex numbers rather than quaternions. This book provided the background for Id's implementation of hypercomplex fractals.

Dewdney, A. K., "Computer Recreations" columns in "Scientific American" — 8/85, 7/87, 11/87, 12/88, 7/89.

Devaney, Robert L., "An Introduction to Chaotic Dynamical Systems", Addison-Wesley, 1989. A detailed mathematical treatment of chaotic dynamical systems, assuming only a knowledge of calculus as a starting point.

Edgar, Gerald A.: "Classics on Fractals", Addison-Wesley, 1993. Collects papers related to fractals and the idea of fractional dimension that appeared before Benoit Mandelbrot first coined the term fractal in 1975.

Falconer, Kenneth: "Fractal Geometry", Wiley, 1990. Detailed treatement of the underlying mathematics of fractals, including excursions into topology and measure theory.

Feder, Jens: "Fractals", Plenum, 1988. Quite technical, with good coverage of applications in fluid percolation, game theory, and other areas.

Field, Michael and Golubitsky, Martin, "Symmetry in Chaos", Oxford University Press, 1992. A well illustrated book that introduced the "icon" fractal type.

Gleick, James: "Chaos: Making a New Science", Viking Press, 1987. The best non-technical account of the revolution in our understanding of dynamical systems and its connections with fractal geometry.

Mandelbrot, Benoit: "The Fractal Geometry of Nature", W. H. Freeman & Co., 1982. An even more revised and expanded version of the 1977 work. A rich and sometimes confusing stew of formal and informal mathematics, the prehistory of fractal geometry, and everything else. Best taken in small doses.

Mandelbrot, Benoit: "Fractals: Form, Chance, and Dimension", W. H. Freeman & Co., 1977. A much revised translation of "Les objets fractals: forme, hasard, et dimension", Flammarion, 1975.

Peitgen, Heinz-Otto & Richter, Peter: "The Beauty of Fractals", Springer-Verlag, 1986. THE coffee-table book of fractal images, knowledgeable on computer graphics as well as the mathematics they portray.

Peitgen, Heinz-Otto & Saupe, Ditmar: "The Science of Fractal Images", Springer-Verlag, 1988. A fantastic work, with a few nice pictures, but mostly filled with **equations**!!!

Peitgen, Heinz-Otto, Juergens, Hartmut & Saupe, Dietmar: "Chaos and Fractals: New Frontiers of Science", Springer-Verlag, 1992.

Pickover, Clifford: "Computers, Pattern, Chaos, and Beauty", St. Martin's Press, 1990.

Schroeder, Manfred: "Fractals, Chaos, Power Laws", W. H. Freeman & Co., 1991.

Sprott, Julien C., "Strange Attractors", M & T Books, 1993. This is available online as a PDF:

https://sprott.physics.wisc.edu/fractals/booktext/SABOOK.PDF

Thompson, J. M. T. and Stewart, H. B.: "Nonlinear Dynamics and Chaos", Wiley, 1986. Detailed discussion of nonlinear dynamical systems, bifurcation points and attractors. Discusses the Henon map, Rossler's band and the Lorenz attractor.

Viscek, Tamas: "Fractal Growth Phenomena", World Scientific, 1989. Discusses diffusion-limited aggregation, cluster-cluster aggregation and other models of growth of structures and their fractal properties.

Revision History

Iterated Dynamics is a fork of the DOS program Fractint. Fractint has its own history spanning many years. Please consult the Fractint help for its history. To avoid being confused with a new release of Fractint, the code has been rebranded Iterated Dynamics, or Id for short, throughout.

Version 1.4.2 is a bug fix release for version 1.4.0.

Bugs Fixes in Version 1.4.2

- The image random number maximum constant no longer conflicts with system headers that define RANDOM_MAX. (Fixes #433)
- Runtime dependency scanning is only used when installing Windows builds. (Fixes #434)

Version 1.4.1 is a bug fix release for version 1.4.0.

Bugs Fixed in Version 1.4.1

- The "3D" Commands help topic now renders correctly in HTML documentation.
- Linux release binaries are now built on Ubuntu 22.04 so they run on distributions with older glibc, such as Debian Bookworm.

Version 1.4.0 is an interim release on the way to version 2.0.

New Features in Version 1.4.0

- The Linux port is complete. It provides native Linux text and graphics windows and uses the standard id.cfg video mode table.
- The macOS port is complete. See Id on Linux and macOS. (Fixes #42)
- Libraries are used to group data files for Id. See Libraries. (Fixes #95)
- The documentation is now available in PDF format. (Fixes #262)
- The raytrace output format can now be specified by name instead of just by number. (Fixes #318)
- The DXF raytrace output format is documented in the help file. (Fixes #317)
- The previous browse history limit of 16 files was relaxed; it is now limited only by available memory.

- Random-dot stereogram generation now honors rseed=, making RDS output deterministic when a seed is specified.
- Random-dot stereograms can now be generated in batch mode using rds= and rds-texture=. (Fixes #227)
- Formula rand now honors rseed=, making formulas that use rand repeatable when a seed is specified. (Fixes #363)
- Formula comparison operators now document that they compare only the real parts of operands. (Fixes #358)
- The Burning Ship fractal type now documents its formula and parameters. (Fixes #426) o makepar now writes legacy color ranges when saving parameters from legacy GIF images. (Fixes #428)
- Fractal types using random numbers now consistently honor rseed=, making seeded random fractals repeatable across fractal types.
- Formula, IFS, and L-system item names now preserve user-entered case in parameter output while still matching entries case-insensitively. (Fixes #364)
- Image tests now cover periodicity checking with non-default settings. (Fixes #354)
- The bibliography now includes "Chaos and Fractals: New Frontiers of Science". (Fixes #365)
- Online HTML documentation is now published to versioned permalink URLs before approval for the latest URL. (Fixes #366)
- Autokey files can now use caret notation for control-key command initiators, such as ^S for `Ctrl` + `S`.
- MIG piece generation now writes a makemig.sh shell script on Linux and macOS. (Fixes #370)
- Disk-video modes can now generate GIF-sized images up to 65535x65535. Windowed display modes still require a physical display large enough for the configured mode.

Bugs Fixed in Version 1.4.0

- Doxygen pages are now being generated from the source code. Documentation comments will be added over time to improve the generated documentation. (Fixes #263)
- @parfile functionality restored. (It processes the first parameter set in the file.)
- List of fractal types in the repository ReadMe linked and condensed. (Fixes #332)
- Solid guessing no longer leaves parts of images unfinished when symmetry splits work, such as in 640x480 video mode. (Fixes #257)
- Solid guessing no longer crashes in disk-video modes wider than 2048 pixels, such as the 8K disk-video mode.
- Lambdafn with function=exp now honors the reset version when selecting the Fractint-compatible bailout behavior. (Fixes #107)
- Legacy parameter sets now use the Fractint-compatible logmap table and outside coloring behavior selected by reset. (Fixes #104)
- FrothyBasin now honors legacy Fractint parameter semantics selected by reset, allowing @id.par/EvilFrog to render correctly. (Fixes #105)

- The @id.par/mandala parameter set now explicitly selects reset=1730 so legacy outside=real coloring is used. (Fixes #106)
- The @example1.par/fireworks parameter set was removed because its appearance depends on Fractint fixed-point artifacts. (Fixes #391)
- Ant rule strings from params= are now preserved so @example2.par/Ant renders with the intended rule. (Fixes #393)
- Ant rules can now use exact integer text in params=, allowing rule strings wider than Fractint's numeric formatting permits.
- Legacy outside=atan coloring now uses the Fractint-compatible color scaling selected by reset. (Fixes #394)
- @example2.par/Barnsleyj2-manr now selects the Fractint reset version needed for its logmap=old coloring. (Fixes #395)
- The default inside color now matches Fractint, allowing @example2.par/TileMandel to render correctly. (Fixes #399)
- Legacy 6-bit palette values are now expanded to the full 8-bit range when loading old parameter colors, affected Id GIF files, and legacy map files.
- Parameter colors=@name now finds name.map when the extension is omitted.
- @lyapunov.par/Blob1 now renders correctly by restoring the legacy Lyapunov reset behavior used by Fractint. (Fixes #400)
- Compatibility version handling now follows Fractint's model: `Insert` restores the current Id version, bare reset selects Fractint 17.30 behavior, and explicit reset= values preserve old parameter behavior.
- Included parameter sets now render consistently with the resolved Fractint compatibility expectations. (Fixes #117)
- Setting type=formula, type=lsystem, or type=ifs on the command line without specifying an item name now behaves like selecting the type interactively. (Fixes #71)
- The function= parameter documentation now lists the default variable functions. (Fixes #390)
- More mathematical formulas now render correctly in the AsciiDoc documentation.
- Debug output files are now written under the debug subdirectory of the save library. New debug files honor overwrite=no by choosing an incremented name, while append-mode debug logs keep appending to the same file.
- Scratch files now use the temporary directory instead of the working directory.
- makepar= now writes PAR files correctly when Id is launched from a directory other than the home library. (Fixes #368)
- Library searches now accept FRACTINT-style formulas, maps, and pars subdirectories as read-only aliases. (Fixes #421)
- Formula entries embedded in a selected PAR file now load when formulafile= names a missing formula file, matching Fractint fallback behavior. (Fixes #389)
- MIG image component files are now removed from the image save library after the final image is assembled.

- The Linux executable is now named xid to avoid conflicting with the standard id system utility.
- Windows executables now embed icon and version resources directly. (Fixes #386)
- 3D Targa output now writes rendered pixels to the image save library.
- The bignum storage arena now matches the 64 KB allocator limit, avoiding memory corruption in arbitrary precision math.
- sound=off now suppresses completion buzzers.
- Restore orbitdelay= paced timing for orbit sound (Fixes #325).

Code Cleanup in Version 1.4.0

- Many small changes to avoid buffer overruns or other unbounded string operations as reported by SonarQube.
- The incomplete X11 driver for linux has been removed. (Fixes #343)
- All identifiers are now inside a namespace to avoid collisions with external libraries. wxWidgets on linux depends on glib which uses g_ prefix for all identifiers and this conflicted with the same prefix used by Id to identify all global variables. (Fixes #330)
- File output comparison tests were added for raytrace output and orbit output to detect major regressions in the formatted output.
- An autokey-driven UI test was added for generating MIG PAR and BAT files.
- Help text spelling was normalized to American English.
- More image comparison tests were added to cover more fractal types, rotation and skew of the zoom box and the inversion, biomorph and decomp options. (Fixes #335, #336, #337, #341, #342)
- The code has been restructured to increase the separation of image computation from user interface.

Known limitations:

- This initial implementation of perturbation does not support periodicity checking and only supports a limited number of inside and outside coloring algorithms. Suspend and resume is not supported with perturbation.
- Sound output does nothing, although all controls can be set through parameters or interactive screens. Sound output will be restored once a suitable audio library has been identified.
- Some formulas don’t render correctly. These problems have been around forever as xfractint also renders them incorrectly. This will be fixed in a subsequent release.

Version 1.3.2 is an interim bug fix release on the way to version 2.0.

Bugs Fixed in Version 1.3.2

- Interactively changing key assignments in the select video mode screen works again (Fixes #383)
- When curdir=no (the default), locate id.cfg from the user’s save directory before looking for id.cfg in the current directory. (Fixes #384)

- On Windows, advertise Id as **not** high DPI aware. (Fixes #385)

New Features in Version 1.3.1

- Colormap channels are no longer restricted to 6 bits. Encoded colors when generating parameter sets are now extended to encode 8-bit color channels values without loss of precision. When reading previous parameter sets that encoded 6-bit values the value is loaded into the 6 most significant bits of the 8-bit color channel value. Colors written to image files use the full 8-bit colors. (Fixes #47, #61)
- The palette editor was updated to work with 8-bit colors. (Fixes #306)

Bugs Fixed in Version 1.3.1

- Displaying the view window parameter screen could crash. (Fixes #308)
- Periodic automatic image save, activated by the savetime parameter, has been restored. (Fixes #43)
- Some Fractal of the Day image regressions have been fixed. (Fixes #99, #103)
- Issue templates for bug reports and feature requests have been added to the github repository. (Fixes #265)
- The help compiler documentation has been overhauled and updated for the AsciiDoc output features. (Fixes #276)
- Reset and `Insert` now reset "inside" color consistently to zero. (Fixes #279)
- Improvements were made to the continuous integration build and test mechanisms on github. (Fixes #280, #293)
- Code configuration was generalized to test for features and not be based on the platform name to enable building on FreeBSD. (Fixes #292)
- When log(z) stores the output in the same variable as the input, it no longer corrupts the output variable. (Fixes #295)
- The float parameter has been deprecated and is ignored. Discussion of integer math removed from the documentation. (Fixes #303)
- Clarify operator precedence and line continuation in the documentation for type=formula. (Fixes #304)
- Descriptions of the fractal types are now arranged alphabetically in the plain text and HTML documentation files, making it easier to find information on a specific fractal type. (Fixes #305)
- The first value to the mathtolerance parameter is ignored; it previously was used to control the transition from integer to floating-point math. When writing parameter sets, the first value is now omitted.
- The delay value is now properly displayed for type=ant. (Fixes #387)

Code Cleanup in Version 1.3.1

- More unused integer math routines were removed.
- Functions used to initialize the FractalSpecific array were renamed to remove floating-point

annotations and match the FractalSpecific member name (e.g. orbit, per_pixel, per_image, calc_type).

Version 1.2

Version 1.2 is mostly a code cleanup release that removes integer fractal types and replaces them exclusively with floating-point types. This change was made to simplify the existing code paths in preparation for migration to wxWidgets in version 2.0.

Many thanks to Paul de Leeuw in helping us find a bunch of bugs that we fixed for this release!

New Features in Version 1.2

- Native Windows file browse dialog for opening files.
- Improved version handling with the reset parameter.
- Improved version handling when reading GIF files.
- Improved version handling when writing GIF files.
- Improved rendering of mathematics in the HTML help.

When prompting the user to locate a file, Id now uses the native Windows Open File dialog. This replaces a bunch of homebrew code for locating a file under DOS using the full screen choice mechanism on the text screens. There are likely more filesystem related issues that need to be cleaned up; see issue #268.

Version Handling in Version 1.2 Onward

Previous versions of Iterated Dynamics reset the version to 1.0 in a breaking change from the legacy DOS FRACTINT version numbering. While this properly served to indicate the reality of Id being a fork from DOS FRACTINT, it created some interopability problems with parameter sets and image files between the two programs.

When creating a parameter file, the reset parameter is now written as reset=major/minor/patch/tweak. The major and minor version components are always written. The patch and tweak components appear if either component is non-zero. Legacy FRACTINT version numbers, such as reset=2004, when read from a parameter set are all treated as version 1.0 of Id for the purposes of backward compatibility.

Currently, no backwards compatibility with old FRACTINT behavior is applied to parameter sets. Legacy DOS FRACTINT parameter sets that rely on the reset parameter to have behavior adjusted may not render identical images in Id. Such parameter sets can be upgraded by loading the parameter set into DOS FRACTINT and either saving an image or creating a new parameter set in DOS FRACTINT.

When reading an image file, integer fractal types are converted to their corresponding floating-point fractal type. Legacy FRACTINT version numbers in image files are used to migrate legacy FRACTINT behavior to the current behavior in Id.

When writing an image file, Id writes the full major/minor/patch/tweak version number into the file and writes a legacy FRACTINT version number of 20.04. Such files may load in DOS FRACTINT,

but no guarantees of compatibility with DOS FRACTINT are made.

Bugs Fixed in Version 1.2

- Id will search standard locations for images specified on the command- line: current directory, savedir parameter location, and FRACTDIR environment variable location. (Fixes #267)
- Id could overwrite image files in separate sessions, even when the overwrite parameter was not set. Id now properly checks for the existence of a file in the save location. (Fixes #264)
- File system errors are now properly reported when writing image parameters. (Fixes #231)
- The orbitname parameter for julibrot types is now documented. (Fixes #238)
- The chip, quadruptwo and threeply types documentation was improved. (Fixes #252)
- Versions 1.0, 1.1 and 1.1.1 wrote a version number into image files that was not compatible with loading into legacy DOS FRACTINT. That has been corrected.

Code Cleanup in Version 1.2

- All integer fractal code paths and associated data have been removed.
- A bunch of legacy FRACTINT files were removed from the repository. The remaining files were moved to a directory named 'legacy'.
- Leftover files from the Sphinx version of HTML help have been removed.
- More magic value integers were replaced with symbolic constants as enumerations.

New Features in Version 1.1:

- Ctrl `+` and `-` can now be used to rotate the zoom box, in addition to the numeric keypad `+` and `-` keys. Some laptops don't have a numeric keypad.
- Perturbation drawing algorithm. Paul de Leeuw contributed an implementation of the perturbation drawing algorithm with support for the Mandelbrot (type=mandel) and Mandelbrot power (type=manzpower) fractal types. See Drawing Method and Perturbation Algorithm for details. (PR #160)
- Configuration files 'sstools.ini' and 'id.cfg' can now be loaded from outside the installation directory. The following locations are searched in this order: user's Documents directory, FRACTDIR environment variable location and then current directory. The directory related parameter values do not change this search order. When video mode key assignments are changed interactively, the updated id.cfg is written to the location specified by the savedir parameter, defaulting to the user's Documents directory. Changing function key assignments while selecting video modes saves the updated id.cfg file to the savedir, defaulting to the user's Documents directory. (Fixes #170)
- Mouse support for the zoom box has been restored. (Fixes #14, #152)
- Mouse support for the Orbits Window has been restored.
- Mouse support for the inverse Julia set cursor has been restored. (Fixes #151)
- The color palette editor can be positioned using the mouse and the current pixel whose color is displayed in the editor can also be selected with the mouse. (Fixes #150)

- Disk video rendering performance and robustness improved. Requested memory buffers are now unlimited in disk video mode and only fall back to using a disk file if memory allocation fails. The operating system's virtual memory will perform better than manual disk caches. Disk cache files are unique per-process, allowing multiple instances of Id to be running simultaneously without interfering with each other. The the status screen will now update at most ten times per second. This prevents GUI updates from being the bottleneck on disk video rendering. (Fixes #173, #174)

Bugs Fixed in Version 1.1:

- When selecting a formula, L-system or IFS fractal type and the item is wide and/or needs scrolling, such as the Fractint entry of example2.frm, the surrounding box was improperly displayed. (Fixes #182)
- The makepar parameter, or `B` commnd, now saves any generated parameter and batch files to the savedir location. (Fixes #176, #177)
- Generated parameter file comments now properly handle escaped backslashes preceding another special character. (Fixes #175)
- The version number is now correctly displayed in the title of help screens. (Fixes #154)
- When run from the Visual Studio debugger, the working directory of the program is properly set to the home directory. (Fixes #153)
- Corrected an assert popup during a Debug build. (Fixes #147, #140)
- When the monitor configuration changed since the last time Id was run and the previous window position is no longer on-screen, the previous window position is forgotten and the window is placed by the system. (Fixes #136)
- On Windows, crash dump files are saved to the savedir location. (Fixes #133)
- When changing fractal types, if the the fractal parameters screen or entry name screen (for formula, ifs or L-system fractal types) were canceled, the previous fractal type and associated parameters were lost. Now, canceling those screens restores the previous state. (Fixes #129)

Code Cleanup in Version 1.1:

- Code style guide created to assist contributors. (Fixes #166)
- More unit tests added, when reasonable, to prove buggy behavior and validate their fixes.
- Library libid source and header files organized into subdirectories by role: 3d, engine, fractals, io, math, misc and ui.
- test-id unit tests files organized into subdirectories, matching the function or file they test.
- Identifier names renamed to conform to style guide.
- Names of files updated to conform to style guide.
- Calls to standard library functions qualified with namespace std to conform to style guide.
- All platform (e.g. linux vs. Windows) variations isolated to CMake checks in the config library.
- All calls to stricmp, strnicmp, and strncasecmp replaced with string_case_compare and string_case_equal from config library.

- Platform driver types selected by config library in driver_types.h.
- Platform file descriptor I/O header selected by config library in fdio.h.
- Platform filelength function selected by config library in filelength.h.
- Platform getpid function selected by config library in getpid.h.
- Platform path limits selected by config library in path_limits.h.
- All XFRACT conditionally compiled code eliminated by using config library.
- Many more small improvements, see the git commit history for details.

Version 1.0

- Fork of Fractint 20.04 patch level 4
- New formula file MandelbrotDerivatives.frm with Burning Ship, Perpendicular Mandelbrot, Heart Mandelbrot, Mandelbar and other Mandelbrot set formula variations. The parameter file BurningShip.par provides example parameters for these variations. Contributed by Paul de Leeuw.
- New cpu comment variable expands to a description of your CPU.
- Long filenames are now supported throughout.
- All filename parameters are now case sensitive.
- When browsing for files, e.g. `R`estoring *.GIF files, filenames are compared case insensitive against wildcard search masks.
- Added new fractal types Mandelbrot Mix 4 and DivideBrot5 by Jim Muth. DivideBrot5 has arbitrary precision (AP) math support. Note that image skewing and inversion are not supported by the AP math routines. See Arbitrary Precision and Deep Zooming for other features not supported.
- The orbitsavename parameter was added to specify the name of the raw orbits file. The orbits file, if it exists, is overwritten when overwrite=yes. Otherwise, the value of orbitsavename is incremented until a filename is formed that does not exist.
- The autokeyname parameter was enhanced to create a unique filename when autokey=record unless overwrite=yes is specified.
- Disk video buffer sizes were dramatically increased.
- The formula parser has been enhanced to remove internal size limitations. The only limit on formula parsing is available memory.
- The savedir parameter was added to specify the default directory used for writing any files. Since a Windows program is typically installed to the Program Files directory, which is read-only, it cannot be assumed that Id can write to the directory in which it is located.

Additional changes in Version 1.0:

- Most of the changes involve internal restructuring of the DOS Fractint code to build a bridge to the future.
- Iterated Dynamics is a native 64-bit Windows program written in C++ with a CMake build system and vcpkg for obtaining 3rd party library dependencies.

- All assembly language code has been removed and, where practical, replaced with C++ code that performs the same function.
- A CGA text display is simulated for all prompt screens and help screens.
- A "video mode" determines the resolution of all saved images and is provided by a driver. Iterated Dynamics comes with two drivers: Disk and GDI. The GDI driver renders to the screen through the Windows GDI interface. The disk driver can be used to render images larger than your physical screen, but still uses your screen's GUI for displaying status with a CGA style window.
- Id.cfg, which defines the default video modes, was updated to remove all modes smaller than 800x600 and very large disk video modes were added.
- DOS related information in video modes was dropped.
- All online help text has been reviewed and revised to drop DOS specific discussions. Where still appropriate the text has been updated to reflect that Id is a program operating in a windowed environment.
- The release parameter resets all values to their default but does not enable any backwards compatible behavior in this release. For backward compatibility with old behavior, use Fractint.
- The monitorwidth parameter was renamed to stereowidth. The old parameter name is still accepted but will be deprecated in a future release.
- Printing an image has been removed. Use your operating system for that. :)
- The following parameter file commands are deprecated and support will be removed in a future release. In this release the arguments are parsed and validated and the results ignored. adapter halftone ramvideo afi iterincr rleps biospalette linefeed textsafe colorps maxcolorres title comport noninterlaced tplus epsf pixelzoom translate exitmode plotstyle tweaklzw fpu printer vesadetect gif87a printfile